Relazione finale
ACDC_Supermercato

Carmona Contreras Davide Anthony

Dentella Chiara

Frassi Alessandro

Tremolada Chiara

Indice

1 PROJECT PLAN

1.1 Software Engineering Management

1.1.1 Introduzione

1.1.2 Modello di processo

1.1.3 Organizzazione del progetto

1.1.4 Standard, linee guida, procedure

1.1.5 Attività di gestione

1.1.6 Rischi

1.1.7 Personale

1.1.8 Metodi e tecniche

1.1.8.1 Ingegneria dei requisiti

1.1.8.2 Progettazione

1.1.8.3 Implementazione

1.1.8.4 Test

1.1.8.5 Gestione della documentazione tecnica

1.1.8.6 Controllo di versione e configurazione

1.1.9 Garanzia di qualità

1.1.10 Risorse

1.1.11 Budget e schedule

1.1.12 Cambiamenti

1.1.13 Consegna

1.1.14 Cronologia delle versioni

2 GESTIONE DEL PROGETTO

2.1 Software Life Cycle

2.1.1 Processo di sviluppo adottato

2.1.2 Ruoli nel team

2.1.3 Struttura degli sprint

2.1.4 Pianificazione degli sprint

2.1.5 Approccio MDA con Papyrus

2.2 Configuration Management

2.2.1 Introduzione

2.2.2 Strumenti e regole adottate

2.2.3 Branching e pull request

2.2.4 Issue e project Kanban board

2.2.5 Commit e convenzioni

2.3 People Management and Team Organization

2.3.1 Modello organizzativo

2.3.2 Ruoli e responsabilità

2.3.3 Eventi e comunicazione

3 REQUISITI

3.1 Requirements Engineering

3.1.1 Introduzione

3.1.1.1 Obiettivo

3.1.1.2 Scopo e ambito

3.1.1.3 Definizioni, acronimi e abbreviazioni

3.1.1.4 Riferimenti

3.1.1.5 Panoramica del documento

3.1.1.6 Elicitazione dei requisiti

3.1.2 Descrizione generale

3.1.2.1 Prospettiva del prodotto

3.1.2.2 Funzioni del prodotto

3.1.2.3 Caratteristiche dell'utente

3.1.2.4 Vincoli

3.1.2.5 Presupposti e dipendenze

3.1.3 Requisiti specifici

3.1.3.1 Requisiti dell'interfaccia esterna

3.1.3.2 Requisiti funzionali

3.1.3.2.1 Funzioni dei clienti

3.1.3.2.2 Funzioni dei dipendenti

3.1.3.3 Requisiti di prestazione

3.1.3.4 Vincoli di progettazione

3.1.3.5 Attributi del software

3.1.3.6 Altri requisiti

3.1.3.7 Tracciabilità dei requisiti

3.1.3.8 Casi d'uso

3.1.4 Codice PlantUML dei diagrammi dei casi d'uso

3.2 Software Quality

3.2.1 Correttezza

3.2.2 Affidabilità

3.2.3 Usabilità

3.2.4 Manutenibilità

3.2.5 Testabilità

3.2.6 Portabilità

3.2.7 Integrità

4 DESIGN

4.1 Modelling

4.1.1 Introduzione

4.1.2 Regole di business rappresentate

4.1.3 Diagramma della macchina a stati

4.1.4 Diagramma di sequenza

4.1.5 Diagramma di comunicazione

4.1.6 Diagramma di attività

4.1.7 Codice PlantUML dei diagrammi

4.2 Software Architecture

4.2.1 Stile architetturale

4.2.2 Vista del modulo

4.2.2.1 Punto di vista 1 – Decomposizione

4.2.2.2 Punto di vista 2 – Uses (dipendenze)

4.2.2.3 Punto di vista 3 – Stile a strati (Layered)

4.2.3 Vista C&C – Componenti e Connettori

4.2.4 Vista di allocazione

4.2.4.1 Deployment

4.2.4.2 Implementazione (struttura del repository)

4.2.5 Codice PlantUML dei diagrammi

4.3 Software Design

4.3.1 Introduzione

4.3.2 Diagrammi UML del design

4.3.2.1 Diagramma delle classi

4.3.3 Complessità ciclomatica di McCabe

4.3.3.1 Metodo analizzato

4.3.3.2 Calcolo di CV

4.3.3.3 Cammini indipendenti e casi di test

4.3.4 Misurazione del codice

4.3.4.1 Strumenti e versione analizzata

4.3.4.2 Complessità ciclomatica con PMD

4.3.4.3 Metriche di package con JDepend

4.3.4.4 Analisi con SonarQube

4.3.5 Design pattern

4.3.5.1 Observer

4.3.5.2 Singleton

4.3.6 Codice PlantUML dei diagrammi

5 TESTING

5.1 Software Testing

5.1.1 Organizzazione dei test

5.1.2 Casi di test rappresentativi

5.1.2.1 Tracciabilità tra requisiti e test

5.1.3 Risultati dell'esecuzione

5.1.4 Copertura dei test

5.1.4.1 Copertura del codice di produzione

5.1.4.2 Considerazioni

6 MAINTENANCE

6.1 Software Maintenance

6.1.1 Attività di refactoring

6.1.1.1 R1: eliminazione delle dipendenze cicliche tra package

6.1.1.2 R2: rimozione della classe Segnalazione

6.1.1.3 R3: centralizzazione dei parametri di magazzino

6.1.1.4 R4: configuratore dei ruoli letti dal database

6.1.1.5 R5: orologio iniettabile

6.1.2 Reverse engineering

BIBLIOGRAFIA

1 PROJECT PLAN

1.1 Software Engineering Management

1.1.1 Introduzione

Il presente software implementa la gestione di un supermercato, in particolare l'amministrazione del magazzino e delle scorte, la distinzione dei vari utenti (clienti e personale) e le loro possibili azioni. L'obiettivo è creare una piattaforma che velocizzi il flusso di lavoro ed ottimizzi la qualità del servizio a disposizione degli utenti.

Il software sarà implementato in Java, con database embedded (H2), e adotterà standard e metodologie di ingegneria del software. Le persone responsabili del progetto sono Carmona Contreras Davide Anthony, Dentella Chiara, Frassi Alessandro, Tremolada Chiara.

Il prototipo non dispone di un'interfaccia grafica: l'interazione con clienti e dipendenti è simulata da un'applicazione dimostrativa a riga di comando (classe App), che si esegue dalla console di Eclipse e offre un menu di scelte per le funzionalità principali (self-scanning, gestione dei dipendenti, magazzino).

1.1.2 Modello di processo

Il ciclo di vita del software è sviluppato secondo il modello agile, in particolare seguendo il processo Scrum. La scelta di Scrum consente di adattarsi rapidamente al cambiamento dei requisiti, ricevere feedback frequenti ed effettuare consegne parziali funzionali, riducendo i rischi legati all'accumulo di problemi a fine progetto.

I principali punti di questo processo sono: Product Backlog, Sprint Planning e Daily Scrum. Ogni sprint si conclude con una Sprint Review interna al team, in cui viene verificato lo stato di avanzamento rispetto agli obiettivi pianificati (milestone).

1.1.3 Organizzazione del progetto

Il progetto segue la distinzione dei ruoli adottata da Scrum, ossia:

- **Product Owner** - Frassi Alessandro: responsabile della definizione dei requisiti funzionali e delle priorità.

- **Scrum Master** - Dentella Chiara: facilita l'applicazione del processo Scrum e promuove la comunicazione tra i membri.
- **Development Team** - Carmona Contreras Davide Anthony, Dentella Chiara, Frassi Alessandro, Tremolada Chiara: si occupa di analisi, modellazione UML, sviluppo delle funzionalità, scrittura dei test unitari e verifica della qualità del prodotto.

1.1.4 Standard, linee guida, procedure

Il team adotta le Java Code Conventions (Oracle). La documentazione tecnica e UML è mantenuta aggiornata nel repository GitHub e include commenti esaustivi nel codice, documenti di progettazione e diagrammi creati con Papyrus Designer e PlantUML.

1.1.5 Attività di gestione

Secondo la logica Scrum, il progetto è suddiviso in sprint di una settimana, ognuno caratterizzato da specifiche attività di pianificazione, monitoraggio e controllo. Il progresso è monitorato tramite riunioni quotidiane di breve durata (Daily Scrum), durante le quali il team discute eventuali problematiche e programma il lavoro del giorno seguente.

Al termine di ogni sprint viene redatto un breve resoconto di stato (avanzamento, deviazioni rispetto al piano, priorità del prossimo sprint), condiviso con il team e, quando necessario, comunicato tramite issue GitHub.

1.1.6 Rischi

Il progetto può incorrere in ritardi dovuti a stime iniziali non precise o a imprevisti tecnici. I membri del team possono avere idee contrastanti che portano a possibili conflitti: in questi casi una buona comunicazione interna è fondamentale. Un altro possibile rischio è legato all'incorrettezza/disattenzione nella stesura del codice: un uso attivo delle GitHub issue permette di monitorare problemi e impedimenti.

1.1.7 Personale

Le competenze del team coprono l'intero ciclo di vita dello sviluppo software: analisi, modellazione, implementazione e testing. La squadra spazia in altre abilità fondamentali per la realizzazione del progetto, quali la conoscenza del linguaggio Java e l'utilizzo di Papyrus Designer attraverso Eclipse.

1.1.8 Metodi e tecniche

In questa sezione sono indicati i metodi e le tecniche utilizzati nelle diverse fasi del progetto, la gestione della documentazione tecnica e il controllo di versione/configurazione.

1.1.8.1 Ingegneria dei requisiti

Sono state utilizzate tecniche come interviste, sessioni di brainstorming e analisi degli scenari (e dei casi d'uso).

1.1.8.2 Progettazione

La progettazione è condotta tramite modellazione UML: il class diagram è realizzato con Papyrus Designer (con generazione automatica di una prima versione del codice Java), mentre gli altri diagrammi (casi d'uso, macchine a stati, sequenza, comunicazione/timing, attività, componenti, package) sono realizzati in PlantUML.

1.1.8.3 Implementazione

Lo sviluppo procede in modo incrementale, package per package, seguendo la struttura a strati definita nella Software Architecture (Presentation → Business Logic → Data).

1.1.8.4 Test

I test unitari sono scritti con JUnit 5, con il supporto di Mockito per isolare i Manager della business logic dalle dipendenze verso il layer di persistenza. I test sono organizzati in src/test/java, in package speculari a quelli di produzione (sistemaaccesso, selfscanning, …).

1.1.8.5 Gestione della documentazione tecnica

La documentazione tecnica (requisiti, diagrammi UML, architettura, design, test) è mantenuta in file separati nella cartella documenti/ del repository GitHub, ed è aggiornata progressivamente man mano che procede lo sviluppo.

1.1.8.6 Controllo di versione e configurazione

Il controllo di versione è gestito con GitHub: i comandi principali sono git add, git commit, git push, git pull. Ogni funzionalità è associata a una issue e sviluppata su un branch dedicato; l'integrazione nel branch principale (main) avviene tramite pull request con revisione del codice da parte di un compagno di team.

1.1.9 Garanzia di qualità

Il progetto si ispira ai principi dello standard ISO 9126 come modello di qualità del prodotto, applicando un approccio sistematico e orientato al miglioramento continuo. Ciò include un'attenta pianificazione, controllo e verifica di tutte le attività di sviluppo.

1.1.10 Risorse

Le risorse necessarie alla completa realizzazione del progetto sono i PC dei membri del team, una buona connessione internet e i vari software utilizzati: Eclipse IDE come ambiente di sviluppo e di esecuzione dell'applicazione dimostrativa tramite la sua console, Maven per la gestione delle build, Papyrus Designer e

PlantUML per la modellazione UML, GitHub per la collaborazione, JUnit (con Mockito) per il testing automatico e la libreria log4j per il logging applicativo.

Ogni componente del gruppo è essenziale in ogni fase del processo: confrontarsi di volta in volta limita gli errori e i rischi, garantendo una maggiore affidabilità del prodotto finale.

1.1.11 Budget e schedule

Ad ogni membro del team è assegnata una porzione equa del lavoro da svolgere. Per organizzare in modo chiaro l'assegnazione dei compiti, il lavoro è scomposto nei seguenti macro-task, ciascuno assegnato a un responsabile principale:

Task	Descrizione	Responsabile principale	Sprint
T1	Setup progetto (repository GitHub, struttura Maven/Eclipse, pom.xml, configurazione log4j2), prima versione diagrammi UML	Tutto il team	Sprint 1
T2	Stesura documenti Project Plan e Ingegneria dei Requisiti	Tutto il team	Sprint 2
T3	Stesura documento Software Life Cycle	Tutto il team	Sprint 3
T4	Stesura documento People Management and Team Organization, creazione diagramma delle classi in Papyrus	Tutto il team	Sprint 4
T5	Stesura documento Software Quality, creazione dello scheletro del software (package, classi vuote, interfacce base)	Tutto il team	Sprint 5
T6	Stesura documento Modelling, sistemazione classi e package	Tutto il team	Sprint 6
T7	Stesura documento Software Architecture	Tutto il team	Sprint 7
T8	Stesura documento Software Design	Tutto il team	Sprint 8
T9	Ripresa del progetto: sistemazione incongruenze tra codice e diagrammi, revisione Project Plan	Tutto il team	Sprint 9
T10	Implementazione delle classi, test unitari JUnit	Ciascuno responsabile del proprio task	Sprint 10
T11	Revisione del codice completo e sistemazione degli errori, stesura documento Software Testing	Ciascuno responsabile del proprio task	Sprint 11
T12	Aggiunta classi di test, revisione codice, Javadoc, metriche del software, refactoring, stesura documento Software Maintenance, implementazione demo	Ciascuno responsabile del proprio task	Sprint 12
T13	Documentazione finale, stesura presentazione in ppt	Tutto il team	Sprint 13

Si prevede di finire il lavoro entro tredici sprint (tredici settimane).

1.1.12 Cambiamenti

Dato che il metodo adottato nella realizzazione del progetto è agile, le modifiche vengono gestite in modo leggero: ogni cambiamento del codice e della documentazione viene comunicato durante i Daily Scrum, in modo da evitare incongruenze nel software, ed evidenziato su GitHub tramite le issue.

1.1.13 Consegna

Al termine del progetto verrà consegnato un pacchetto completo contenente il software sviluppato, corredato del codice sorgente e delle librerie esterne utilizzate. La documentazione sarà fornita in formato digitale e includerà il project plan, la specifica dei requisiti, la modellazione UML completa, i piani e i risultati dei test.

1.1.14 Cronologia delle versioni

Versione	Data	Modifiche
1.0	04/05/2026	Versione iniziale del project plan
2.0	29/09/2026	Aggiunta stesura dei punti mancanti
3.0	04/11/2026	Versione finale del documento
4.0	06/11/2026	Ultime piccole modifiche

2 GESTIONE DEL PROGETTO

2.1 Software Life Cycle

2.1.1 Processo di sviluppo adottato

Il team ha adottato Scrum come framework di sviluppo agile. Scrum è un processo iterativo e incrementale che permette di consegnare funzionalità software funzionanti al termine di ogni iterazione, adattandosi ai cambiamenti dei requisiti in modo flessibile. È stato scelto per la dimensione ridotta del team e per la necessità di avere un feedback continuo sull'avanzamento del lavoro. Il processo effettivamente seguito è descritto in questo documento.

2.1.2 Ruoli nel team

All'interno del framework Scrum sono stati definiti i seguenti ruoli:

Ruolo	**Assegnato a**	**Responsabilità**
Product Owner	Frassi Alessandro	Gestione e priorizzazione del Product Backlog
Scrum Master	Dentella Chiara	Facilitatore del processo, rimozione degli ostacoli
Development Team	Tutti i membri	Analisi, progettazione, implementazione e test

2.1.3 Struttura degli sprint

Ogni sprint (idealmente) ha una durata fissa di una settimana, in coerenza con le iterazioni effettivamente eseguite. Si utilizzano iterazioni brevi per massimizzare la visibilità del progresso; al termine di ogni sprint vengono aggiornati il Product Backlog e il piano del progetto. Le pratiche Scrum adottate sono:

Evento	**Quando**	**Scopo**
Sprint Planning	Inizio sprint	Il team seleziona le attività dal Product Backlog e le porta nello Sprint Backlog
Sprint Review	Fine sprint	Il team presenta l'incremento sviluppato e raccoglie feedback

Sprint Retrospective	Fine sprint	Il team analizza il processo e identifica azioni di miglioramento

2.1.4 Pianificazione degli sprint

Sprint eseguiti durante il progetto:

Sprint	Data inizio	Data fine	Obiettivo / attività principali
Sprint 1	20/04/2026	27/04/2026	Setup progetto (repository GitHub, struttura Maven/Eclipse, pom.xml, configurazione log4j2), prima versione diagrammi UML
Sprint 2	28/04/2026	04/05/2026	Stesura documenti Project Plan e Ingegneria dei Requisiti
Sprint 3	05/05/2026	11/05/2026	Stesura documento Software Life Cycle
Sprint 4	12/05/2026	18/05/2026	Stesura documento People Management and Team Organization, creazione diagramma delle classi in Papyrus
Sprint 5	19/05/2026	25/05/2026	Stesura documento Software Quality, creazione dello scheletro del software (package, classi vuote, interfacce base)
Sprint 6	26/05/2026	01/06/2026	Stesura documento Modelling, sistemazione classi e package
Sprint 7	02/06/2026	08/06/2026	Stesura documento Software Architecture
Sprint 8	09/06/2026	15/06/2026	Stesura documento Software Design
Sprint 9	24/09/2026	30/09/2026	Ripresa del progetto: sistemazione incongruenze tra codice e diagrammi, revisione Project Plan
Sprint 10	09/10/2026	16/10/2026	Implementazione delle classi, test unitari JUnit
Sprint 11	17/10/2026	23/10/2026	Revisione del codice completo e sistemazione degli errori, stesura documento Software Testing
Sprint 12	27/10/2026	03/11/2026	Aggiunta classi di test, revisione codice, Javadoc, metriche del software, refactoring, stesura documento Software Maintenance, implementazione demo
Sprint 13	04/11/2026	10/11/2026	Documentazione finale, stesura presentazione in ppt

2.1.5 Approccio MDA con Papyrus

Il team ha applicato una parte dell'approccio Model Driven Architecture (MDA). In MDA lo sviluppo parte da modelli astratti che vengono trasformati automaticamente in codice, spostando il centro del lavoro dalla scrittura manuale del codice alla modellazione. I tre livelli adottati sono:

Livello MDA	**Artefatto nel progetto**	**Strumento**
CIM (Computation Independent Model)	Requisiti del sistema: diagrammi dei casi d'uso e linguaggio naturale	PlantUML, documento di Ingegneria dei Requisiti
PIM (Platform Independent Model)	Class diagram indipendente dalla piattaforma Java	Papyrus
PSM → Codice	Scheletro del codice Java generato dal class diagram	Papyrus Designer

Il class diagram realizzato in Papyrus è stato usato per generare una prima versione del codice Java. Le classi generate sono state poi completate manualmente (corpi dei metodi) e modificate opportunamente.

Dopo la generazione iniziale il codice è stato sviluppato manualmente.

2.2 Configuration Management

2.2.1 Introduzione

Il documento descrive come sono state gestite le configurazioni del progetto Supermercato: il sistema di controllo di versione, la struttura del repository, la strategia di branching, l'uso di issue, pull request e project board Kanban, la gestione dei file esclusi dal versionamento e le statistiche sull'attività del team. Il repository è ospitato su GitHub all'indirizzo *https://github.com/Alessandro13021304/ACDC_Supermercato*.

Il team è composto da quattro persone:

Membro	**Ruolo Scrum**
Alessandro Frassi	Product Owner, Development Team
Chiara Dentella	Scrum Master, Development Team
Chiara Tremolada	Development Team
Davide Anthony Carmona Contreras	Development Team

2.2.2 Strumenti e regole adottate

Ambito	**Strumento / regola**
Sistema di configuration management	Git, con repository remoto su GitHub; il repository è usato lungo tutto lo sviluppo e non solo per la consegna finale
Comandi	git add, git commit, git push, git pull; creazione e integrazione di branch con git branch, git checkout e git merge (anche dall'interfaccia di GitHub)
Branching	Un branch principale (main) e un branch feature/<nome> per ogni funzionalità o attività, creato a partire da main
Integrazione	Ogni branch feature è integrato in main tramite pull request
Issue	Le attività sono tracciate con le issue di GitHub, assegnate ai membri del team e chiuse al completamento
Project board	Kanban board collegata alle issue per visualizzare lo stato di avanzamento
Ambiente di sviluppo	Eclipse con progetto Maven; i file .project e .classpath sono versionati
Modelli UML	La cartella PapyrusProject, con il progetto Papyrus dei diagrammi, è versionata insieme al codice
File esclusi	File .gitignore nella radice e nel progetto Maven
Documentazione	I documenti sono nella cartella documenti/ e sono aggiornati con commit dedicati

2.2.3 Branching e pull request

Lo sviluppo è organizzato in branch di funzionalità, ognuno dedicato a un'attività o a un gruppo di package.

Alcuni interventi di piccola entità, come il caricamento di documenti o piccole modifiche al codice, sono stati eseguiti direttamente su main con commit dedicati.

2.2.4 Issue e project Kanban board

La project board è organizzata come una Kanban board e raccoglie le issue del repository: ogni scheda corrisponde a un'attività e si sposta tra le colonne fino al completamento, quando l'issue viene chiusa. La board permette di vedere in ogni momento le attività da svolgere, quelle in corso e quelle concluse, e di ripartire il lavoro tra i membri del team.

2.2.5 Commit e convenzioni

- I commit sono piccoli e frequenti, spesso uno per package o per insieme di classi di test.
- I messaggi descrivono l'intervento (ad esempio "Implementazione test" o "Aggiunta delle classi di test").
- Il refactoring dei package è documentato nei messaggi, con il motivo dell'intervento (ad esempio l'analisi di copertura con EclEmma).

2.3 People Management and Team Organization

2.3.1 Modello organizzativo

L'organizzazione del team si basa sul modello Scrum, progettato per ottimizzare la cooperazione, lo scambio di informazioni e l'adattabilità. La sua efficacia risiede in una struttura basata su tre ruoli fondamentali che permettono di applicare concretamente i principi della metodologia Agile.

2.3.2 Ruoli e responsabilità

Ruolo	Assegnato a	Responsabilità
Product Owner (PO)	Frassi Alessandro	Fa da tramite tra il mercato e il team: definisce le caratteristiche principali e massimizza il valore del prodotto; gestisce e dà priorità al Product Backlog, allineando il lavoro alle necessità degli stakeholder
Scrum Master	Dentella Chiara	Guida e facilitatore: garantisce che il framework sia applicato correttamente, rimuove gli ostacoli, promuove l'autonomia del team e supporta sia il PO sia gli sviluppatori
Development Team	Tutti i membri	Gruppo multidisciplinare che trasforma le idee in parti concrete del prodotto: analisi, progettazione, implementazione e test; opera in autogestione e pianifica le attività dello Sprint Backlog

Nel progetto gli sprint hanno durata di una settimana. Il framework Scrum prevede in generale sprint da 1 a 4 settimane; la durata breve è stata scelta per ottenere un feedback frequente su un team di dimensioni ridotte.

2.3.3 Eventi e comunicazione

- Sprint Planning all'inizio dello sprint: scelta delle attività e definizione dello Sprint Backlog.
- Sprint Review a fine sprint: presentazione dell'incremento al Product Owner e raccolta del feedback.
- Sprint Retrospective a fine sprint: analisi del processo e azioni di miglioramento.
- Il Product Backlog è lo strumento condiviso di priorità e viene aggiornato dal Product Owner al termine di ogni sprint.

3 REQUISITI

3.1 Requirements Engineering

Il documento segue la struttura dello standard IEEE 830.

3.1.1 Introduzione

3.1.1.1 Obiettivo

Questo documento definisce i requisiti per la creazione di una piattaforma che gestisce un supermercato. Il documento risulta utile anche durante la fase di progettazione.

3.1.1.2 Scopo e ambito

Ottimizzare il servizio offerto dal supermercato ai vari utenti, in particolare attraverso la funzionalità di self-scanning. Il sistema comprende self-scanning, programma fedeltà, gestione dei prodotti e delle scorte, ordini ai fornitori, gestione di turni, presenze e stipendi dei dipendenti.

3.1.1.3 Definizioni, acronimi e abbreviazioni

- Sistema di fidelizzazione = sistema basato sull'utilizzo dell'account fedeltà del cliente, che permette di usufruire di promozioni speciali e di accumulare punti da utilizzare come premio
- Self-scanning = spesa supportata dall'utilizzo di un lettore ottico
- Utente = sia i clienti del supermercato sia i suoi dipendenti
- Totem = postazione di pagamento self-service
- Sessione di spesa = intervallo che va dall'inizio della scansione al completamento o alla segnalazione dell'acquisto
- Carrello fisico = i prodotti realmente presenti nel carrello del cliente, confrontati dal cassiere con il carrello virtuale
- Carrello virtuale = i prodotti scansionati dal cliente con il lettore ottico
- Segnalazione = registrazione di un'anomalia emersa dal controllo della spesa, associata al cliente
- RF = requisito funzionale
- RP = requisito di prestazione.

3.1.1.4 Riferimenti

IEEE 830, ISO/IEC 9126, "Software Engineering: Principles and Practice" di Hans van Vliet, "UML @ Classroom" di Martina Seidl, Marion Scholz, Christian Huemer e Gerti Kappel.

3.1.1.5 Panoramica del documento

Il capitolo 3.1.2 descrive il prodotto, gli utenti e i vincoli; il capitolo 3.1.3 contiene i requisiti specifici (funzionali, di prestazione, di progettazione, attributi di qualità e requisiti normativi), la tracciabilità e i casi; il codice PlantUML dei diagrammi dei casi d'uso è riportato nel paragrafo 3.1.4.

3.1.1.6 Elicitazione dei requisiti

I requisiti sono stati raccolti e raffinati in modo iterativo, a partire dallo Sprint 1 in cui si è stesa una prima versione dei diagrammi UML, e rivisti in seguito alla verifica di coerenza con il codice. Le tecniche utilizzate sono le seguenti.

- Interviste: confronto con un dipendente di un supermercato sulle esigenze delle diverse categorie di utenti.
- Sessioni di brainstorming: individuazione delle funzionalità del self-scanning, del programma fedeltà e della gestione di magazzino e personale.
- Analisi degli scenari (e dei casi d'uso): descrizione dei flussi principali con i diagrammi dei casi d'uso.

3.1.2 Descrizione generale

3.1.2.1 Prospettiva del prodotto

Il software si appoggia su tre database embedded: uno per i clienti iscritti al programma fedeltà (anagrafica, punti, storico acquisti, segnalazioni), uno per i dipendenti (dati anagrafici, ruolo, turni e presenze) e uno per i prodotti (catalogo con quantità sugli scaffali e in magazzino, ordini ai fornitori e righe d'ordine).

3.1.2.2 Funzioni del prodotto

Gestione anagrafe prodotti (sugli scaffali e in magazzino), gestione personale (dati anagrafici, turni, presenze e stipendi), sicurezza attraverso autenticazione degli utenti, gestione della spesa attraverso il self-scanning, gestione del programma fedeltà, gestione delle scorte e degli ordini ai fornitori.

3.1.2.3 Caratteristiche dell'utente

Le diverse categorie di utenti hanno esigenze specifiche:

- Clienti: dispongono di un'interfaccia intuitiva e veloce per il self-scanning.
- Cassieri: garantiscono rapidità e affidabilità nel servizio; intervengono al totem quando una spesa richiede un controllo e confrontano il carrello virtuale con quello fisico.
- Magazzinieri: verificano la disponibilità, segnalano la mancanza di prodotti in magazzino e spostano la merce dal magazzino agli scaffali.
- Manager: assegnano i turni ai dipendenti e calcolano gli stipendi.
- Responsabili di reparto: gestiscono il catalogo dei prodotti e si occupano degli ordini ai fornitori per i prodotti del proprio reparto.

- Fornitori: quando ricevono un ordine dal responsabile di reparto, riforniscono il magazzino del supermercato con la merce mancante

3.1.2.4 Vincoli

I clienti non possono accedere al magazzino né scansionare prodotti non disponibili sugli scaffali.

3.1.2.5 Presupposti e dipendenze

Ogni punto cassa è dotato di un PC; i dati anagrafici iniziali dei prodotti sono caricati dal reparto amministrativo e successivamente aggiornati dal responsabile di reparto; l'orario di apertura del supermercato è definito a priori e non è gestito dal sistema; il servizio di pagamento esterno è accessibile tramite l'interfaccia IPagamento; i magazzinieri e i responsabili di reparto sono registrati come osservatori delle scorte all'avvio del sistema.

3.1.3 Requisiti specifici

3.1.3.1 Requisiti dell'interfaccia esterna

- Interfaccia utente: nel prototipo l'interazione con clienti e dipendenti è simulata da un'applicazione dimostrativa a riga di comando; le funzionalità sono esposte dalle classi di dominio (LettoreOttico, Totem, Cassiere, Manager, Magazziniere, ResponsabileDiReparto).
- Dispositivi: lettore ottico per la scansione dei codici a barre; totem per il pagamento e per la richiesta di controllo.
- Sistema di pagamento esterno: accessibile tramite l'interfaccia IPagamento (nel prototipo simulato da PaymentGateway, che accetta ogni importo positivo).
- Persistenza: database H2 embedded, configurato in database.properties e database-test.properties.
- Tracciamento: eventi rilevanti registrati tramite Log4j 2.

3.1.3.2 Requisiti funzionali

Ogni requisito ha un identificativo (RF-…) usato nella tabella di tracciabilità del paragrafo 3.1.3.7.

3.1.3.2.1 Funzioni dei clienti

RF-C1: Registrazione e autenticazione. Il cliente, se desidera effettuare la spesa in modalità self-scanning, deve essere registrato al programma fedeltà, in modo da ottenere sconti sui prodotti. Il sistema richiede email, password e alcuni dati personali (nome, cognome, data di nascita) per creare il profilo; l'email deve essere univoca. Se è già registrato, può autenticarsi con il proprio account sul lettore ottico.

RF-C2: Gestione profilo e punti fedeltà. Il cliente può visualizzare i punti accumulati, lo storico degli acquisti e le segnalazioni ricevute.

RF-C3: Gestione del carrello e sconti. Il cliente scansiona il codice a barre di ogni prodotto; se desidera togliere un articolo dispone di una funzione per rimuoverlo dal lettore, che riduce di una unità la quantità e, all'ultima unità, elimina l'articolo dal carrello. Un prodotto non può essere aggiunto se non è disponibile sullo scaffale

o se la quantità nel carrello ha raggiunto quella disponibile; il sistema comunica l'esito dell'operazione. Lo sconto del prodotto è applicato automaticamente subito dopo la scansione. Terminata la spesa, non sono più ammesse nuove scansioni.

RF-C4: Pagamento, premio fedeltà e punti. Terminata la spesa, il cliente si reca al totem e sceglie la modalità di pagamento (contanti, carta di credito o carta di debito). Prima di pagare può utilizzare il premio fedeltà: se possiede almeno 100 punti, ottiene uno sconto percentuale sul totale pari a punti/20, con un massimo del 30%; si scalano i punti utilizzati, al massimo 600. Il premio può essere utilizzato una sola volta per sessione, e i punti sono scalati definitivamente solo se l'acquisto viene completato: se il pagamento è rifiutato o la sessione viene segnalata, i punti restano invariati. A pagamento completato e controllo superato, il cliente accumula 1 punto ogni 2 euro pagati (parte intera), lo storico dell'acquisto viene registrato e la quantità a scaffale dei prodotti acquistati viene diminuita; se scende sotto la soglia minima si applica la procedura di rifornimento del requisito RF-D8. Al termine è disponibile la ricevuta, che riporta totale, modalità di pagamento e data.

RF-C5: Controllo della spesa. Il pagamento può richiedere un controllo da parte di un dipendente. Il controllo è richiesto se il cliente è già stato segnalato in precedenza (controllo obbligatorio) oppure in modo casuale, con probabilità del 50%. Il controllo viene richiesto solo se un cassiere è disponibile al totem; in caso contrario il pagamento non viene addebitato e la sessione resta in attesa di pagamento. Il controllo parte quando il cassiere è presente e il carrello fisico è disponibile. Se i prodotti del carrello virtuale e quelli del carrello fisico non corrispondono (carrello mancante, numero di prodotti diverso, prodotto assente o quantità diversa), la sessione e il cliente vengono segnalati, la segnalazione è registrata in modo persistente e l'acquisto non viene registrato (punti, storico e scorte non sono aggiornati); la gestione dell'importo già addebitato è affidata al personale di cassa. Se corrispondono, l'acquisto viene completato e registrato. Al termine del controllo il cassiere non risulta più presente al totem.

3.1.3.2.2 Funzioni dei dipendenti

RF-D1: Registrazione del dipendente. Quando viene assunto, il dipendente è registrato nel sistema: sono richiesti dati personali, email (univoca), password, ruolo (Manager, Cassiere, Magazziniere, Responsabile di reparto) e matricola (univoca).

RF-D2: Autenticazione e presenza. All'inizio del turno il dipendente si autentica con email e password digitate; il sistema verifica che le credenziali appartengano proprio al dipendente che registra la presenza e registra l'ora di ingresso sul turno del giorno non ancora iniziato. Alla fine del turno ripete la procedura e il sistema registra l'ora di uscita, completando il turno. Un turno senza uscita registrata non è considerato completato e non concorre allo stipendio.

RF-D3: Assegnazione dei turni. Il manager assegna i turni ai dipendenti.

RF-D4: Vincoli sui turni. L'orario di inizio di un turno precede quello di fine. Un dipendente non può avere due turni sovrapposti nella stessa giornata; in caso di sovrapposizione l'assegnazione non viene effettuata e il manager ne riceve notizia (esito negativo dell'assegnazione). Turni consecutivi, in cui l'orario di fine di un turno coincide con l'orario di inizio del successivo, sono consentiti. Ogni turno inizia e finisce nella stessa giornata (non sono previsti turni a cavallo della mezzanotte).

RF-D5: Calcolo degli stipendi. Alla fine di ogni mese il manager calcola lo stipendio di ogni dipendente sommando le ore effettivamente lavorate nei turni completati del mese, tra ora di ingresso e ora di uscita,

moltiplicate per la tariffa oraria del ruolo (Cassiere 10 €/h, Magazziniere 11 €/h, Responsabile di reparto 14 €/h, Manager 16 €/h).

RF-D6: Visualizzazione dei turni. Ogni dipendente può consultare i propri turni assegnati.

RF-D7: Visualizzazione del catalogo prodotti. Il magazziniere e il responsabile di reparto possono visualizzare la disponibilità di ogni prodotto in magazzino e sullo scaffale, il prezzo e la descrizione. In base alle quantità valutano se richiedere nuovi prodotti (vedi RF-D8).

RF-D8: Gestione dei prodotti e delle scorte. Il responsabile di reparto può aggiungere, modificare e rimuovere prodotti. Quando la quantità a scaffale scende sotto la soglia minima di 5 pezzi, il sistema avvisa il magazziniere, che sposta dal magazzino allo scaffale fino a 20 pezzi (o quelli disponibili) e registra in modo persistente la nuova quantità. Quando la quantità in magazzino scende sotto la soglia minima di 5 pezzi, il magazziniere avvisa il responsabile del reparto del prodotto, che elabora un ordine al fornitore di 100 pezzi per ciascun prodotto.

RF-D9: Ricezione degli ordini. Alla consegna dell'ordine da parte del fornitore, le quantità ricevute vengono sommate alla giacenza di magazzino dei prodotti corrispondenti e lo stato dell'ordine passa da creato a ricevuto; un ordine già ricevuto non incrementa nuovamente le scorte.

RF-D10: Controllo self-scanner. Al cassiere viene notificata la richiesta di controllo con il numero del totem presso cui recarsi. All'arrivo del cassiere il sistema avvia il controllo della spesa (RF-C5).

3.1.3.3 Requisiti di prestazione

Il sistema deve garantire tempi di risposta adeguati durante le principali operazioni.

- RP-1 Self-scanning: autenticazione del cliente, ricerca di un prodotto tramite codice a barre, aggiunta o rimozione dal carrello in al massimo 2 secondi nel 95% dei casi.
- RP-2 Consultazione: visualizzazione del catalogo, delle scorte e dei turni in al massimo 2 secondi nel 95% dei casi.
- RP-3 Pagamento: elaborazione della richiesta al totem in al massimo 3 secondi nel 95% dei casi, escluso il servizio esterno di pagamento.
- RP-4 Richieste di controllo: registrazione della richiesta e disponibilità del numero del totem per il cassiere in massimo 2 secondi nel 95% dei casi.
- RP-5 Gestione di personale e scorte: consultazione e aggiornamento di dipendenti, turni, prodotti e scorte in al massimo 2 secondi nel 95% dei casi.

3.1.3.4 Vincoli di progettazione

- Linguaggio: Java (versione 17).
- Ambiente di sviluppo: Eclipse.
- Gestione delle dipendenze: Maven, con almeno una libreria esterna (Log4j 2).
- Database: embedded (H2), senza installazione di un database server esterno.
- Servizi esterni: nessun servizio da installare, salvo eccezioni concordate.

- Persistenza: i dati di clienti, dipendenti, turni, prodotti e ordini sono gestiti dal package persistenzadati.
- Modellazione UML: il class diagram è realizzato con Papyrus e usato per la generazione iniziale del codice; gli altri diagrammi UML sono realizzati con PlantUML.
- Struttura del progetto: test in una cartella separata, nessun riferimento assoluto ai file del computer.
- Struttura del codice: package privi di dipendenze cicliche, con le interfacce di accesso ai dati possedute dal dominio.
- Qualità del codice: analisi statica con PMD.
- Gestione del codice: sorgenti e documentazione su GitHub.

3.1.3.5 Attributi del software

- **Correttezza**: il sistema deve eseguire correttamente le operazioni di spesa, pagamento, punti fedeltà, scorte e stipendi.
- **Affidabilità**: il sistema deve garantire consistenza e integrità dei dati durante le principali operazioni; il completamento di un acquisto (punti, storico, scorte) dovrebbe avvenire in un'unica transazione. Nel prototipo le operazioni usano connessioni separate (limite noto, vedi 4.1.4); la consistenza è garantita da aggiornamenti atomici condizionati (ad esempio la quantità a scaffale non scende mai sottozero e i punti non diventano negativi).
- **Usabilità**: le funzioni per i clienti devono essere semplici e intuitive; quelle per i dipendenti devono consentire di svolgere rapidamente le operazioni del proprio ruolo.
- **Manutenibilità**: il sistema deve facilitare la modifica o l'estensione delle singole funzionalità, senza dipendenze cicliche tra package.
- **Testabilità**: il sistema deve consentire la verifica delle singole funzionalità con test automatizzati e indipendenti (JUnit 5, Mockito).
- **Portabilità**: il sistema deve poter essere eseguito su computer diversi compatibili con Java 17, senza percorsi assoluti né servizi esterni.
- **Integrità**: il sistema deve proteggere i dati gestiti, impedendo modifiche non autorizzate; le credenziali devono essere memorizzate in forma protetta (hash) e le funzioni devono essere accessibili solo al ruolo corrispondente.
- **Tracciabilità**: gli eventi di login, accessi falliti, pagamenti e segnalazioni devono essere registrati nel log.

3.1.3.6 Altri requisiti

- Protezione dei dati personali (GDPR, D. Lgs. 196/2003): il sistema deve limitare il trattamento ai dati necessari e proteggere le credenziali; la conformità completa (consenso, cancellazione, conservazione) è fuori dall'ambito del prototipo.

- Fatturazione elettronica e fiscale (Legge 205/2017): nel prototipo il sistema genera la ricevuta di acquisto; l'emissione di documenti fiscali elettronici è demandata a un sistema esterno.
- Diritto del lavoro (D. Lgs. 66/2003): turni e presenze devono permettere la verifica dell'orario di lavoro; nel prototipo si controlla la sola sovrapposizione dei turni.

3.1.3.7 Tracciabilità dei requisiti

La tabella collega ogni requisito funzionale ai casi d'uso, alle classi principali che lo realizzano e ai casi di test del capitolo 5.

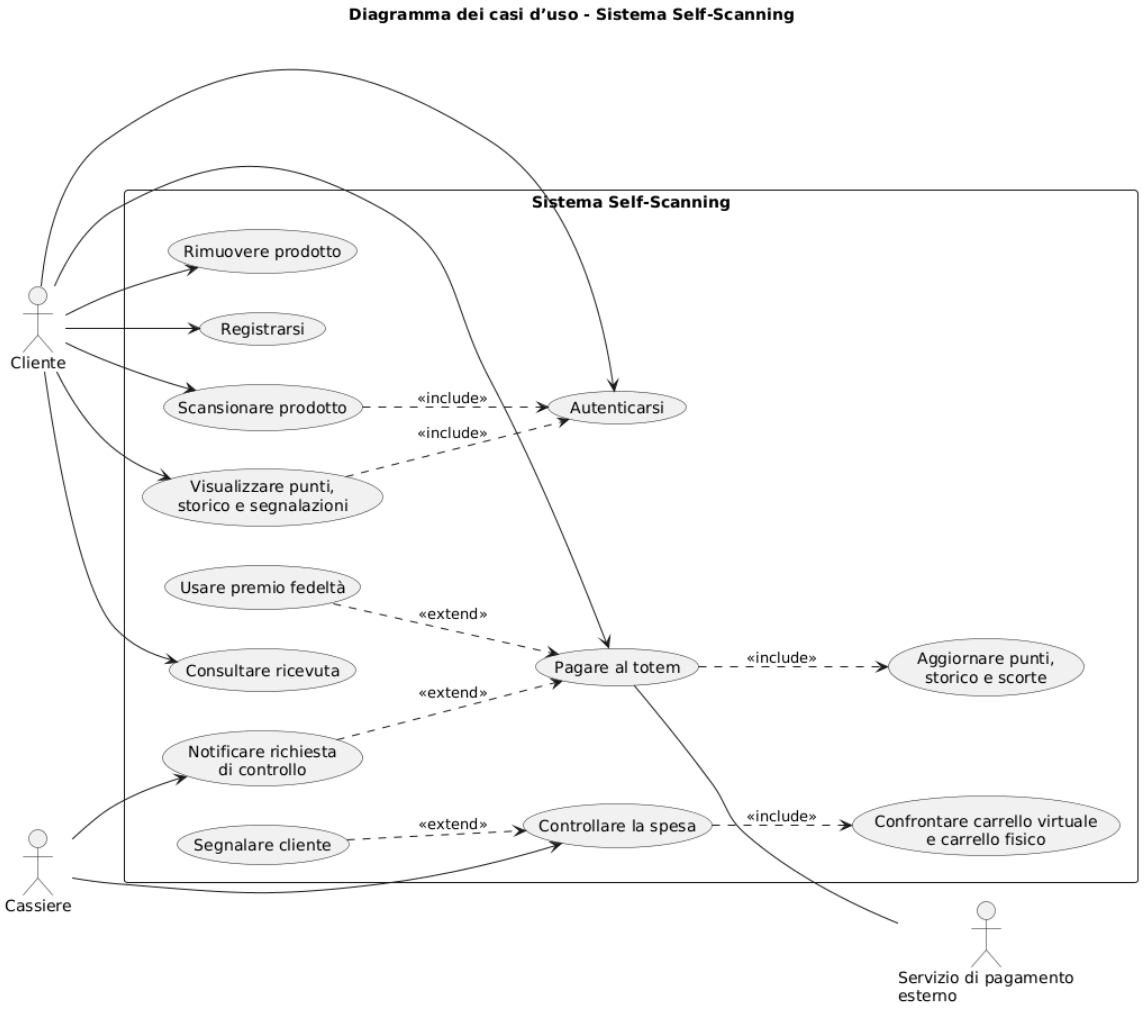
Requisito	Caso d'uso	Classi principali	Test
RF-C1	Registrazione, autenticazione	AuthService, Cliente, DBClienti	TC-01, TC-02
RF-C2	Visualizzazione punti, storico e segnalazioni	Cliente, DBClienti	TC-12
RF-C3	Scansione e rimozione prodotto	LettoreOttico, Carrello, ElementoCarrello, CalcolatoreSconto	TC-04
RF-C4	Pagamento al totem, premio fedeltà	SessioneSpesa, Totem, Pagamento, GeneratoreRicevuta, Promozione, PuntiAccumulati	TC-05, TC-14, TC-15, TC-17, TC-19, TC-23
RF-C5	Controllo della spesa	SessioneSpesa, Totem, Cassiere, ControlloSpesa, CarrelloFisico	TC-05, TC-16, TC-19, TC-23
RF-D1	Registrazione	AuthService, DBDipendenti	TC-03
RF-D2	Autenticazione (inizio/fine turno)	GestorePresenze, Turno	TC-08, TC-22
RF-D3, RF-D4	Assegnazione turni	Manager, GestoreTurni, Turno	TC-07, TC-21
RF-D5	Calcolo stipendi	CalcoloStipendi, Stipendio, Presenza	TC-11
RF-D6	Visualizzazione turni	Dipendente, GestoreTurni	TC-07
RF-D7, RF-D8	Visualizzazione prodotti, rifornimento scaffale	GestoreProdotti, GestoreScorte, ParametriScorte, Magazziniere, ResponsabileDiReparto	TC-06, TC-09, TC-24, TC-25
RF-D8, RF-D9	Elaborazione e ricezione ordine	ResponsabileDiReparto, GestoreOrdini, OrdineFornitore	TC-10, TC-18
RF-D10	Controllo scanner	Totem, Cassiere	TC-05, TC-16, TC-20, TC-23

3.1.3.8 Casi d'uso

I casi d'uso del sistema sono rappresentati da due diagrammi: il primo riguarda il self-scanning (clienti e cassieri), il secondo la gestione di dipendenti e magazzino. Il codice PlantUML dei diagrammi è riportato nel paragrafo 3.1.4. La macchina a stati della sessione di spesa è riportata nel capitolo 4.1.

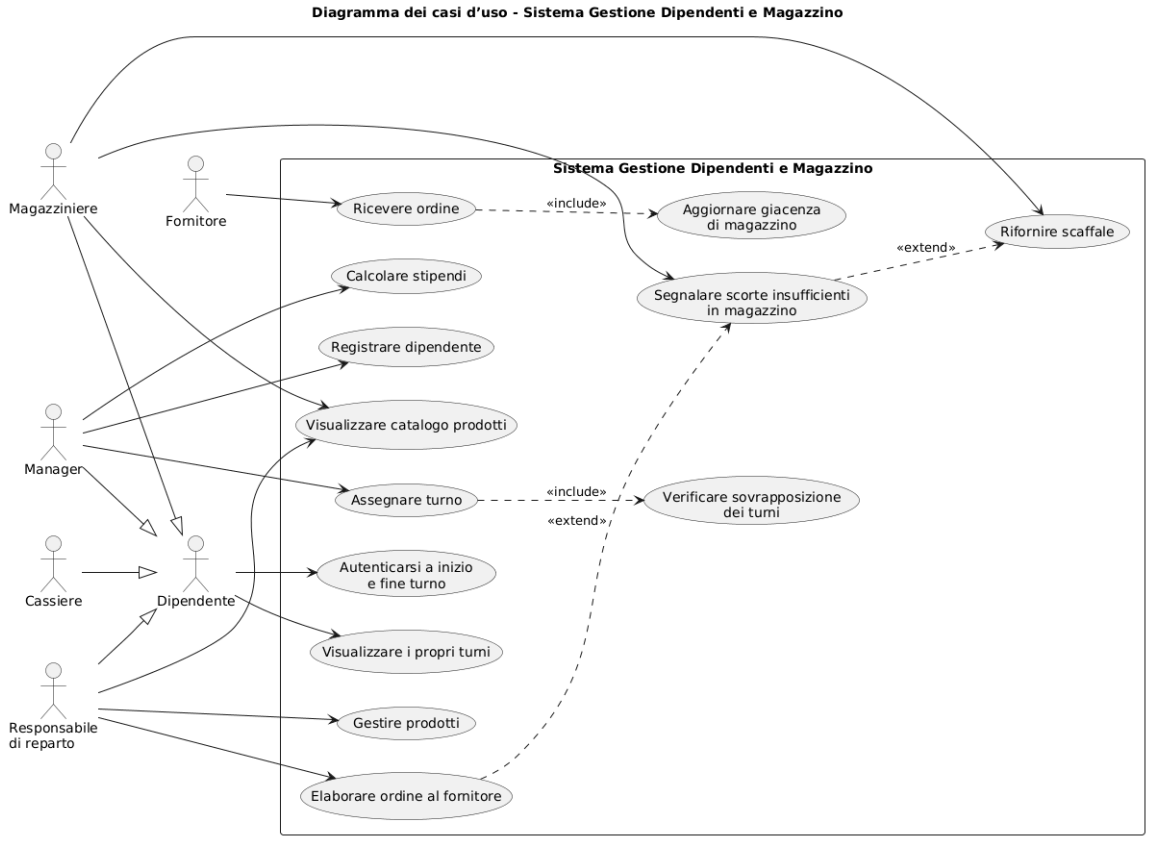
Self-scanning

Attori: Cliente, Cassiere e servizio di pagamento esterno. Il cliente si registra e si autentica, scansiona e rimuove i prodotti, può visualizzare punti, storico e segnalazioni, paga al totem e consulta la ricevuta. L'utilizzo del premio fedeltà e la richiesta di controllo estendono il pagamento. Il cassiere esegue il controllo della spesa, che include il confronto tra carrello virtuale e carrello fisico e può estendersi nella segnalazione del cliente.



Gestione dipendenti e magazzino

Attori: Manager, Cassiere, Magazziniere, Responsabile di reparto (tutti specializzazioni di Dipendente) e Fornitore. Il dipendente si autentica a inizio e fine turno e consulta i propri turni; il manager registra i dipendenti, assegna i turni (con verifica dell'assenza di sovrapposizioni) e calcola gli stipendi; magazziniere e responsabile consultano il catalogo; il magazziniere rifornisce lo scaffale e segnala le scorte insufficienti in magazzino; il responsabile gestisce i prodotti ed elabora gli ordini; il fornitore consegna l'ordine, la cui ricezione aggiorna le scorte.



3.1.4 Codice PlantUML dei diagrammi dei casi d'uso

Diagramma dei casi d'uso del sistema self-scanning

```
@startuml
title Diagramma dei casi d'uso – Sistema Self-Scanning
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor Cliente
actor Cassiere
actor "Servizio di pagamento\nesterno" as Pagamento

rectangle "Sistema Self-Scanning" {
  usecase "Registrarsi" as UC1
  usecase "Autenticarsi" as UC2
  usecase "Scansionare prodotto" as UC3
  usecase "Rimuovere prodotto" as UC4
  usecase "Visualizzare punti,\nstorico e segnalazioni" as UC5
  usecase "Pagare al totem" as UC6
  usecase "Usare premio fedeltà" as UC7
  usecase "Notificare richiesta\ndi controllo" as UC8
  usecase "Consultare ricevuta" as UC9
  usecase "Controllare la spesa" as UC10
  usecase "Confrontare carrello virtuale\ne carrello fisico" as UC11
  usecase "Segnalare cliente" as UC12
  usecase "Aggiornare punti,\nstorico e scorte" as UC13
}

Cliente --> UC1
Cliente --> UC2
```

```
Cliente --> UC3
Cliente --> UC4
Cliente --> UC5
Cliente --> UC6
Cliente --> UC9
Cassiere --> UC8
Cassiere --> UC10
UC6 -- Pagamento

UC3 ..> UC2 : <<include>>
UC5 ..> UC2 : <<include>>
UC6 ..> UC13 : <<include>>
UC7 ..> UC6 : <<extend>>
UC8 ..> UC6 : <<extend>>
UC10 ..> UC11 : <<include>>
UC12 ..> UC10 : <<extend>>
@enduml
```

Diagramma dei casi d'uso del sistema gestione dipendenti e magazzino

```
@startuml
title Diagramma dei casi d’uso – Sistema Gestione Dipendenti e Magazzino
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor Dipendente
actor Manager
actor Cassiere
actor Magazziniere
actor "Responsabile\ndi reparto" as Responsabile
actor Fornitore

Manager --|> Dipendente
Cassiere --|> Dipendente
Magazziniere --|> Dipendente
Responsabile --|> Dipendente

rectangle "Sistema Gestione Dipendenti e Magazzino" {
  usecase "Autenticarsi a inizio\ne fine turno" as D1
  usecase "Visualizzare i propri turni" as D2
  usecase "Registrare dipendente" as D3
  usecase "Assegnare turno" as D4
  usecase "Verificare sovrapposizione\ndei turni" as D5
  usecase "Calcolare stipendi" as D6
  usecase "Visualizzare catalogo prodotti" as D7
  usecase "Gestire prodotti" as D8
  usecase "Rifornire scaffale" as D9
  usecase "Segnalare scorte insufficienti\nin magazzino" as D10
  usecase "Elaborare ordine al fornitore" as D11
  usecase "Ricevere ordine" as D12
  usecase "Aggiornare giacenza\ndi magazzino" as D13
}

Dipendente --> D1
Dipendente --> D2
Manager --> D3
Manager --> D4
Manager --> D6
Magazziniere --> D7
Responsabile --> D7
Responsabile --> D8
Magazziniere --> D9
Magazziniere --> D10
Responsabile --> D11
Fornitore --> D12
```

```
D4 ..> D5 : <<include>>
D12 ..> D13 : <<include>>
D10 ..> D9 : <<extend>>
D11 ..> D10 : <<extend>>
@enduml
```

3.2 Software Quality

Per il sistema di gestione del supermercato, il team ha identificato le qualità che hanno effettivamente guidato le scelte di progettazione e implementazione. Sono descritte secondo la tassonomia di McCall e le caratteristiche ISO 9126, ciascuna con la spiegazione relativa al contesto del progetto.

3.2.1 Correttezza

Il sistema gestisce operazioni con impatto diretto sui dati economici del cliente (scontrino, punti fedeltà, pagamento) e sui dati aziendali (scorte di magazzino, stipendi dei dipendenti). Un errore nel calcolo del totale del carrello o nell'applicazione di una promozione produrrebbe un danno economico reale o una contestazione legale. La correttezza è stata perseguita tramite:

- definizione precisa dei casi d'uso e dei requisiti funzionali;
- una suite di test automatici JUnit 5 sulle regole di business critiche (sconti, punti fedeltà, disponibilità a scaffale, turni, presenze, stipendi, ordini), descritta nel documento di Testing;
- l'uso di interfacce Java per contrattualizzare il comportamento atteso dei componenti: IAutenticazione, ICarrello, ICarrelloFisico, IControlloSpesa, ISessioneControllabile, IPagamento, IScorte, IScorteObserver, IOrdini, IProdotti, IPresenze, ITurni, IStipendi e le interfacce IRepository*;
- classi dedicate alle regole di business: CalcolatoreSconto (sconto di prodotto), Promozione (premio fedeltà: soglia di 100 punti, sconto pari a punti/20 con tetto del 30% e scalo di al massimo 600 punti), PuntiAccumulati (1 punto ogni 2 euro spesi, parte intera), Presenza e Stipendio (ore lavorate tra ingresso e uscita per la tariffa oraria del ruolo);
- il controllo di corrispondenza tra carrello virtuale e carrello fisico per codice a barre: Prodotto definisce equals e hashCode sul codice a barre, quindi due istanze dello stesso prodotto sono considerate uguali (anche nelle mappe usate da ControlloSpesa);
- i punti del premio fedeltà scalati solo al completamento dell'acquisto: usaPremioFedelta memorizza i punti da scalare e lo scalo avviene in registraAcquistoCompletato, quindi un pagamento rifiutato o una sessione segnalata non fanno perdere punti al cliente;
- la ricezione degli ordini idempotente: lo stato dell'ordine è l'enumerazione EStatoOrdine (CREATO, RICEVUTO), persistito nel database; GestoreOrdini.riceviOrdine ignora gli ordini già ricevuti e, dopo l'aggiornamento delle scorte, segna l'ordine come ricevuto e ne salva lo stato;
- la validazione dei turni nel dominio: Turno.isOrarioValido() richiede che l'orario di inizio preceda quello di fine (quindi ogni turno inizia e finisce nella stessa giornata) e GestoreTurni rifiuta i turni sovrapposti, ammettendo quelli consecutivi; assegnaTurno restituisce un boolean che indica se il turno è stato assegnato, e Manager.assegnaTurno lo propaga al chiamante;
- la centralizzazione dei parametri di magazzino nella classe ParametriScorte (soglia minima 5, rifornimento 20, ordine standard 100).

3.2.2 Affidabilità

Il sistema opera in un ambiente retail con sessioni di spesa attive in contemporanea. Un'interruzione durante una sessione di pagamento o una perdita di dati sulle scorte di magazzino causerebbe problemi operativi gravi. L'affidabilità ha guidato le seguenti scelte:

- uso di H2 embedded con accesso JDBC tramite PreparedStatement e try-with-resources, che garantisce la chiusura delle risorse e la propagazione degli errori come eccezioni (RuntimeException con causa originale);
- SessioneSpesa modella lo stato della spesa con una macchina a stati (EStatoSessione: IN_CORSO, PAGAMENTO, CONTROLLO, SEGNALATA, COMPLETATA); le operazioni sono consentite solo negli stati corretti;
- il pagamento viene addebitato solo dopo aver verificato che il controllo, se necessario, possa essere eseguito: se la spesa richiede un controllo e nessun cassiere è disponibile al totem, effettuaPagamento restituisce false prima di chiamare il servizio di pagamento e la sessione resta nello stato PAGAMENTO;
- il controllo della spesa parte solo quando il cassiere lo richiede: SessioneSpesa.avviaControllo si limita a portare lo stato a CONTROLLO, il confronto è eseguito da Totem.effettuaControlloSpesa con il carrello fisico disponibile e, a controllo terminato, Totem azzera lo stato del cassiere (registraPartenzaCassiere);
- l'aggiornamento delle scorte è disaccoppiato con il pattern Observer (IScorteObserver): al calo dello scaffale sotto soglia il Magazziniere sposta merce dal magazzino e al calo del magazzino il ResponsabileDiReparto genera l'ordine al fornitore del proprio reparto; la demo registra gli osservatori una sola volta per dipendente;
- gli spostamenti di quantità sono operazioni atomiche del database: DBProdotti.decrementaQuantitaScaffale e spostaDalMagazzinoAlloScaffale usano un UPDATE condizionato dalla disponibilità (quantita >= ?) e restituiscono se l'aggiornamento è avvenuto; il rifornimento dello scaffale eseguito dal Magazziniere viene quindi salvato tramite il repository e non si perde al riavvio. Ogni aggiornamento è quindi atomico singolarmente, anche se non lo è la sequenza complessiva;
- lo scalo dei punti fedeltà è protetto da una condizione sul saldo (punti_fedelta >= ?), che impedisce punteggi negativi;
- limite noto: il completamento di un acquisto (scalo dei punti, accumulo dei nuovi punti, registrazione dello storico e decremento delle scorte) avviene con operazioni su connessioni separate, senza un'unica transazione. Un'interruzione a metà della sequenza potrebbe quindi lasciare i dati parzialmente aggiornati. Il rischio è ridotto dagli aggiornamenti atomici condizionati descritti sopra e dall'ordine delle operazioni (i punti del premio vengono scalati solo dopo il completamento dell'acquisto, quindi un pagamento rifiutato o una sessione segnalata non modificano i dati); una transazione unica è indicata come evoluzione futura (vedi 3.1.3.5 e 4.1.4);
- la segnalazione di anomalie (cliente segnalato, controllo casuale) è gestita da Totem e Cassiere e registrata in modo persistente (tabella segnalazioni e contrassegno del cliente);

- Log4j 2 è incluso come dipendenza Maven e usato per tracciare l'avvio, l'inizializzazione dei database e gli errori; un eventuale fallimento del logging non deve compromettere il flusso principale.

3.2.3 Usabilità

Il sistema ha tre tipologie di utenti con background molto diversi:

- il cliente usa il lettore ottico e il totem di pagamento: interazioni ridotte al minimo (scansione, conferma, pagamento), senza necessità di formazione; le operazioni di scansione e rimozione restituiscono un esito, che permette di segnalare subito prodotti esauriti o assenti dal carrello;
- il cassiere interviene sul totem quando una sessione richiede un controllo (notifica con il numero del totem, confronto tra carrello e spesa fisica, sblocco o segnalazione), senza distrarsi dal flusso del negozio;
- il dipendente/manager (Manager, Magazziniere, ResponsabileDiReparto) usa le funzionalità di back-office per turni, presenze, stipendi, magazzino e ordini.

3.2.4 Manutenibilità

Il dominio del supermercato è soggetto a cambiamenti frequenti: nuove promozioni, variazioni dei turni, aggiornamento del catalogo prodotti, nuovi fornitori. Il sistema deve essere modificabile localmente senza effetti a cascata su componenti non coinvolti. La manutenibilità ha guidato:

- la suddivisione in package per responsabilità: sistemaaccesso, selfscanning, gestionemagazzino, gestionedipendenti, gestionefedelta, persistenzadati, pagamentiesterni;
- l'uso di interfacce (IRepositoryClienti, IRepositoryDipendenti, IRepositoryProdotti, IRepositoryOrdini) che disaccoppia la logica dall'implementazione dei DAO (DBClienti, DBDipendenti, DBProdotti, DBOrdini); le interfacce sono possedute dal dominio (Dependency Inversion Principle);
- la centralizzazione dell'accesso ai dati in DBConnection (Singleton), che legge la configurazione da database.properties, e dei parametri di magazzino in ParametriScorte, che elimina le soglie duplicate in più classi;
- l'assenza di dipendenze cicliche tra package, verificabile con JDepend: Cassiere dipende solo da porte definite in sistemaaccesso (ISessioneControllabile, IControlloSpesa, ICarrelloFisico), implementate da SessioneSpesa, ControlloSpesa e CarrelloFisico in selfscanning; Manager usa le porte ITurni e IStipendi, implementate in gestionedipendenti; persistenzadati dipende dal dominio e mai viceversa;
- l'uso delle caratteristiche di Java per l'information hiding e l'astrazione: campi privati, interfacce, classi astratte (Persona, Dipendente), @Override e pattern matching per instanceof (Java 17);
- il logging con Log4j 2, che permette di diagnosticare i guasti analizzando i log senza modificare il codice sorgente;
- il controllo statico del codice con PMD, configurato nel progetto con violazioni trattate come errori.

3.2.5 Testabilità

La testabilità ha avuto un impatto diretto sulle scelte architetturali. In particolare:

- le interfacce IRepository* rendono possibile la sostituzione dei DAO con mock (Mockito) o stub; i gestori e i ruoli ricevono i repository nel costruttore o tramite setter;
- il controllo casuale della spesa usa un Random iniettabile (e il metodo controlloCasuale può essere ridefinito nei test), che permette di rendere deterministici i test;
- GestorePresenze accetta un Clock nel costruttore e Turno offre metodi registraInizio e registraFine che ricevono l'ora, quindi le presenze possono essere verificate senza dipendere dall'orologio di sistema;
- i DAO possono essere istanziati in modalità test (parametro usaDatabaseTest), che usa database-test.properties; DBConnection offre getTestConnection e resetTestDatabase per riportare il database di test a uno stato noto;
- la separazione tra logica di dominio, accesso ai dati e componenti di interazione consente di testare le classi di dominio senza istanziare il database;
- la struttura Maven standard (src/test/java) separa il codice di test da quello di produzione.

3.2.6 Portabilità

Il progetto è sviluppato come progetto Eclipse/Maven con Java 17. Il vincolo del corso richiede che il progetto sia apribile su qualsiasi PC senza riferimenti assoluti e senza servizi esterni da installare. La portabilità ha guidato:

- l'uso di H2 embedded (scope runtime): nessuna installazione di database server; le tabelle sono create dagli script init-clienti.sql, init-prodotti.sql, init-dipendenti.sql eseguiti da DBConnection;
- l'uso di Maven: mvn install scarica tutte le librerie (H2, Log4j 2, JUnit 5, Mockito);
- la configurazione della stringa JDBC in database.properties, che consente di cambiare database modificando solo un file di configurazione;
- il versionamento del file .project su GitHub, che consente di importare il progetto in Eclipse.

3.2.7 Integrità

Il sistema gestisce dati sensibili: profili dei clienti (punti fedeltà, storico acquisti), stipendi dei dipendenti e dati di pagamento. L'integrità/sicurezza ha guidato:

- il meccanismo di autenticazione centralizzato in AuthService (package sistemaaccesso), che verifica le credenziali di clienti e dipendenti; il ruolo (Cliente, Cassiere, Manager, Magazziniere, ResponsabileDiReparto) è determinato dalla classe e dal metodo getRuolo dell'interfaccia IRuoloUtente;

- la separazione in tre database H2 logici (clienti, dipendenti, prodotti): i dati dei dipendenti sono isolati da quelli dei clienti e dal catalogo (prodotti, ordini ai fornitori, righe d'ordine);
- l'uso di PreparedStatement in tutti i DAO, che previene le SQL injection;
- la verifica delle presenze con le credenziali digitate: GestorePresenze autentica email e password tramite IAutenticazione e controlla che la matricola dell'utente autenticato coincida con quella del dipendente che registra ingresso o uscita;
- il logging con Log4j 2 degli errori di inizializzazione; la tracciatura estesa di login, accessi falliti e pagamenti nelle classi di dominio è prevista.

4 DESIGN

4.1 Modelling

4.1.1 Introduzione

Il documento raccoglie i diagrammi UML comportamentali del sistema Supermercato relativi alla sessione di spesa in modalità self-scanning, che è il flusso principale del sistema: dall'autenticazione del cliente al completamento dell'acquisto, con il pagamento al totem e il controllo da parte del cassiere. I diagrammi strutturali (classi, componenti, package) sono nel capitolo 4.3, i casi d'uso nel capitolo 3.1 e le viste architetturali nel capitolo 4.2.

I diagrammi sono stati realizzati in PlantUML e aggiornati all'ultima versione del codice. Il codice PlantUML di ciascuna figura è riportato nel paragrafo 4.1.7.

4.1.2 Regole di business rappresentate

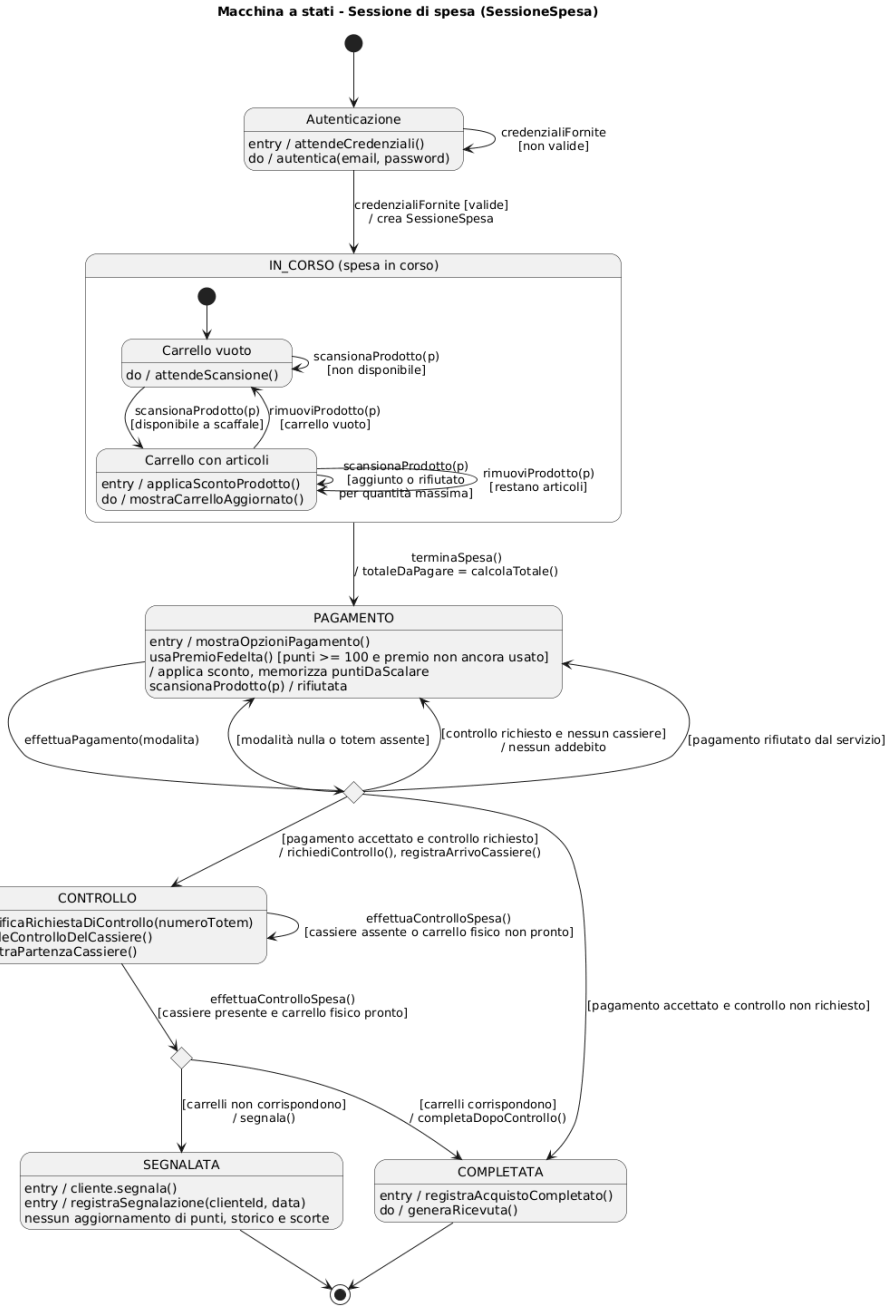
- Il controllo della spesa è richiesto se il cliente è già stato segnalato oppure con probabilità del 50%.
- Se il controllo è richiesto e nessun cassiere è disponibile al totem, il pagamento non viene addebitato e la sessione resta nello stato PAGAMENTO.
- Il premio fedeltà richiede almeno 100 punti, applica uno sconto pari a punti/20 con massimo 30% e può essere usato una sola volta per sessione; i punti sono scalati solo al completamento dell'acquisto.
- Il controllo parte quando il cassiere lo esegue con il carrello fisico disponibile; se i carrelli non corrispondono la sessione è segnalata e non vengono aggiornati punti, storico e scorte.
- Quando l'acquisto viene completato si accumulano 1 punto ogni 2 euro, si registra lo storico, si decrementa la quantità a scaffale e, sotto la soglia di 5 pezzi, si notificano gli osservatori delle scorte.

4.1.3 Diagramma della macchina a stati

La macchina a stati descrive il ciclo di vita della sessione di spesa (classe SessioneSpesa), con gli stati IN_CORSO, PAGAMENTO, CONTROLLO, COMPLETATA e SEGNALATA definiti dall'enumerazione EStatoSessione, preceduti dallo stato di autenticazione del cliente.

Elementi UML utilizzati: stato iniziale e stati finali, stato composito (IN_CORSO, con i sotto-stati Carrello vuoto e Carrello con articoli), azioni entry, do, exit, transizioni con evento, condizione di guardia e azione, pseudostati di scelta (choice) per l'esito del pagamento e del controllo, transizioni su se stesso e azioni interne.

- In IN_CORSO la scansione è accettata se il prodotto è disponibile a scaffale e la quantità nel carrello non ha raggiunto quella disponibile; la rimozione dell'ultimo articolo riporta il carrello vuoto.
- Da PAGAMENTO l'esito di effettuaPagamento dipende dalla modalità, dalla necessità del controllo e dalla disponibilità del cassiere, e dall'esito del servizio di pagamento.
- Da CONTROLLO si passa a COMPLETATA o SEGNALATA solo dopo il controllo del cassiere.

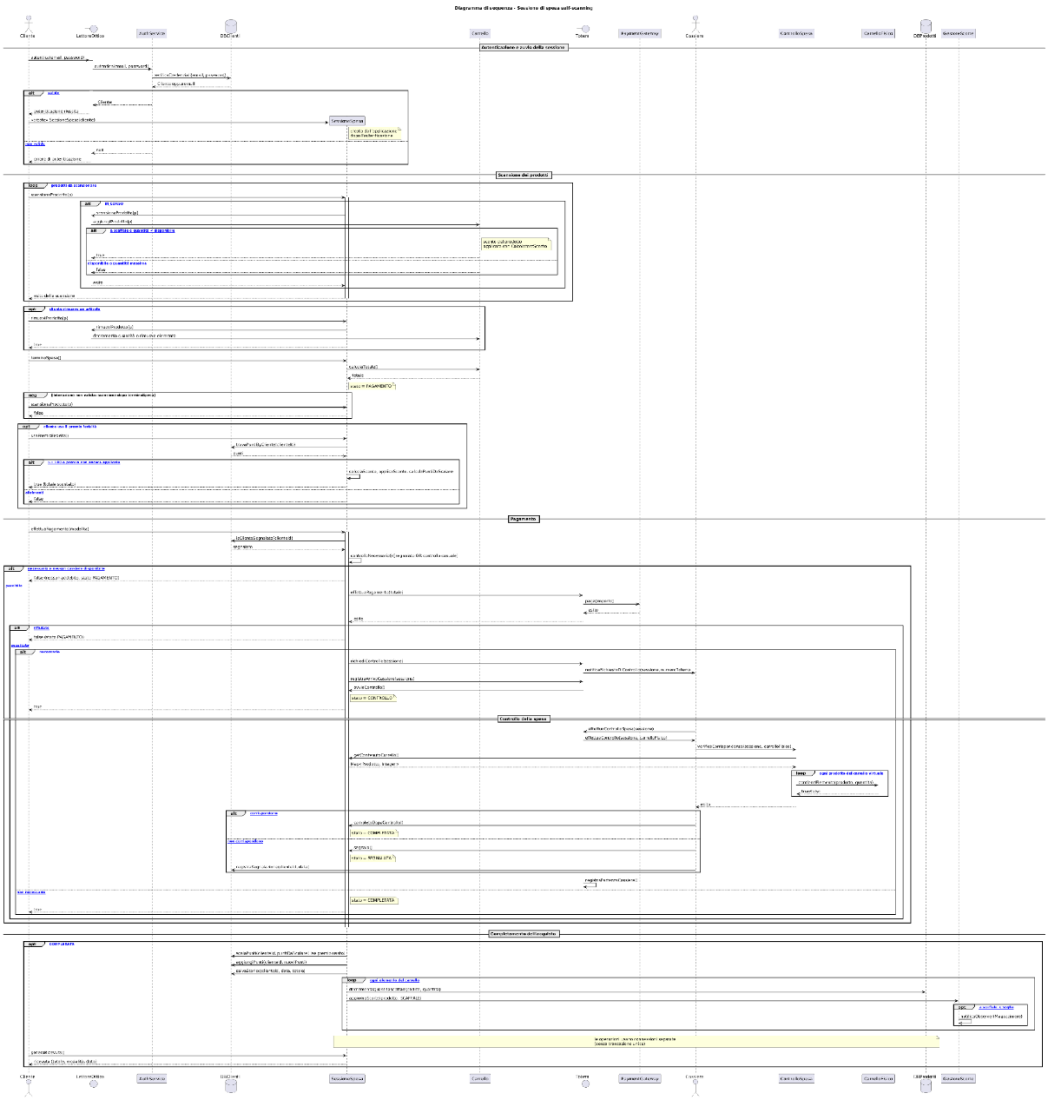


4.1.4 Diagramma di sequenza

Il diagramma di sequenza mostra, in ordine temporale, i messaggi scambiati tra il cliente, i dispositivi (lettore ottico e totem), gli oggetti del dominio e i database durante una sessione completa: autenticazione, scansione, pagamento, controllo ed eventuale segnalazione, aggiornamento di punti, storico e scorte, ricevuta.

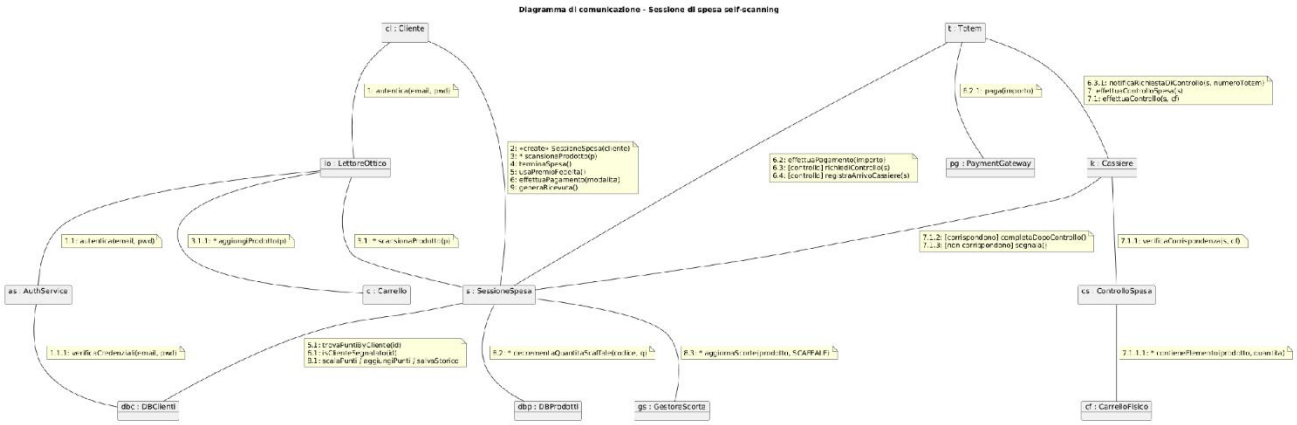
Elementi UML utilizzati: attori e partecipanti di tipo boundary e database, creazione di un oggetto, barre di attivazione, frammenti combinati alt, opt, loop e neg (interazione non valida), messaggi sincroni, di ritorno e asincroni, auto-messaggi, note e separatori di fase.

- Il frammento neg mostra l'interazione non valida di una scansione dopo terminaSpesa, che viene rifiutata.
- Il ramo di controllo include la notifica al cassiere con il numero del totem, il confronto dei carrelli tramite ControlloSpesa e CarrelloFisico, e l'esito (completamento o segnalazione).
- Le operazioni di completamento usano connessioni separate: Si tratta di un limite noto, descritto nel paragrafo 3.2.2 e richiamato dal requisito di affidabilità in 3.1.3.5.



4.1.5 Diagramma di comunicazione

Il diagramma di comunicazione rappresenta gli stessi messaggi del diagramma di sequenza organizzandoli sui collegamenti tra gli oggetti. I messaggi sono numerati con la notazione gerarchica (1, 1.1, 1.1.1, …), l'asterisco indica un messaggio iterato e le parentesi quadre una condizione.

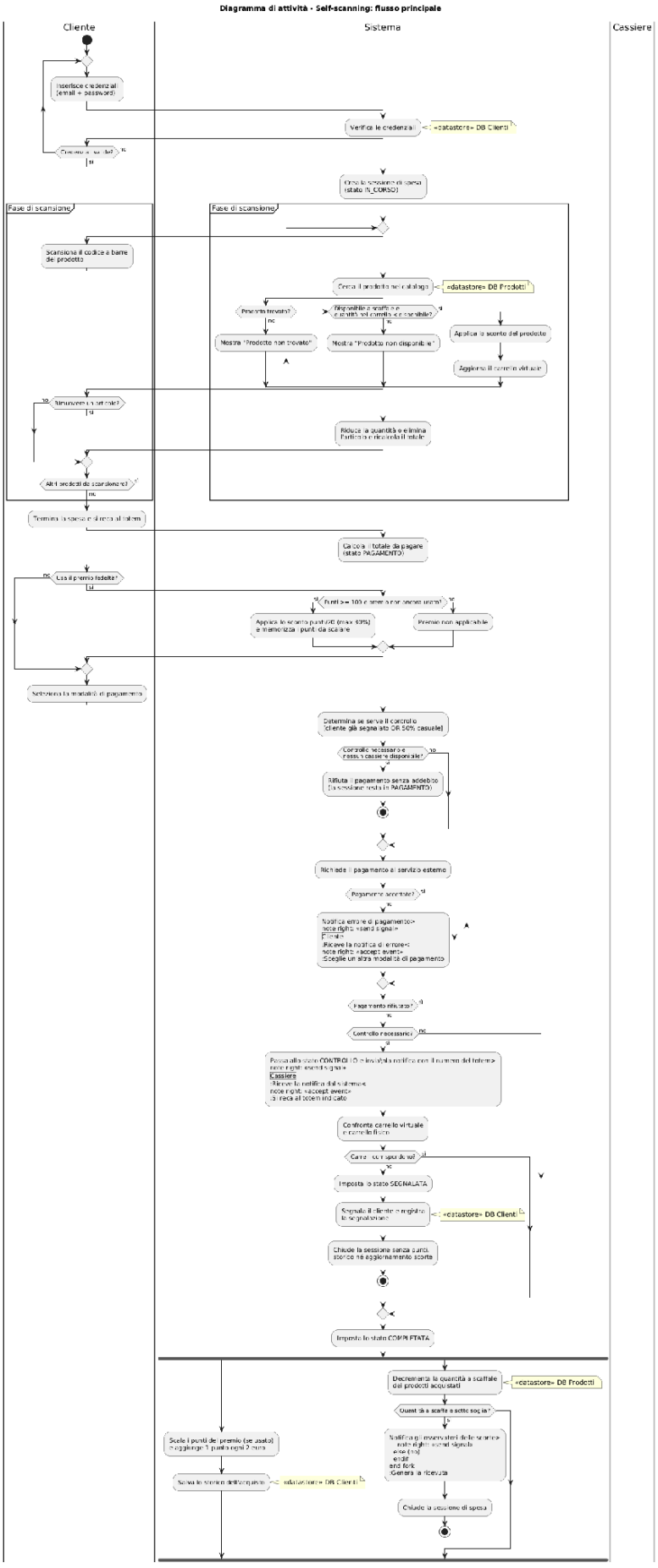


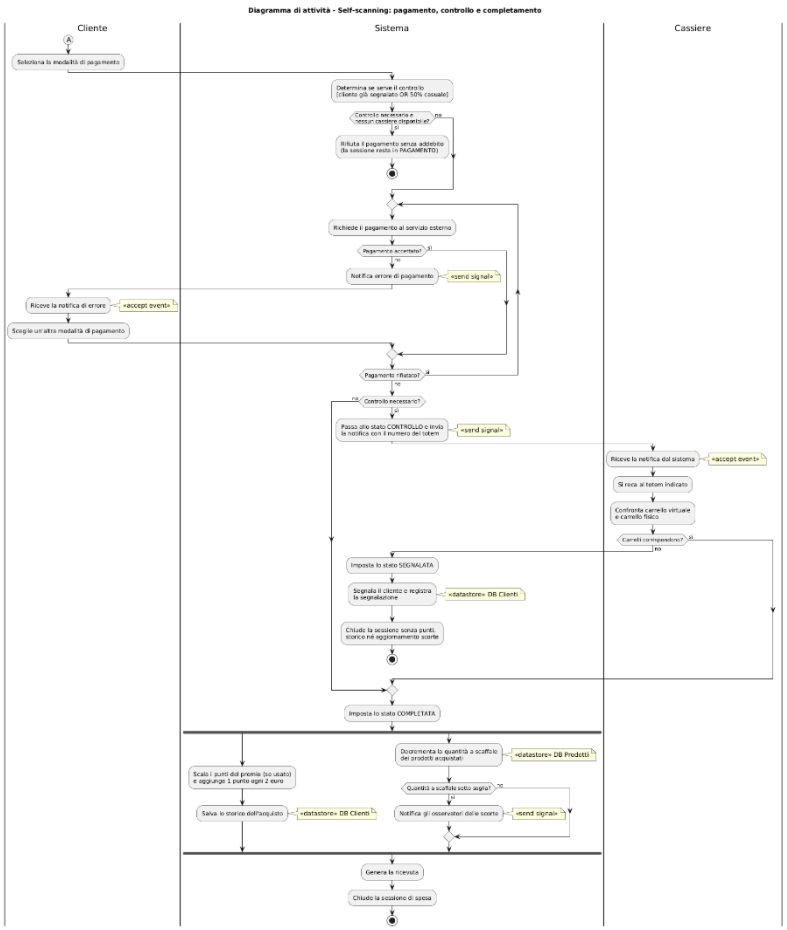
4.1.6 Diagramma di attività

Il diagramma di attività descrive il flusso di lavoro del self-scanning con tre corsie (swimlane): Cliente, Sistema e Cassiere.

Elementi UML utilizzati: nodo iniziale e finale, azioni, decisioni, cicli (repeat), biforcazione e sincronizzazione (fork e join), corsie, gruppo (fase di scansione), invio di segnale e accettazione di evento, riferimenti ai datastore.

- Un prodotto non trovato o non disponibile non interrompe il flusso: si torna alla scansione.
- Se serve il controllo e non c'è un cassiere, il pagamento è rifiutato senza addebito e il flusso termina in attesa.
- Se il pagamento è rifiutato, il cliente sceglie un'altra modalità e il sistema riprocessa il pagamento.
- Se i carrelli non corrispondono, la sessione è segnalata e chiusa senza punti, storico né aggiornamento delle scorte.
- A completamento avvenuto, l'aggiornamento di punti e storico e quello delle scorte sono rappresentati come due rami paralleli.


Diagramma di attività - Self-scanning: flusso principale
Cliente
Sistema
Cassiere
Inserisce credenziali (email + password)
Verifica le credenziali
«datastore» DB Clienti
Credenziali valide?
no
sì
Crea la sessione di spesa (stato IN_CORSO)
Fase di scansione
Fase di scansione
Scansiona il codice a barre del prodotto
Cerca il prodotto nel catalogo
«datastore» DB Prodotti
Prodotto trovato?
Disponibile a scaffale e quantità nel carrello ≤ disponibile?
sì
no
no
Mostra "Prodotto non trovato"
Mostra "Prodotto non disponibile"
Applica lo sconto del prodotto
Aggiorna il carrello del cliente
Rimuovere un articolo?
no
sì
Riduce la quantità e elimina l'articolo e ricalcola il totale
Altri articoli da scansionare?
no
Termina la spesa e chiede al totem
Calcola il totale da pagare (stato PAGAMENTO)
Usa il premio fedeltà?
no
sì
Punti ≥ 100 e spesa ≥ soglia minima spesa?
sì
no
Applica lo sconto punti/20 (max 50%) e memorizza i punti da scalare
Premio non applicabile
Seleziona la modalità di pagamento
Determina se serve il controllo cliente (già segnalato OR 50% casuale)
Controllo necessario e cassa/cassiere disponibile?
no
sì
Rifiuta il pagamento senza addebito (la sessione resta in PAGAMENTO)
Richiede il pagamento al servizio esterno
Pagamento accettato?
no
sì
Notifica errore di pagamento
note right: «card signal»
Cliente
Riceve la notifica di errore
note right: «accept event»
Sceglie un altra modalità di pagamento
Pagamento rifiutato?
no
sì
Controllo necessario
no
sì
Passa alla stato CONTROLLO e invia/stampa notifica con il numero del totem
note right: «send signal»
Cassiere
Riceve la notifica dal sistema
note right: «accept event»
si reca al totem indicato
Confronta carrello virtuale e carrello fisico
Carrello corrispondente?
no
sì
Imposta lo stato SEGNALATA
Segnala il cliente e registra la segnalazione
«datastore» DB Clienti
Chiude la sessione senza punti, storico né aggiornamento scorte
Imposta lo stato COMPLETATA
Decrementa la quantità a scaffale dei prodotti acquistati
«datastore» DB Prodotti
Quantità a scaffale sotto soglia?
sì
Notifica gli addetti alla scansione
note right: «send signal»
else [no]
endif
end fork
Genera la ricevuta
Scala i punti del premio (se usato) e aggiunge 1 punto ogni 2 euro
Salva lo storico dell'acquisto
«datastore» DB Clienti
Chiude la sessione di spesa



4.1.7 Codice PlantUML dei diagrammi

Macchina a stati della sessione di spesa

```
@startuml
title Macchina a stati – Sessione di spesa (SessioneSpesa)
skinparam shadowing false
hide empty description

[*] --> AUTH

state "Autenticazione" as AUTH
AUTH : entry / attendeCredenziali()
AUTH : do / autentica(email, password)

AUTH --> AUTH : credenzialiFornite\n[non valide]
AUTH --> SPESA : credenzialiFornite [valide]\n/ crea SessioneSpesa

state "IN_CORSO (spesa in corso)" as SPESA {
  [*] --> CV
  state "Carrello vuoto" as CV
  state "Carrello con articoli" as CA
  CV : do / attendeScansione()
  CA : entry / applicaScontoProdotto()
  CA : do / mostraCarrelloAggiornato()

  CV --> CA : scansionaProdotto(p)\n[disponibile a scaffale]
  CV --> CV : scansionaProdotto(p)\n[non disponibile]
  CA --> CA : scansionaProdotto(p)\n[aggiunto o rifiutato\nper quantità massima]
```

```
  CA --> CA : rimuoviProdotto(p)\n[restano articoli]
  CA --> CV : rimuoviProdotto(p)\n[carrello vuoto]
}

SPESA --> PAG : terminaSpesa()\n/ totaleDaPagare = calcolaTotale()

state "PAGAMENTO" as PAG
PAG : entry / mostraOpzioniPagamento()
PAG : usaPremioFedelta() [punti >= 100 e premio non ancora usato]\n/ applica sconto,
memorizza puntiDaScalare
PAG : scansionaProdotto(p) / rifiutata

state sceltaPag <<choice>>
PAG --> sceltaPag : effettuaPagamento(modalita)
sceltaPag --> PAG : [modalità nulla o totem assente]
sceltaPag --> PAG : [controllo richiesto e nessun cassiere]\n/ nessun addebito
sceltaPag --> PAG : [pagamento rifiutato dal servizio]
sceltaPag --> CTRL : [pagamento accettato e controllo richiesto]\n/ richiediControllo(),
registraArrivoCassiere()
sceltaPag --> COMP : [pagamento accettato e controllo non richiesto]

state "CONTROLLO" as CTRL
CTRL : entry / notificaRichiestaDiControllo(numeroTotem)
CTRL : do / attendeControlloDelCassiere()
CTRL : exit / registraPartenzaCassiere()

state esito <<choice>>
CTRL --> CTRL : effettuaControlloSpesa()\n[cassiere assente o carrello fisico non pronto]
CTRL --> esito : effettuaControlloSpesa()\n[cassiere presente e carrello fisico pronto]
esito --> COMP : [carrelli corrispondono]\n/ completaDopoControllo()
esito --> SEG : [carrelli non corrispondono]\n/ segnala()

state "COMPLETATA" as COMP
COMP : entry / registraAcquistoCompletato()
COMP : do / generaRicevuta()
COMP --> [*]

state "SEGNALATA" as SEG
SEG : entry / cliente.segnala()
SEG : entry / registraSegnalazione(clienteId, data)
SEG : nessun aggiornamento di punti, storico e scorte
SEG --> [*]
@enduml
```

Diagramma di sequenza della sessione di spesa

```
@startuml
title Diagramma di sequenza – Sessione di spesa self-scanning
skinparam sequenceArrowThickness 1.5
skinparam shadowing false

actor Cliente
boundary "LettoreOttico" as LO
participant "AuthService" as AS
database "DBClienti" as DBC
participant "SessioneSpesa" as SS
participant "Carrello" as CA
boundary "Totem" as T
participant "PaymentGateway" as PG
actor Cassiere as K
participant "ControlloSpesa" as CS
participant "CarrelloFisico" as CF
database "DBProdotti" as DBP
participant "GestoreScorte" as GS

== Autenticazione e avvio della sessione ==
```

```
Cliente -> LO : autentica(email, password)
activate LO
LO -> AS : autentica(email, password)
AS -> DBC : verificaCredenziali(email, password)
DBC --> AS : Cliente oppure null
alt [credenziali valide]
  AS --> LO : Cliente
  LO --> Cliente : autenticazione riuscita
  deactivate LO
  create SS
  Cliente -> SS : <<create>> SessioneSpesa(cliente)
  note right of SS : creata dall'applicazione\ndopo l'autenticazione
else [credenziali non valide]
  AS --> LO : null
  LO --> Cliente : errore di autenticazione
  deactivate LO
end

== Scansione dei prodotti ==
loop [altri prodotti da scansionare]
  Cliente -> SS : scansionaProdotto(p)
  activate SS
  alt [stato IN_CORSO]
    SS -> LO : scansionaProdotto(p)
    LO -> CA : aggiungiProdotto(p)
    alt [disponibile a scaffale e quantità < disponibile]
      note right of CA : sconto del prodotto\napplicato con CalcolatoreSconto
      CA --> LO : true
    else [non disponibile o quantità massima]
      CA --> LO : false
    end
    LO --> SS : esito
  end
  SS --> Cliente : esito della scansione
  deactivate SS
end

opt [il cliente rimuove un articolo]
  Cliente -> SS : rimuoviProdotto(p)
  SS -> LO : rimuoviProdotto(p)
  LO -> CA : decrementa quantità o rimuove elemento
  SS --> Cliente : true
end

Cliente -> SS : terminaSpesa()
SS -> CA : calcolaTotale()
CA --> SS : totale
note right of SS : stato = PAGAMENTO

group neg [Interazione non valida: scansione dopo terminaSpesa]
  Cliente -> SS : scansionaProdotto(p)
  SS --> Cliente : false
end

opt [il cliente usa il premio fedeltà]
  Cliente -> SS : usaPremioFedelta()
  SS -> DBC : trovaPuntiByCliente(clienteId)
  DBC --> SS : punti
  alt [punti >= 100 e premio non ancora applicato]
    SS -> SS : calcolaSconto, applicaSconto, calcolaPuntiDaScalare
    SS --> Cliente : true (totale scontato)
  else [altrimenti]
    SS --> Cliente : false
  end
end

== Pagamento ==
```

```
Cliente -> SS : effettuaPagamento(modalita)
activate SS
SS -> DBC : isClienteSegnalato(clienteId)
DBC --> SS : segnalato
SS -> SS : controlloNecessario() [segnalato OR controllo casuale]
alt [controllo necessario e nessun cassiere disponibile]
  SS --> Cliente : false (nessun addebito, stato PAGAMENTO)
else [pagamento possibile]
  SS -> T : effettuaPagamento(totale)
  T -> PG : paga(importo)
  PG --> T : esito
  T --> SS : esito
  alt [pagamento rifiutato]
    SS --> Cliente : false (stato PAGAMENTO)
  else [pagamento accettato]
    alt [controllo necessario]
      SS -> T : richiediControllo(sessione)
      T -> K : notificaRichiestaDiControllo(sessione, numeroTotem)
      SS -> T : registraArrivoCassiere(sessione)
      T -> SS : avviaControllo()
      note right of SS : stato = CONTROLLO
      SS --> Cliente : true

      == Controllo della spesa ==
      K -> T : effettuaControlloSpesa(sessione)
      T -> K : effettuaControllo(sessione, carrelloFisico)
      K -> CS : verificaCorrispondenza(sessione, carrelloFisico)
      CS -> SS : getContenutoCarrello()
      SS --> CS : Map<Prodotto, Integer>
      loop [per ogni prodotto del carrello virtuale]
        CS -> CF : contieneElemento(prodotto, quantita)
        CF --> CS : true/false
      end
      CS --> K : esito
      alt [carrelli corrispondono]
        K -> SS : completaDopoControllo()
        note right of SS : stato = COMPLETATA
      else [carrelli non corrispondono]
        K -> SS : segnala()
        note right of SS : stato = SEGNALATA
        K -> DBC : registraSegnalazione(clienteId, data)
      end
      T -> T : registraPartenzaCassiere()
    else [controllo non necessario]
      note right of SS : stato = COMPLETATA
      SS --> Cliente : true
    end
  end
end
deactivate SS

== Completamento dell'acquisto ==
opt [sessione COMPLETATA]
  SS -> DBC : scalaPunti(clienteId, puntiDaScalare) [se premio usato]
  SS -> DBC : aggiungiPunti(clienteId, nuoviPunti)
  SS -> DBC : salvaStorico(clienteId, data, totale)
  loop [per ogni elemento del carrello]
    SS -> DBP : decrementaQuantitaScaffale(codice, quantita)
    SS -> GS : aggiornaScorte(prodotto, SCAFFALE)
    opt [quantità a scaffale < soglia]
      GS ->> GS : notificaObserver(Magazziniere)
    end
  end
  note over SS, DBP : le operazioni usano connessioni separate\n(senza transazione unica)
  Cliente -> SS : generaRicevuta()
  SS --> Cliente : ricevuta (totale, modalità, data)
end
@enduml
```

Diagramma di comunicazione della sessione di spesa

```
@startuml
title Diagramma di comunicazione – Sessione di spesa self-scanning
skinparam shadowing false
skinparam objectStyle rectangle
skinparam nodesep 100
skinparam ranksep 120

together {
  object "cl : Cliente" as CL
  object "lo : LettoreOttico" as LO
}
object "as : AuthService" as AS
object "dbc : DBClienti" as DBC
together {
  object "s : SessioneSpesa" as S
  object "c : Carrello" as C
}
together {
  object "t : Totem" as T
  object "pg : PaymentGateway" as PG
}
together {
  object "k : Cassiere" as K
  object "cs : ControlloSpesa" as CS
  object "cf : CarrelloFisico" as CF
}
together {
  object "dbp : DBProdotti" as DBP
  object "gs : GestoreScorte" as GS
}

CL -- LO
note on link
  1: autentica(email, pwd)
end note

LO -- AS
note on link
  1.1: autentica(email, pwd)
end note

AS -- DBC
note on link
  1.1.1: verificaCredenziali(email, pwd)
end note

CL -- S
note on link
  2: <<create>> SessioneSpesa(cliente)
  3: * scansionaProdotto(p)
  4: terminaSpesa()
  5: usaPremioFedelta()
  6: effettuaPagamento(modalita)
  9: generaRicevuta()
end note

S -- LO
note on link
  3.1: * scansionaProdotto(p)
end note

LO -- C
note on link
  3.1.1: * aggiungiProdotto(p)
end note
```

```
S -- DBC
note on link
  5.1: trovaPuntiByCliente(id)
  6.1: isClienteSegnalato(id)
  8.1: scalaPunti / aggiungiPunti / salvaStorico
end note

S -- T
note on link
  6.2: effettuaPagamento(importo)
  6.3: [controllo] richiediControllo(s)
  6.4: [controllo] registraArrivoCassiere(s)
end note

T -- PG
note on link
  6.2.1: paga(importo)
end note

T -- K
note on link
  6.3.1: notificaRichiestaDiControllo(s, numeroTotem)
  7: effettuaControlloSpesa(s)
  7.1: effettuaControllo(s, cf)
end note

K -- CS
note on link
  7.1.1: verificaCorrispondenza(s, cf)
end note

CS -- CF
note on link
  7.1.1.1: * contieneElemento(prodotto, quantita)
end note

K -- S
note on link
  7.1.2: [corrispondono] completaDopoControllo()
  7.1.3: [non corrispondono] segnala()
end note

S -- DBP
note on link
  8.2: * decrementaQuantitaScaffale(codice, q)
end note

S -- GS
note on link
  8.3: * aggiornaScorte(prodotto, SCAFFALE)
end note
@enduml
```

Diagramma di attività del self-scanning: flusso principale

```
@startuml
title Diagramma di attività – Self-scanning: flusso principale
skinparam shadowing false
skinparam defaultFontName Arial

|Cliente|
|Sistema|
|Cassiere|

|Cliente|
```

```
start
repeat
  :Inserisce credenziali\n(email + password);
  |Sistema|
  :Verifica le credenziali;
  note right: <<datastore>> DB Clienti
  |Cliente|
repeat while (Credenziali valide?) is (no) not (sì)

|Sistema|
:Crea la sessione di spesa\n(stato IN_CORSO);

group Fase di scansione {
  repeat
    |Cliente|
    :Scansiona il codice a barre\ndel prodotto;
    |Sistema|
    :Cerca il prodotto nel catalogo;
    note right: <<datastore>> DB Prodotti
    if (Prodotto trovato?) then (no)
      :Mostra "Prodotto non trovato";
    elseif (Disponibile a scaffale e\nquantità nel carrello < disponibile?) then (no)
      :Mostra "Prodotto non disponibile";
    else (sì)
      :Applica lo sconto del prodotto;
      :Aggiorna il carrello virtuale;
    endif
    |Cliente|
    if (Rimuovere un articolo?) then (sì)
      |Sistema|
      :Riduce la quantità o elimina\nl'articolo e ricalcola il totale;
      |Cliente|
    else (no)
    endif
  repeat while (Altri prodotti da scansionare?) is (sì) not (no)
}

:Termina la spesa e si reca al totem;
|Sistema|
:Calcola il totale da pagare\n(stato PAGAMENTO);

|Cliente|
if (Usa il premio fedeltà?) then (sì)
  |Sistema|
  if (Punti >= 100 e premio non ancora usato?) then (sì)
    :Applica lo sconto punti/20 (max 30%)\ne memorizza i punti da scalare;
  else (no)
    :Premio non applicabile;
  endif
else (no)
endif

|Cliente|
:Seleziona la modalità di pagamento;

|Sistema|
:Determina se serve il controllo\n[cliente già segnalato OR 50% casuale];
if (Controllo necessario e\nnessun cassiere disponibile?) then (sì)
  :Rifiuta il pagamento senza addebito\n(la sessione resta in PAGAMENTO);
  stop
else (no)
endif

repeat
  :Richiede il pagamento al servizio esterno;
  if (Pagamento accettato?) then (no)
    :Notifica errore di pagamento>
```

```
    note right: <<send signal>>
    |Cliente|
    :Riceve la notifica di errore<
    note right: <<accept event>>
    :Sceglie un'altra modalità di pagamento;
    |Sistema|
  else (sì)
  endif
repeat while (Pagamento rifiutato?) is (sì) not (no)

if (Controllo necessario?) then (sì)
  :Passa allo stato CONTROLLO e invia\nla notifica con il numero del totem>
  note right: <<send signal>>
  |Cassiere|
  :Riceve la notifica dal sistema<
  note right: <<accept event>>
  :Si reca al totem indicato;
  :Confronta carrello virtuale\ne carrello fisico;
  if (Carrelli corrispondono?) then (no)
    |Sistema|
    :Imposta lo stato SEGNALATA;
    :Segnala il cliente e registra\nla segnalazione;
    note right: <<datastore>> DB Clienti
    :Chiude la sessione senza punti,\nstorico né aggiornamento scorte;
    stop
  else (sì)
    |Sistema|
  endif
else (no)
endif

:Imposta lo stato COMPLETATA;
fork
  :Scala i punti del premio (se usato)\ne aggiunge 1 punto ogni 2 euro;
  :Salva lo storico dell'acquisto;
  note right: <<datastore>> DB Clienti
fork again
  :Decrementa la quantità a scaffale\ndei prodotti acquistati;
  note right: <<datastore>> DB Prodotti
  if (Quantità a scaffale sotto soglia?) then (sì)
    :Notifica gli osservatori delle scorte>
    note right: <<send signal>>
  else (no)
  endif
end fork
:Genera la ricevuta;
:Chiude la sessione di spesa;
stop
@enduml
```

Diagramma di attività del self-scanning: pagamento, controllo e completamento

```
@startuml
title Diagramma di attività – Self-scanning: pagamento, controllo e completamento
skinparam shadowing false
skinparam defaultFontName Arial

|Cliente|
|Sistema|
|Cassiere|

|Cliente|
(A)
:Seleziona la modalità di pagamento;

|Sistema|
```

```
:Determina se serve il controllo\n[cliente già segnalato OR 50% casuale];
if (Controllo necessario e\nnessun cassiere disponibile?) then (sì)
  :Rifiuta il pagamento senza addebito\n(la sessione resta in PAGAMENTO);
  stop
else (no)
endif

repeat
  :Richiede il pagamento al servizio esterno;
  if (Pagamento accettato?) then (no)
    :Notifica errore di pagamento;
    note right: <<send signal>>
    |Cliente|
    :Riceve la notifica di errore;
    note right: <<accept event>>
    :Sceglie un'altra modalità di pagamento;
    |Sistema|
  else (sì)
  endif
repeat while (Pagamento rifiutato?) is (sì) not (no)

if (Controllo necessario?) then (sì)
  :Passa allo stato CONTROLLO e invia\nla notifica con il numero del totem;
  note right: <<send signal>>
  |Cassiere|
  :Riceve la notifica dal sistema;
  note right: <<accept event>>
  :Si reca al totem indicato;
  :Confronta carrello virtuale\ne carrello fisico;
  if (Carrelli corrispondono?) then (no)
    |Sistema|
    :Imposta lo stato SEGNALATA;
    :Segnala il cliente e registra\nla segnalazione;
    note right: <<datastore>> DB Clienti
    :Chiude la sessione senza punti,\nstorico né aggiornamento scorte;
    stop
  else (sì)
    |Sistema|
  endif
else (no)
endif

:Imposta lo stato COMPLETATA;
fork
  :Scala i punti del premio (se usato)\ne aggiunge 1 punto ogni 2 euro;
  :Salva lo storico dell'acquisto;
  note right: <<datastore>> DB Clienti
fork again
  :Decrementa la quantità a scaffale\ndei prodotti acquistati;
  note right: <<datastore>> DB Prodotti
  if (Quantità a scaffale sotto soglia?) then (sì)
    :Notifica gli osservatori delle scorte;
    note right: <<send signal>>
  else (no)
  endif
end fork
:Genera la ricevuta;
:Chiude la sessione di spesa;
stop
@enduml
```

4.2 Software Architecture

L'architettura è descritta con tre viste per punti di vista differenti: la vista del modulo (decomposizione, dipendenze e strati), la vista dei componenti e connettori (C&C) e la vista di allocazione. Il component diagram e il package diagram completi sono riportati nel capitolo 4.3. Il codice PlantUML di tutte le figure è riportato nel paragrafo 4.2.5.

4.2.1 Stile architetturale

Il sistema adotta uno stile ibrido Repository + Layered, con un meccanismo di invocazione implicita basata su eventi (pattern Observer) per le scorte.

- **Repository**: i dati persistenti sono conservati in tre database H2 condivisi (clienti, dipendenti, prodotti) e acceduti dai componenti di business tramite i DAO; il database è il corpo centrale di informazioni e i componenti non comunicano direttamente tra loro per scambiare dati persistenti.
- **Layered**: i package sono organizzati in tre strati (Presentation, Business Logic, Data); uno strato può usare solo quelli sottostanti, e le dipendenze verso il basso passano da interfacce possedute dal dominio (Dependency Inversion Principle). Questo isola la logica di business dai dettagli JDBC e rende possibile sostituire i DAO nei test.
- **Invocazione implicita**: GestoreScorte notifica gli IScorteObserver (Magazziniere e ResponsabileDiReparto) senza conoscerli; gli osservatori possono essere aggiunti o rimossi a runtime.

Librerie esterne gestite con Maven, con Java 17: H2 (database embedded), Log4j (logging), JUnit 5 e Mockito 5 (test).

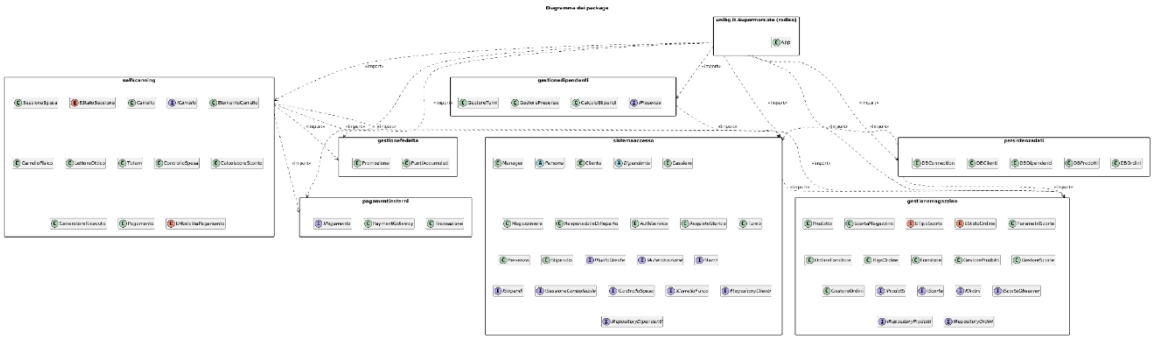
4.2.2 Vista del modulo

4.2.2.1 Punto di vista 1 - Decomposizione

Definizione: gli elementi sono correlati dalla relazione "sotto modulo di"; gli elementi più grandi vengono scomposti in parti più piccole.

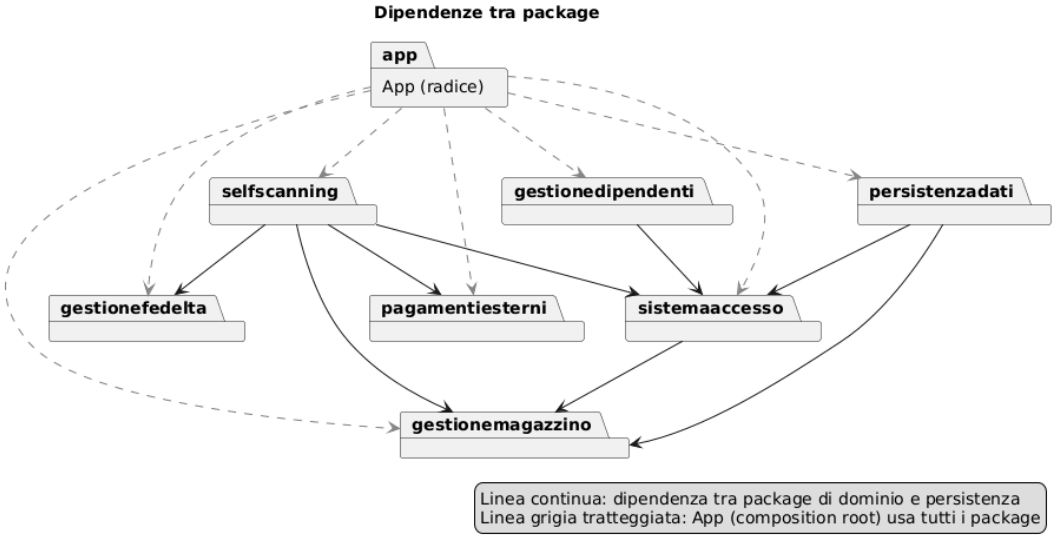
Stakeholder di riferimento: gli sviluppatori sanno in quale package creare una nuova classe; il Project Manager organizza il lavoro dividendo i task per package; i manutentori capiscono dove intervenire per una modifica funzionale.

Motivazione della scelta: il sistema ha sei aree funzionali distinte (accesso, self-scanning, fedeltà, magazzino, dipendenti, pagamenti) più un package trasversale di persistenza. La Decomposizione mostra questa scomposizione top-down in sette package Java, che corrispondono 1:1 alle directory Maven (src/main/java/unibg/it/Supermercato/<package>/). La classe App, che contiene l'applicazione dimostrativa, si trova nel package radice unibg.it.Supermercato.



4.2.2.2 Punto di vista 2 - Uses (dipendenze)

La relazione "usa" tra package è aciclica e segue il Dependency Inversion Principle: il dominio possiede le interfacce (porte) di cui ha bisogno e persistenzadati le implementa. Nessun package di dominio importa persistenzadati. L'assenza di cicli è verificabile con JDepend.



Soluzioni adottate per rendere il grafo aciclico:

- le interfacce IRepository* si trovano nel package di dominio proprietario; i parametri sono tipi del dominio o valori semplici (ad esempio salvaStorico riceve identificativo, data e totale, e registraSegnalazione identificativo e data), così persistenzadati non importa classi di selfscanning;
- Cassiere dipende solo da porte definite in sistemaaccesso: ISessioneControllabile (implementata da SessioneSpesa; espone stato, cliente, data e contenuto del carrello), IControlloSpesa (implementata da ControlloSpesa) e ICarrelloFisico (implementata da CarrelloFisico); Totem usa Cassiere, quindi la dipendenza va solo da selfscanning a sistemaaccesso;
- Manager usa le porte ITurni e IStipendi, implementate da GestoreTurni e CalcoloStipendi in gestionedipendenti, e riceve le implementazioni per iniezione;

- i ruoli letti dal database sono completati senza che persistenzadati conosca gestionedipendenti o selfscanning: DBDipendenti espone un configuratore di ruoli (Consumer<Dipendente>) impostato dal composition root, che collega a ogni ruolo i suoi collaboratori;
- Turno, Presenza, Stipendio e AcquistoStorico risiedono in sistemaaccesso, così Dipendente e Cliente non dipendono da altri package di dominio;
- Cliente e SessioneSpesa usano soltanto le interfacce dei repository, iniettate dall'esterno; GestoreProdotti e GestoreOrdini ricevono il repository nel costruttore.

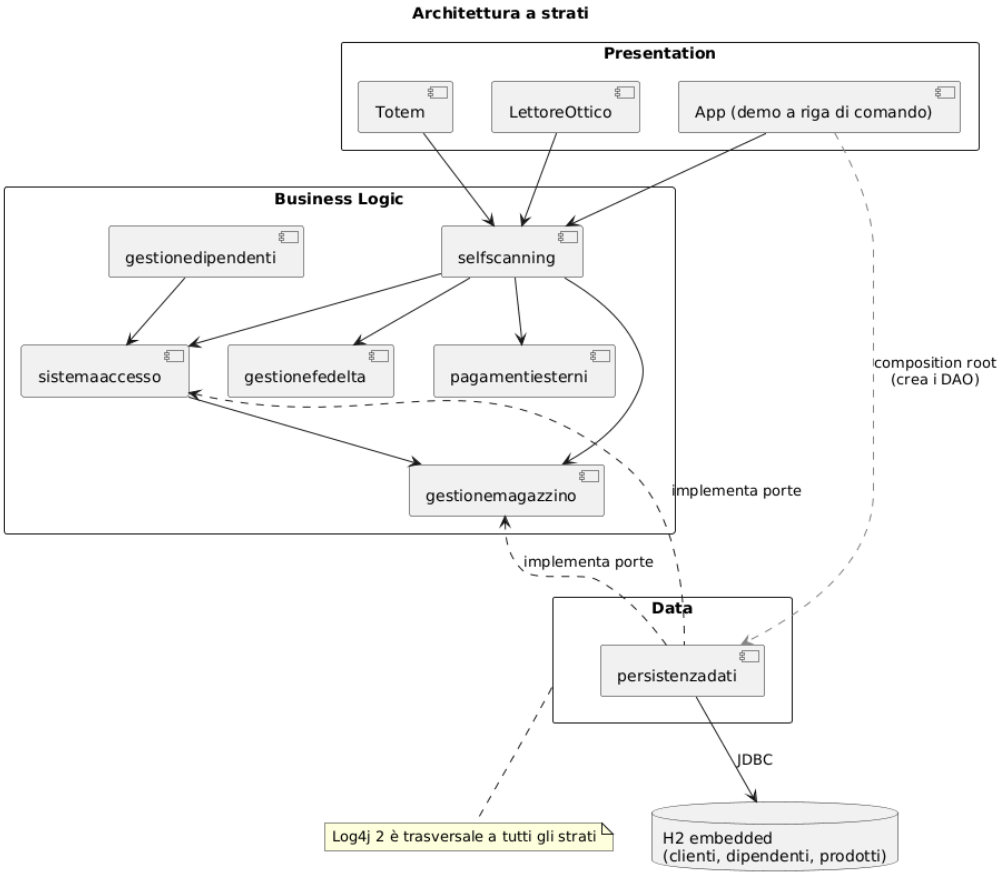
4.2.2.3 Punto di vista 3 – Stile a strati (Layered)

Uno strato può usare solo gli strati sottostanti: Presentation → Business Logic → Data.

- **Presentation Layer**: nel prototipo l'interazione con gli utenti è simulata da un'applicazione dimostrativa a riga di comando (classe App); non è prevista un'interfaccia grafica. LettoreOttico e Totem rappresentano i dispositivi fisici verso il cliente.
- **Business Logic Layer**: sistemaaccesso, selfscanning, gestionefedelta, gestionemagazzino, gestionedipendenti, pagamentiesterni.
- **Data Layer**: persistenzadati, che centralizza l'accesso a H2 tramite JDBC (DBConnection) e implementa i DAO.

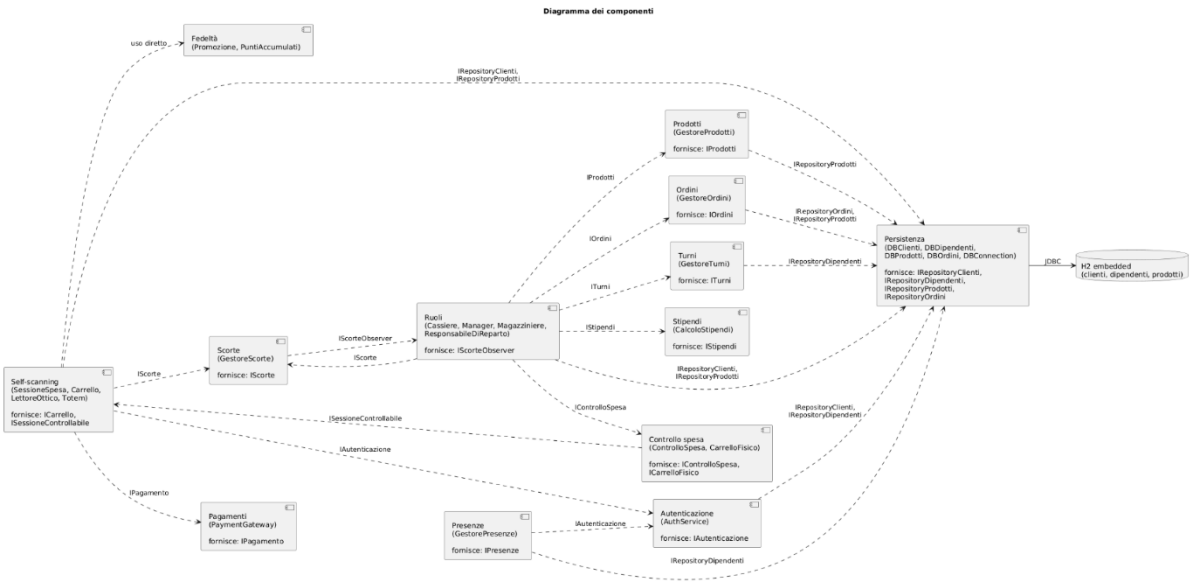
Il vincolo "mai viceversa" è verificabile nel codice: nessuna classe di dominio importa persistenzadati; i DAO importano solo classi del dominio che implementano porte; il Totem non conosce i DAO. Il logging con Log4j 2 è trasversale a tutti gli strati.

Strato	Macchina virtuale offerta	Verso il basso
Presentation	Applicazione dimostrativa e dispositivi (lettore ottico, totem)	Chiama la business logic
Business Logic	Regole di dominio: sessione, fedeltà, scorte, turni, presenze, stipendi, ruoli	Usa le porte IRepository* e IPagamento
Data	Lettura/scrittura su H2 via JDBC; non conosce regole di business	H2 embedded



4.2.3. Vista C&C – Componenti e Connettori

Stile ibrido Repository + Layered. Il corpo centrale di informazioni è costituito da tre database H2 embedded (componenti di tipo Memoria), acceduti dai DAO tramite JDBC; la business logic è organizzata a strati.



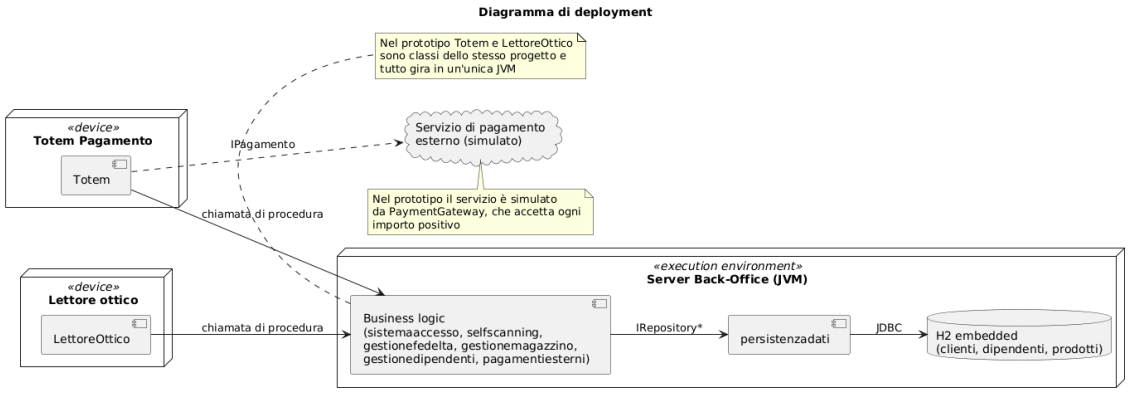
Componente	Tipo	Responsabilità
clienti (H2)	Memoria	Anagrafica clienti, punti fedeltà, storico spese, segnalazioni
dipendenti (H2)	Memoria	Profili dipendenti, ruolo, turni e presenze
prodotti (H2)	Memoria	Prodotti, giacenze a scaffale e in magazzino, ordini ai fornitori (con stato) e righe d'ordine
AuthService	Gestore	Registrazione e autenticazione di clienti e dipendenti
SessioneSpesa	Gestore	Stato della sessione di spesa (macchina a stati), pagamento, controllo, ricevuta, aggiornamento di punti, storico e scorte
GestoreProdotti, GestoreScorte, GestoreOrdini	Gestore	Catalogo, scorte (notifica agli osservatori) e riordino ai fornitori con ricezione idempotente
GestoreTurni, GestorePresenze	Gestore	Assegnazione turni (validi e senza sovrapposizioni) e registrazione di ingresso e uscita con verifica delle credenziali
Promozione, PuntiAccumulati, CalcolatoreSconto, CalcoloStipendi, Presenza, ControlloSpesa, ParametriScorte	Computazionale	Calcoli su sconti, punti, ore lavorate, stipendi, confronto carrello/spesa fisica e parametri di soglia delle scorte
LettoreOttico, Totem	Boundary	Interazione con il cliente e con il cassiere; il Totem coordina pagamento e controllo
PaymentGateway	Boundary esterno	Servizio di pagamento esterno (simulato) dietro IPagamento
Transazione	Dato (entità)	Importo e stato di completamento di un pagamento; il completamento è consentito solo per importi positivi; nel prototipo non è ancora utilizzata da PaymentGateway
DBClienti, DBDipendenti, DBProdotti, DBOrdini	Accesso ai dati	Implementano le porte IRepository* su H2 via JDBC
Logger (Log4j 2)	Computazionale	Registra l'avvio, l'inizializzazione dei database e gli errori; la registrazione di login, pagamenti e segnalazioni è prevista

Connettori:

- Dati condivisi / JDBC: i DAO accedono ai database H2.
- Chiamata di procedura: invocazione esplicita tra componenti (ad esempio SessioneSpesa → Totem → Cassiere, Totem → IPagamento).
- Invocazione implicita: (1) pattern Observer sulle scorte (IScorteObserver): Magazziniere e ResponsabileDiReparto sono notificati da GestoreScorte senza che questo li conosca; (2) il logger Log4j 2 è invocato dai componenti senza influenzarne il flusso.

4.2.4. Vista di allocazione

4.2.4.1 Deployment



Nodo	Componenti	Note
Server Back-Office (JVM)	Tutti i package di business logic, persistenzadati e H2 embedded	Unica JVM nel prototipo; in produzione sarebbe un application server dedicato
Totem Pagamento	Totem, integrazione tramite IPagamento	Il pagamento esterno è simulato nel prototipo
Lettore ottico	LettoreOttico	Dispositivo del cliente per la scansione
H2 embedded	clienti, dipendenti, prodotti	Connettore JDBC; nessuna installazione di server

4.2.4.2 Implementazione (struttura del repository)

Il codice è separato dai documenti; il progetto Eclipse/Maven non contiene riferimenti assoluti ai file del computer.

Elemento	Percorso	Contenuto
Documenti	documenti/	Documenti di progetto
Progetto Eclipse/Maven	Supermercato/	pom.xml, .project, .classpath, .pmd, .ruleset, .gitignore
Package Java	src/main/java/unibg/it/Supermercato/<package>/	Sette package corrispondenti alla vista di decomposizione; la classe App nel package radice
Risorse	src/main/resources/	database.properties, database-test.properties, init-clienti.sql, init-dipendenti.sql, init-prodotti.sql, configurazione Log4j 2

Test	src/test/java/unibg/it/Supermercato/<package>/	Test JUnit 5 e Mockito, organizzati per package; i test di scenario sono nel package radice
Build	pom.xml	Java 17, H2, Log4j 2, JUnit 5, Mockito; plugin Surefire
Qualità del codice	Supermercato/.pmd, .ruleset	Regole PMD con violazioni trattate come errori

4.2.5 – Codice PlantUML dei diagrammi

Diagramma dei package

```
@startuml
title Diagramma dei package
skinparam packageStyle rectangle
skinparam shadowing false
hide empty members

package "unibg.it.Supermercato (radice)" as app {
  class App
}

package "sistemaaccesso" as sa {
  abstract class Persona
  class Cliente
  abstract class Dipendente
  class Cassiere
  class Manager
  class Magazziniere
  class ResponsabileDiReparto
  class AuthService
  class AcquistoStorico
  class Turno
  class Presenza
  class Stipendio
  interface IRuoloUtente
  interface IAutenticazione
  interface ITurni
  interface IStipendi
  interface ISessioneControllabile
  interface IControlloSpesa
  interface ICarrelloFisico
  interface IRepositoryClienti
  interface IRepositoryDipendenti
}

package "selfscanning" as ss {
  class SessioneSpesa
  enum EStatoSessione
  class Carrello
  interface ICarrello
  class ElementoCarrello
  class CarrelloFisico
  class LettoreOttico
  class Totem
  class ControlloSpesa
  class CalcolatoreSconto
  class GeneratoreRicevuta
  class Pagamento
```

```
  enum EModalitaPagamento
}

package "gestionefedelta" as gf {
  class Promozione
  class PuntiAccumulati
}

package "gestionemagazzino" as gm {
  class Prodotto
  class ScortaMagazzino
  enum ETipoScorta
  enum EStatoOrdine
  class ParametriScorte
  class OrdineFornitore
  class RigaOrdine
  class Fornitore
  class GestoreProdotti
  class GestoreScorte
  class GestoreOrdini
  interface IProdotti
  interface IScorte
  interface IOrdini
  interface IScorteObserver
  interface IRepositoryProdotti
  interface IRepositoryOrdini
}

package "gestionedipendenti" as gd {
  class GestoreTurni
  class GestorePresenze
  class CalcoloStipendi
  interface IPresenze
}

package "pagamentiesterni" as pe {
  interface IPagamento
  class PaymentGateway
  class Transazione
}

package "persistenzadati" as pd {
  class DBConnection
  class DBClienti
  class DBDipendenti
  class DBProdotti
  class DBOrdini
}

' --- Disposizione: sistemaaccesso (righe da 5) ---
Persona -[hidden]right- Cliente
Cliente -[hidden]right- Dipendente
Dipendente -[hidden]right- Cassiere
Cassiere -[hidden]right- Manager
Magazziniere -[hidden]right- ResponsabileDiReparto
ResponsabileDiReparto -[hidden]right- AuthService
AuthService -[hidden]right- AcquistoStorico
AcquistoStorico -[hidden]right- Turno
Presenza -[hidden]right- Stipendio
Stipendio -[hidden]right- IRuoloUtente
IRuoloUtente -[hidden]right- IAutenticazione
IAutenticazione -[hidden]right- ITurni
IStipendi -[hidden]right- ISessioneControllabile
ISessioneControllabile -[hidden]right- IControlloSpesa
IControlloSpesa -[hidden]right- ICarrelloFisico
ICarrelloFisico -[hidden]right- IRepositoryClienti
Persona -[hidden]down- Magazziniere
```

```
Magazziniere -[hidden]down- Presenza
Presenza -[hidden]down- IStipendi
IStipendi -[hidden]down- IRepositoryDipendenti

' --- Disposizione: selfscanning ---
SessioneSpesa -[hidden]right- EStatoSessione
EStatoSessione -[hidden]right- Carrello
Carrello -[hidden]right- ICarrello
ICarrello -[hidden]right- ElementoCarrello
CarrelloFisico -[hidden]right- LettoreOttico
LettoreOttico -[hidden]right- Totem
Totem -[hidden]right- ControlloSpesa
ControlloSpesa -[hidden]right- CalcolatoreSconto
GeneratoreRicevuta -[hidden]right- Pagamento
Pagamento -[hidden]right- EModalitaPagamento
SessioneSpesa -[hidden]down- CarrelloFisico
CarrelloFisico -[hidden]down- GeneratoreRicevuta

' --- Disposizione: gestionemagazzino ---
Prodotto -[hidden]right- ScortaMagazzino
ScortaMagazzino -[hidden]right- ETipoScorta
ETipoScorta -[hidden]right- EStatoOrdine
EStatoOrdine -[hidden]right- ParametriScorte
OrdineFornitore -[hidden]right- RigaOrdine
RigaOrdine -[hidden]right- Fornitore
Fornitore -[hidden]right- GestoreProdotti
GestoreProdotti -[hidden]right- GestoreScorte
GestoreOrdini -[hidden]right- IProdotti
IProdotti -[hidden]right- IScorte
IScorte -[hidden]right- IOrdini
IOrdini -[hidden]right- IScorteObserver
IRepositoryProdotti -[hidden]right- IRepositoryOrdini
Prodotto -[hidden]down- OrdineFornitore
OrdineFornitore -[hidden]down- GestoreOrdini
GestoreOrdini -[hidden]down- IRepositoryProdotti

' --- Disposizione: altri package ---
GestoreTurni -[hidden]right- GestorePresenze
GestorePresenze -[hidden]right- CalcoloStipendi
CalcoloStipendi -[hidden]right- IPresenze
Promozione -[hidden]right- PuntiAccumulati
IPagamento -[hidden]right- PaymentGateway
PaymentGateway -[hidden]right- Transazione
DBConnection -[hidden]right- DBClienti
DBClienti -[hidden]right- DBDipendenti
DBDipendenti -[hidden]right- DBProdotti
DBProdotti -[hidden]right- DBOrdini

' --- Disposizione tra package ---
App -[hidden]down- Persona
Manager -[hidden]right- SessioneSpesa
IRepositoryDipendenti -[hidden]down- Prodotto
GeneratoreRicevuta -[hidden]down- GestoreTurni
GestoreTurni -[hidden]down- Promozione
IRepositoryProdotti -[hidden]down- DBConnection
Promozione -[hidden]down- IPagamento

' --- Dipendenze tra package ---
app ..> sa : <<import>>
app ..> ss : <<import>>
app ..> gf : <<import>>
app ..> gm : <<import>>
app ..> gd : <<import>>
app ..> pe : <<import>>
app ..> pd : <<import>>

ss ..> sa : <<import>>
```

```
ss ..> gf : <<import>>
ss ..> gm : <<import>>
ss ..> pe : <<import>>
sa ..> gm : <<import>>
gd ..> sa : <<import>>
pd ..> sa : <<import>>
pd ..> gm : <<import>>
@enduml
```

Dipendenze tra package

```
@startuml
title Dipendenze tra package
skinparam packageStyle rectangle
skinparam shadowing false

package "App (radice)" as app
package selfscanning
package sistemaaccesso
package gestionedipendenti
package persistenzadati
package gestionemagazzino
package gestionefedelta
package pagamentiesterni

' App (composition root): usa tutti i package
app .[#gray].> selfscanning
app .[#gray].> sistemaaccesso
app .[#gray].> gestionedipendenti
app .[#gray].> persistenzadati
app .[#gray].> gestionemagazzino
app .[#gray].> gestionefedelta
app .[#gray].> pagamentiesterni

' Dipendenze tra i package di dominio e di persistenza
selfscanning -down-> sistemaaccesso
selfscanning -down-> gestionefedelta
selfscanning -down-> gestionemagazzino
selfscanning -down-> pagamentiesterni
gestionedipendenti -down-> sistemaaccesso
persistenzadati -down-> sistemaaccesso
persistenzadati -down-> gestionemagazzino
sistemaaccesso -down-> gestionemagazzino

' Suggerimenti di disposizione
gestionedipendenti -[hidden]right- persistenzadati
gestionefedelta -[hidden]right- pagamentiesterni

legend right
  Linea continua: dipendenza tra package di dominio e persistenza
  Linea grigia tratteggiata: App (composition root) usa tutti i package
endlegend
@enduml
```

Architettura a strati

```
@startuml
title Architettura a strati
skinparam packageStyle rectangle
skinparam shadowing false

package "Presentation" {
  [App (demo a riga di comando)] as App
  [LettoreOttico]
```

```
  [Totem]
}

package "Business Logic" {
  [selfscanning]
  [sistemaaccesso]
  [gestionedipendenti]
  [gestionemagazzino]
  [gestionefedelta]
  [pagamentiesterni]
}

package "Data" {
  [persistenzadati]
}

database "H2 embedded\n(clienti, dipendenti, prodotti)" as H2

App -down-> [selfscanning]
LettoreOttico -down-> [selfscanning]
Totem -down-> [selfscanning]
App .[#gray].> [persistenzadati] : composition root\n(crea i DAO)

[selfscanning] -down-> [sistemaaccesso]
[selfscanning] -down-> [gestionefedelta]
[selfscanning] -down-> [gestionemagazzino]
[selfscanning] -down-> [pagamentiesterni]
[gestionedipendenti] -down-> [sistemaaccesso]
[sistemaaccesso] -down-> [gestionemagazzino]

[persistenzadati] .up.> [sistemaaccesso] : implementa porte
[persistenzadati] .up.> [gestionemagazzino] : implementa porte
[persistenzadati] -down-> H2 : JDBC

[gestionefedelta] -[hidden]right- [pagamentiesterni]
[gestionedipendenti] -[hidden]left- [selfscanning]

note bottom of "Data" : Log4j 2 è trasversale a tutti gli strati
@enduml
```

Diagramma dei componenti

```
@startuml
title Diagramma dei componenti
skinparam shadowing false
skinparam componentStyle uml2
left to right direction

component "Self-scanning\n(SessioneSpesa, Carrello,\nLettoreOttico, Totem)\n\nfornisce:
ICarrello,\nISessioneControllabile" as SS

component "Controllo spesa\n(ControlloSpesa, CarrelloFisico)\n\nfornisce:
IControlloSpesa,\nICarrelloFisico" as CS

component "Fedeltà\n(Promozione, PuntiAccumulati)" as FED

component "Ruoli\n(Cassiere, Manager, Magazziniere,\nResponsabileDiReparto)\n\nfornisce:
IScorteObserver" as RUO

component "Autenticazione\n(AuthService)\n\nfornisce: IAutenticazione" as AUT

component "Prodotti\n(GestoreProdotti)\n\nfornisce: IProdotti" as PRO

component "Scorte\n(GestoreScorte)\n\nfornisce: IScorte" as SCO
```

```
component "Ordini\n(GestoreOrdini)\n\nfornisce: IOrdini" as ORD

component "Turni\n(GestoreTurni)\n\nfornisce: ITurni" as TUR

component "Presenze\n(GestorePresenze)\n\nfornisce: IPresenze" as PRE

component "Stipendi\n(CalcoloStipendi)\n\nfornisce: IStipendi" as STI

component "Pagamenti\n(PaymentGateway)\n\nfornisce: IPagamento" as PAG

component "Persistenza\n(DBClienti, DBDipendenti,\nDBProdotti, DBOrdini,
DBConnection)\n\nfornisce:
IRepositoryClienti,\nIRepositoryDipendenti,\nIRepositoryProdotti,\nIRepositoryOrdini" as
PER

database "H2 embedded\n(clienti, dipendenti, prodotti)" as H2

' --- Self-scanning ---
SS ..> AUT : IAutenticazione
SS ..> SCO : IScorte
SS ..> PAG : IPagamento
SS ..> PER : IRepositoryClienti,\nIRepositoryProdotti
SS ..> FED : uso diretto

' --- Controllo spesa e ruoli ---
CS ..> SS : ISessioneControllabile
RUO ..> CS : IControlloSpesa
RUO ..> TUR : ITurni
RUO ..> STI : IStipendi
RUO ..> PRO : IProdotti
RUO ..> SCO : IScorte
RUO ..> ORD : IOrdini
RUO ..> PER : IRepositoryClienti,\nIRepositoryProdotti
SCO ..> RUO : IScorteObserver

' --- Gestori verso la persistenza ---
AUT ..> PER : IRepositoryClienti,\nIRepositoryDipendenti
PRO ..> PER : IRepositoryProdotti
ORD ..> PER : IRepositoryOrdini,\nIRepositoryProdotti
TUR ..> PER : IRepositoryDipendenti
PRE ..> PER : IRepositoryDipendenti
PRE ..> AUT : IAutenticazione

PER --> H2 : JDBC
@enduml
```

Diagramma di deployment

```
@startuml
title Diagramma di deployment
skinparam shadowing false
left to right direction

node "Totem Pagamento" <<device>> {
  component Totem
}

node "Lettore ottico" <<device>> {
  component LettoreOttico
}

node "Server Back-Office (JVM)" <<execution environment>> {
  component "Business logic\n(sistemaaccesso, selfscanning,\ngestionefedelta,
gestionemagazzino,\ngestionedipendenti, pagamentiesterni)" as BL
  component "persistenzadati" as PD
  database "H2 embedded\n(clienti, dipendenti, prodotti)" as H2
```

```
}

cloud "Servizio di pagamento\nesterno (simulato)" as PAY

LettoreOttico --> BL : chiamata di procedura
Totem --> BL : chiamata di procedura
Totem ..> PAY : IPagamento
BL --> PD : IRepository*
PD --> H2 : JDBC

note bottom of PAY
  Nel prototipo il servizio è simulato
  da PaymentGateway, che accetta ogni
  importo positivo
end note

note bottom of BL
  Nel prototipo Totem e LettoreOttico
  sono classi dello stesso progetto e
  tutto gira in un'unica JVM
end note
@enduml
```

4.3 Software Design

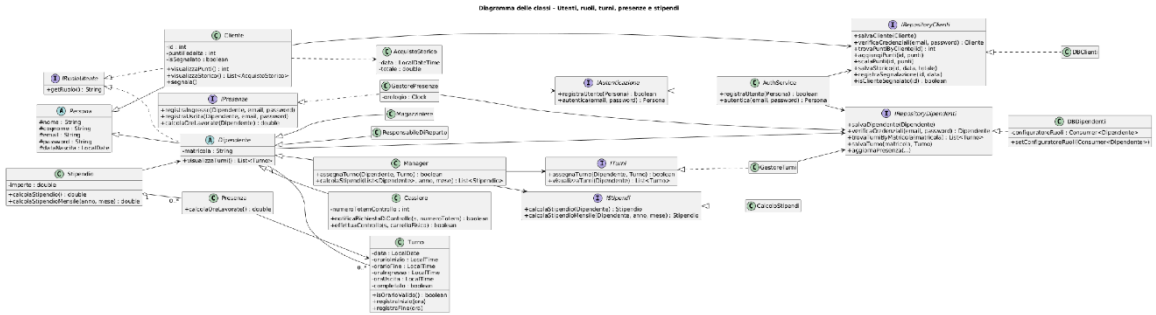
4.3.1 Introduzione

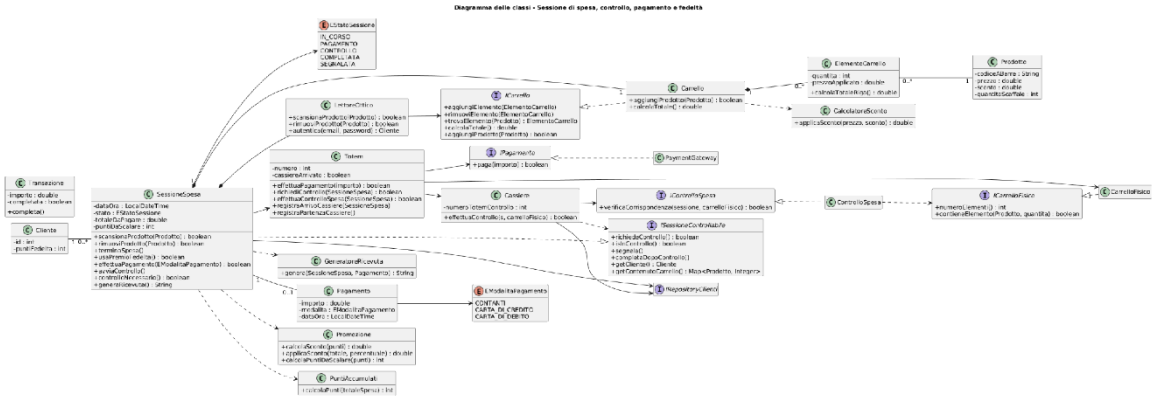
Il documento descrive il design del sistema Supermercato: i diagrammi UML strutturali (classi, componenti, package), il calcolo della complessità di McCabe su una piccola parte del codice, le misurazioni del codice nella sua ultima versione.

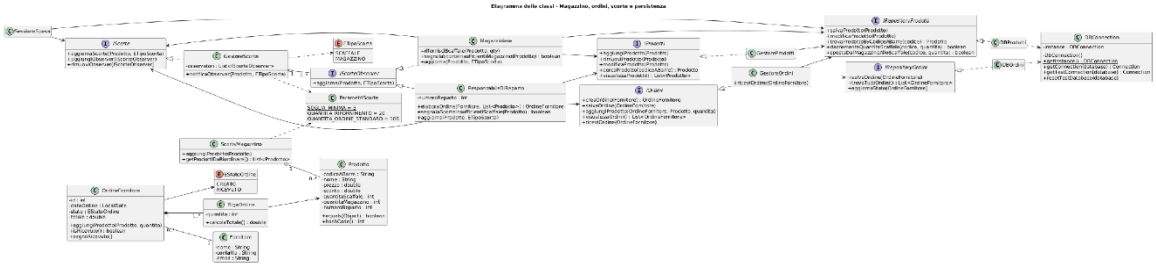
4.3.2 Diagrammi UML del design

4.3.2.1 Diagramma delle classi

Il class diagram è realizzato in Papyrus e rappresenta il modello implementato. Il codice PlantUML dei diagrammi è riportato nel paragrafo 4.3.6.







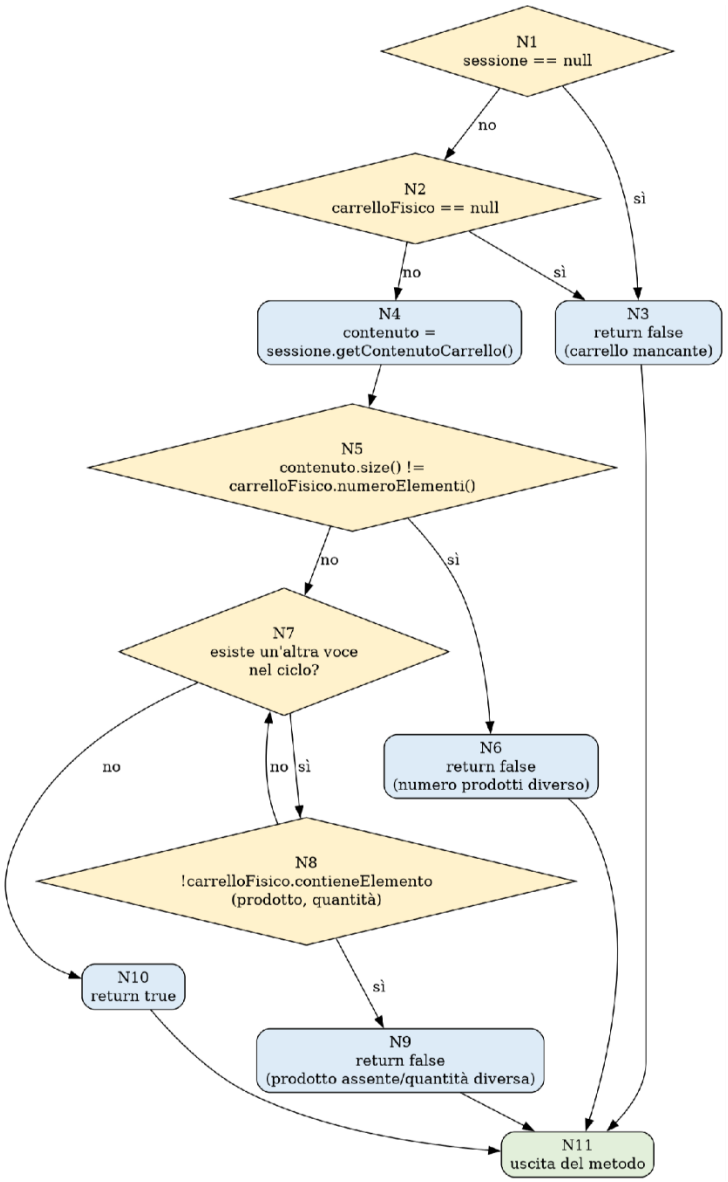
4.3.3 Complessità ciclomatica di McCabe

La complessità ciclomatica di McCabe misura il numero di cammini linearmente indipendenti di un metodo. Per un grafo di flusso con E archi e N nodi, e P componenti connesse, vale CV = E − N + 2P; equivale al numero di punti di decisione più uno, e al numero di regioni del grafo planare. È anche il numero minimo di casi di test necessari per percorrere tutti i cammini indipendenti.

Il calcolo manuale svolto dal team su una piccola parte del codice ha dato CV = 6, valore confermato dall'analisi di PMD. Di seguito è riportato il calcolo di dettaglio sul metodo ControlloSpesa.verificaCorrispondenza (CV = 6 secondo PMD).

4.3.3.1 Metodo analizzato

Il metodo verifica che il contenuto del carrello virtuale corrisponda a quello del carrello fisico: restituisce false se uno dei due carrelli manca, se il numero di prodotti diversi differisce o se un prodotto non è presente con la stessa quantità, altrimenti restituisce true.



4.3.3.2 Calcolo di CV

Metodo di calcolo	Risultato
Punti di decisione + 1	Decisioni in N1, N2, N5, N7, N8 = 5; CV = 5 + 1 = 6
E − N + P + 1	E = 15 archi, N = 11 nodi, P = 1; CV = 15 − 11 + 2 = 6
Regioni del grafo	6 regioni (5 interne e quella esterna)
PMD	CyclomaticComplexity = 6

4.3.4. Misurazione del codice

4.3.4.1 Strumenti e versione analizzata

Strumento	Misura	Oggetto
PMD	Complessità ciclomatica	Codice di produzione (src/main) e di test (src/test)
JDepend	Metriche di package: classi concrete e astratte, accoppiamento afferente ed efferente, astrattezza, instabilità, distanza, cicli	Package del progetto
SonarQube	Segnalazioni di qualità del codice (complessità cognitiva, duplicazioni, pratiche di codifica)	File di produzione e di test del progetto

4.3.4.2 Complessità ciclomatica con PMD

È stato creato un file .ruleset contenente le istruzioni per eseguire l'analisi della complessità ciclomatica con PMD ed è stato generato un file pmd-report.csv contenente l'esito dell'analisi.

Indicatore	Produzione (src/main)	Test (src/test)
Classi analizzate	44	63
Metodi e costruttori	401	326
CC totale	946	330
CC media per metodo	2,36	1,01
CC massima di un metodo	39	3
Metodi con CC > 10	5	0
Metodi con CC ≥ 8	12	0
Violazioni PMD (righe del report)	445	403

Fascia di rischio	Metodi di produzione	Metodi di test
1–10 (bassa)	396	326
11–20 (moderata)	3	0
21–50 (alta)	2	0
> 50 (molto alta)	0	0

Complessità per package (codice di produzione):

Package	Classi	Metodi	CC totale	CC media	CC max	Metodi CC > 10
unibg.it.Supermercato (App)	1	25	193	7,72	39	4
gestionedipendenti	3	21	50	2,38	7	0
gestionefedelta	2	4	9	2,25	3	0
gestionemagazzino	9	83	117	1,41	8	0
pagamentiesterni	2	6	7	1,17	2	0
persistenzadati	5	54	207	3,83	16	1
selfscanning	10	74	144	1,95	8	0
sistemaaccesso	12	134	219	1,63	8	0

Classi di produzione con complessità totale più alta:

Classe	CC totale	CC max metodo	Metodi	CC media
App	193	39	25	7,72
SessioneSpesa	60	8	27	2,22
DBClienti	55	6	16	3,44
DBDipendenti	53	8	14	3,79
DBConnection	38	16	6	6,33
ResponsabileDiReparto	38	8	17	2,24
DBProdotti	36	5	11	3,27
GestorePresenze	28	6	10	2,80
DBOrdini	25	7	7	3,57
Stipendio	24	6	9	2,67

Metodi con complessità maggiore o uguale a 8:

Classe	Metodo	CC	Fascia
App	simulaSpesa	39	alta
App	registraDipendente	22	alta
App	main	19	moderata
DBConnection	inizializzaDatabase	16	moderata
App	gestioneTurniManager	15	moderata
DBConnection	resetTestDatabase	10	bassa

App	creaOrdineDemo	9	bassa
App	registraCliente	9	bassa
GestoreOrdini	riceviOrdine	8	bassa
DBDipendenti	aggiornaPresenza	8	bassa
SessioneSpesa	effettuaPagamento	8	bassa
ResponsabileDiReparto	elaboraOrdine	8	bassa

Osservazioni:

- La complessità si concentra in App, la classe dimostrativa a riga di comando: 193 punti su 946 (circa il 20%), 4 dei 5 metodi con CC > 10 e il metodo con il valore massimo (simulaSpesa, 39). Escludendo App, la CC media scende a 2,00 e la CC massima a 16.
- Nelle classi di dominio nessun metodo supera 8; il caso più alto è DBConnection.inizializzaDatabase (16), dove lo stesso blocco di lettura dello script è ripetuto per i tre database: è un candidato a un refactoring con estrazione di metodo.
- I metodi riceviOrdine, effettuaPagamento ed elaboraOrdine raggiungono 8 perché contengono le condizioni di guardia introdotte con le correzioni (ordine già ricevuto, cassiere non disponibile, argomenti nulli): la complessità resta nella fascia bassa.
- Il 98,8% dei metodi di produzione (396 su 401) ricade nella fascia bassa. I test sono lineari: media 1,01 e massimo 3.

4.3.4.3 Metriche di package con JDepend

JDepend calcola per ogni package il numero di classi concrete (CC) e astratte (AC), l'accoppiamento afferente (Ca, quanti package dipendono da questo) ed efferente (Ce, da quanti package questo dipende), l'astrattezza A = AC / (CC + AC), l'instabilità I = Ce / (Ca + Ce) e la distanza dalla sequenza principale D = |A + I − 1|.

Selected object(s)

Package	CC(concr.cl.)	AC(abstr.cl.)	Ca(aff.)	Ce(eff.)	A	I	D	Cycle!
unibg.it.Supermercato.gestionedipendenti	3	1	1	1	0.25	0.50	0.25	
unibg.it.Supermercato.gestionefedelta	2	0	2	0	0.00	0.00	1.00	
unibg.it.Supermercato.gestionemagazzino	11	6	4	0	0.35	0.00	0.64	
unibg.it.Supermercato.pagamentiesterni	2	1	2	0	0.33	0.00	0.66	
unibg.it.Supermercato.persistenzadati	5	0	1	2	0.00	0.66	0.33	
unibg.it.Supermercato.selfscanning	12	1	1	4	0.07	0.80	0.12	
unibg.it.Supermercato.sistemaaccesso	10	11	4	1	0.52	0.20	0.27	

Osservazioni:

- Nessun package è coinvolto in cicli: la colonna dei cicli è vuota. È l'obiettivo del refactoring che ha spostato le interfacce nel dominio.
- La schermata di JDepend non include il package radice (App) né le librerie esterne; il logging con Log4j 2 è usato solo dalla classe App, in coerenza con quanto dichiarato nei documenti sul logging.

4.3.4.4 Analisi con SonarQube

SonarQube segnala i problemi di qualità del codice. Riassumiamo le segnalazioni principali per tipologia.

Classe App (applicazione dimostrativa):

Tipologia di segnalazione	Dettaglio
Complessità cognitiva	Tre metodi superano il limite di 15 (con valori 32, 42 e 86)
Output su console	Numerose occorrenze di System.out al posto di un logger
Letterali duplicati	Stringhe ripetute da 3 a 8 volte (ad esempio "Email: " 8 volte, "Password: " 6 volte, "Inserisci un numero valido.", "Matricola: ", "Nome: " e "Scelta: " 4 volte, "Credenziali non valide." e "Scelta non valida. Riprova." 3 volte)
Cicli	Tre cicli senza condizione di uscita esplicita; sette cicli con più istruzioni break e continue
Stile	Sostituire instanceof con cast con il pattern matching (Cliente, Dipendente, Manager, Cassiere, Magazziniere, ResponsabileDiReparto); assegnazione inutile; if annidabili
Parametri	Metodo con 8 parametri (limite 7), la configurazione dei ruoli del composition root
Data e ora	Chiamate a now() senza indicare fuso orario o Clock

Altre classi di produzione:

Classe	Segnalazione
Pagamento, OrdineFornitore	now() senza fuso orario o Clock
Manager	Costruttore con 8 parametri (limite 7)
Prodotto	Costruttore con 8 parametri (limite 7)
LettoreOttico	instanceof con cast da sostituire con pattern matching
Carrello	Campo totale sostituibile con variabile locale
DBOrdini	Chiamata a setter invariante nel ciclo
DBConnection	Letterali "db.user" e "db.password" duplicati 3 volte; query SQL costruita dinamicamente in resetTestDatabase

Osservazioni: le segnalazioni più numerose riguardano App (uso di System.out, letterali duplicati, complessità cognitiva), che è una classe dimostrativa; il dominio presenta pochi problemi, per lo più di stile e di robustezza (costruttori con molti parametri, fuso orario di now()). La complessità cognitiva segnalata su tre metodi di App è coerente con la complessità ciclomatica elevata di simulaSpesa, registraDipendente e main.

4.3.5. Design pattern

4.3.5.1 Observer

Contesto: Il sistema deve reagire alle variazioni delle scorte di un prodotto: quando la quantità sullo scaffale o in magazzino scende sotto la soglia critica, il magazziniere deve spostare merce dal magazzino allo scaffale e il responsabile di reparto deve ordinare merce al fornitore. Senza un pattern appropriato, GestoreScorte dovrebbe conoscere direttamente queste classi, con accoppiamento elevato e difficoltà di estensione.

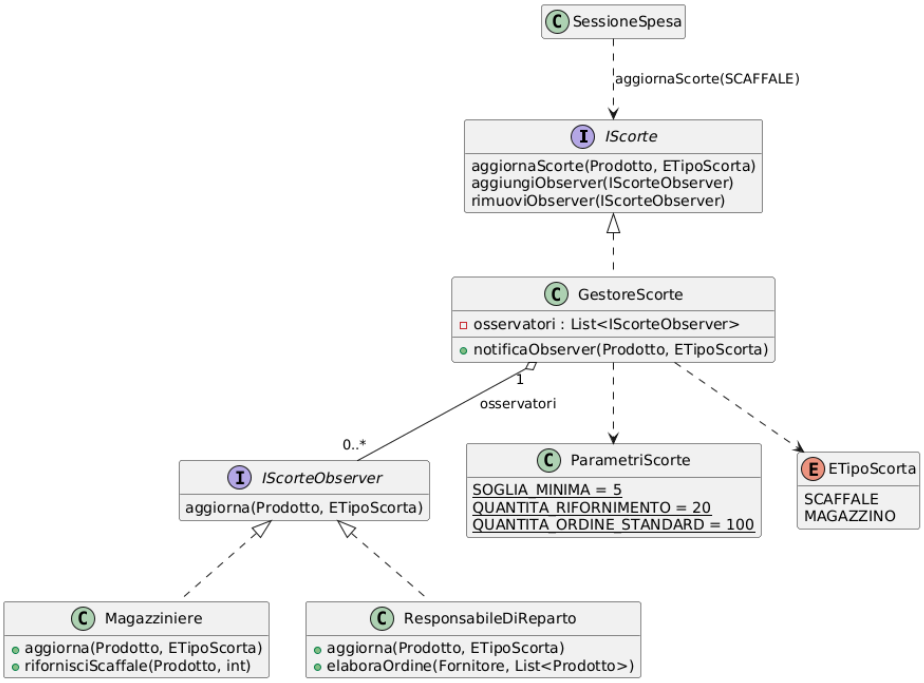
Problema: Come fare in modo che un oggetto (GestoreScorte) comunichi cambiamenti di stato a più oggetti interessati senza conoscere la natura o il numero di tali oggetti?

Forze:

- Minimizzare il CBO (Coupling Between Objects) tra GestoreScorte e le classi che reagiscono agli aggiornamenti.
- Il numero di componenti interessati può variare nel tempo.
- Gli osservatori devono poter essere aggiunti o rimossi senza modificare GestoreScorte.

Soluzione: GestoreScorte (che realizza IScorte) è il soggetto osservabile: mantiene l'elenco degli osservatori (aggiungiObserver, rimuoviObserver), evita le registrazioni duplicate e li notifica con notificaObserver; le classi interessate implementano IScorteObserver. La soglia minima e le quantità di rifornimento e di ordine sono definite in ParametriScorte.

Ruolo nel pattern	**Classe**	**Responsabilità**
Subject	IScorte, GestoreScorte	Gestisce gli osservatori; aggiornaScorte notifica quando la quantità a scaffale è sotto la soglia (ParametriScorte.SOGLIA_MINIMA = 5) e, per il magazzino, quando richiesto
Observer	IScorteObserver	Dichiara aggiorna(Prodotto, ETipoScorta)
ConcreteObserver	Magazziniere	Su notifica di tipo SCAFFALE sposta fino a 20 pezzi dal magazzino, salva lo spostamento tramite repository e, se il magazzino scende sotto la soglia, notifica di tipo MAGAZZINO
ConcreteObserver	ResponsabileDiReparto	Su notifica di tipo MAGAZZINO per un prodotto del proprio reparto elabora un ordine al fornitore di 100 pezzi
Evento scatenante	SessioneSpesa	Dopo il completamento dell'acquisto aggiorna la quantità a scaffale con una scrittura atomica e invoca aggiornaScorte



Conseguenze: GestoreScorte non conosce le classi concrete e il numero di osservatori è modificabile a runtime. Le notifiche sono sincrone e l'ordine di notifica è quello di registrazione. Gli osservatori sono creati dal composition root, che registra ciascun dipendente una sola volta anche quando il ruolo viene letto più volte dal database.

4.3.5.2 Singleton

Contesto: L'accesso ai tre database embedded (clienti, dipendenti, prodotti) richiede la lettura della configurazione da database.properties e l'esecuzione degli script di inizializzazione. Serve un unico punto di accesso e di configurazione, utilizzabile da tutti i DAO.

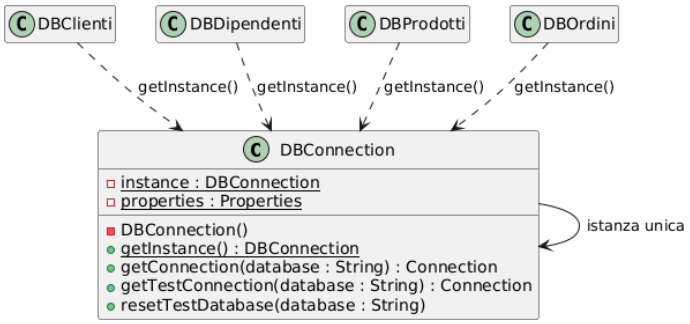
Problema: Come garantire che una classe abbia una sola istanza accessibile globalmente?

Forze:

- La configurazione deve essere letta una sola volta e condivisa da tutti i DAO.
- I DAO devono ottenere le connessioni senza ricevere DBConnection come parametro.
- La modalità di test deve poter usare una configurazione separata (database-test.properties).

Soluzione: DBConnection ha il costruttore privato, un'istanza statica privata creata al caricamento della classe (inizializzazione eager, thread-safe grazie al class loader) e il metodo pubblico statico getInstance(); le proprietà sono lette in un blocco statico. I metodi getConnection(database), getTestConnection(database) e resetTestDatabase(database) forniscono l'accesso ai database e ripristinano lo stato dei test.

Ruolo nel pattern	Classe	Dettaglio
Singleton	DBConnection	Costruttore privato, istanza statica, getInstance()
Client	DBClienti, DBDipendenti, DBProdotti, DBOrdini	Ottengono le connessioni con DBConnection.getInstance()
Verifica	SingletonDBConnectionTest	Verifica che chiamate ripetute di getInstance() restituiscano la stessa istanza



Conseguenze: Accesso e configurazione centralizzati. Lo stato globale rende più difficile sostituire DBConnection nei test: per questo i DAO offrono la modalità di test (parametro usaDatabaseTest) e i repository sono accessibili tramite interfacce.

4.3.6 Codice PlantUML dei diagrammi

Diagramma delle classi utenti, ruoli, turni, presenze e stipendi

```
@startuml
title Diagramma delle classi – Utenti, ruoli, turni, presenze e stipendi
skinparam shadowing false
skinparam classAttributeIconSize 0
skinparam nodesep 15
skinparam ranksep 40
left to right direction
hide empty members

abstract class Persona {
  #nome : String
  #cognome : String
  #email : String
  #password : String
  #dataNascita : LocalDate
}
class Cliente {
  -id : int
  -puntiFedelta : int
  -isSegnalato : boolean
  +visualizzaPunti() : int
  +visualizzaStorico() : List<AcquistoStorico>
  +segnala()
}
abstract class Dipendente {
  -matricola : String
  +visualizzaTurni() : List<Turno>
}
class Cassiere {
```

```
  -numeroTotemControllo : int
  +notificaRichiestaDiControllo(s, numeroTotem) : boolean
  +effettuaControllo(s, carrelloFisico) : boolean
}
class Manager {
  +assegnaTurno(Dipendente, Turno) : boolean
  +calcolaStipendi(List<Dipendente>, anno, mese) : List<Stipendio>
}
class Magazziniere
class ResponsabileDiReparto
class AcquistoStorico {
  -data : LocalDateTime
  -totale : double
}
class AuthService {
  +registraUtente(Persona) : boolean
  +autentica(email, password) : Persona
}
class Turno {
  -data : LocalDate
  -orarioInizio : LocalTime
  -orarioFine : LocalTime
  -oraIngresso : LocalTime
  -oraUscita : LocalTime
  -completato : boolean
  +isOrarioValido() : boolean
  +registraInizio(ora)
  +registraFine(ora)
}
class Presenza {
  +calcolaOreLavorate() : double
}
class Stipendio {
  -importo : double
  +calcolaStipendio() : double
  +calcolaStipendioMensile(anno, mese) : double
}
class GestoreTurni
class GestorePresenze {
  -orologio : Clock
}
class CalcoloStipendi
class DBClienti
class DBDipendenti {
  -configuratoreRuoli : Consumer<Dipendente>
  +setConfiguratoreRuoli(Consumer<Dipendente>)
}

interface IRuoloUtente {
  +getRuolo() : String
}
interface IAutenticazione {
  +registraUtente(Persona) : boolean
  +autentica(email, password) : Persona
}
interface ITurni {
  +assegnaTurno(Dipendente, Turno) : boolean
  +visualizzaTurni(Dipendente) : List<Turno>
}
interface IPresenze {
  +registraIngresso(Dipendente, email, password)
  +registraUscita(Dipendente, email, password)
  +calcolaOreLavorate(Dipendente) : double
}
interface IStipendi {
  +calcolaStipendio(Dipendente) : Stipendio
  +calcolaStipendioMensile(Dipendente, anno, mese) : Stipendio
}
```

```
interface IRepositoryClienti {
  +salvaCliente(Cliente)
  +verificaCredenziali(email, password) : Cliente
  +trovaPuntiByCliente(id) : int
  +aggiungiPunti(id, punti)
  +scalaPunti(id, punti)
  +salvaStorico(id, data, totale)
  +registraSegnalazione(id, data)
  +isClienteSegnalato(id) : boolean
}
interface IRepositoryDipendenti {
  +salvaDipendente(Dipendente)
  +verificaCredenziali(email, password) : Dipendente
  +trovaTurniByMatricola(matricola) : List<Turno>
  +salvaTurno(matricola, Turno)
  +aggiornaPresenza(...)
}

Persona <|-- Cliente
Persona <|-- Dipendente
Dipendente <|-- Cassiere
Dipendente <|-- Manager
Dipendente <|-- Magazziniere
Dipendente <|-- ResponsabileDiReparto
IRuoloUtente <|.. Cliente
IRuoloUtente <|.. Dipendente

IAutenticazione <|.. AuthService
ITurni <|.. GestoreTurni
IPresenze <|.. GestorePresenze
IStipendi <|.. CalcoloStipendi
IRepositoryClienti <|.. DBClienti
IRepositoryDipendenti <|.. DBDipendenti

Cliente ..> AcquistoStorico
Cliente --> IRepositoryClienti
Dipendente --> IRepositoryDipendenti
Dipendente "1" -- "0..*" Turno
Presenza --> Turno
Stipendio --> Dipendente
Stipendio "1" o-- "0..*" Presenza
Manager --> ITurni
Manager --> IStipendi

AuthService --> IRepositoryClienti
AuthService --> IRepositoryDipendenti
GestoreTurni --> IRepositoryDipendenti
GestorePresenze --> IRepositoryDipendenti
GestorePresenze --> IAutenticazione
@enduml
```

Diagramma delle classi – Sessione di spesa, controllo, pagamento e fedeltà

```
@startuml
title Diagramma delle classi – Sessione di spesa, controllo, pagamento e fedeltà
skinparam shadowing false
skinparam classAttributeIconSize 0
skinparam nodesep 15
skinparam ranksep 40
left to right direction
hide empty members

class Cliente {
  -id : int
  -puntiFedelta : int
}
```

```
class Cassiere {
  -numeroTotemControllo : int
  +effettuaControllo(s, carrelloFisico) : boolean
}
class Prodotto {
  -codiceABarre : String
  -prezzo : double
  -sconto : double
  -quantitaScaffale : int
}
class SessioneSpesa {
  -dataOra : LocalDateTime
  -stato : EStatoSessione
  -totaleDaPagare : double
  -puntiDaScalare : int
  +scansionaProdotto(Prodotto) : boolean
  +rimuoviProdotto(Prodotto) : boolean
  +terminaSpesa()
  +usaPremioFedelta() : boolean
  +effettuaPagamento(EModalitaPagamento) : boolean
  +avviaControllo()
  +controlloNecessario() : boolean
  +generaRicevuta() : String
}
enum EStatoSessione {
  IN_CORSO
  PAGAMENTO
  CONTROLLO
  COMPLETATA
  SEGNALATA
}
enum EModalitaPagamento {
  CONTANTI
  CARTA_DI_CREDITO
  CARTA_DI_DEBITO
}
class Carrello {
  +aggiungiProdotto(Prodotto) : boolean
  +calcolaTotale() : double
}
class ElementoCarrello {
  -quantita : int
  -prezzoApplicato : double
  +calcolaTotaleRiga() : double
}
class CarrelloFisico
class LettoreOttico {
  +scansionaProdotto(Prodotto) : boolean
  +rimuoviProdotto(Prodotto) : boolean
  +autentica(email, password) : Cliente
}
class Totem {
  -numero : int
  -cassiereArrivato : boolean
  +effettuaPagamento(importo) : boolean
  +richiediControllo(SessioneSpesa) : boolean
  +effettuaControlloSpesa(SessioneSpesa) : boolean
  +registraArrivoCassiere(SessioneSpesa)
  +registraPartenzaCassiere()
}
class ControlloSpesa
class CalcolatoreSconto {
  +applicaSconto(prezzo, sconto) : double
}
class GeneratoreRicevuta {
  +genera(SessioneSpesa, Pagamento) : String
}
class Pagamento {
  -importo : double
  -modalita : EModalitaPagamento
  -dataOra : LocalDateTime
}
```

```
class PaymentGateway
class Transazione {
  -importo : double
  -completata : boolean
  +completa()
}
class Promozione {
  +calcolaSconto(punti) : double
  +applicaSconto(totale, percentuale) : double
  +calcolaPuntiDaScalare(punti) : int
}
class PuntiAccumulati {
  +calcolaPunti(totaleSpesa) : int
}

interface ICarrello {
  +aggiungiElemento(ElementoCarrello)
  +rimuoviElemento(ElementoCarrello)
  +trovaElemento(Prodotto) : ElementoCarrello
  +calcolaTotale() : double
  +aggiungiProdotto(Prodotto) : boolean
}
interface ISessioneControllabile {
  +richiedeControllo() : boolean
  +isInControllo() : boolean
  +segnala()
  +completaDopoControllo()
  +getCliente() : Cliente
  +getContenutoCarrello() : Map<Prodotto, Integer>
}
interface IControlloSpesa {
  +verificaCorrispondenza(sessione, carrelloFisico) : boolean
}
interface ICarrelloFisico {
  +numeroElementi() : int
  +contieneElemento(Prodotto, quantita) : boolean
}
interface IPagamento {
  +paga(importo) : boolean
}
interface IRepositoryClienti

ICarrello <|.. Carrello
ISessioneControllabile <|.. SessioneSpesa
IControlloSpesa <|.. ControlloSpesa
ICarrelloFisico <|.. CarrelloFisico
IPagamento <|.. PaymentGateway

Cliente "1" -- "0..*" SessioneSpesa
SessioneSpesa "1" *-- "1" Carrello
Carrello "1" *-- "0..*" ElementoCarrello
ElementoCarrello "0..*" -- "1" Prodotto
SessioneSpesa "1" -- "0..1" Pagamento
SessioneSpesa --> EStatoSessione
Pagamento --> EModalitaPagamento
SessioneSpesa *-- LettoreOttico
LettoreOttico --> ICarrello
SessioneSpesa --> Totem
SessioneSpesa ..> GeneratoreRicevuta
SessioneSpesa ..> Promozione
SessioneSpesa ..> PuntiAccumulati
SessioneSpesa --> IRepositoryClienti
Carrello ..> CalcolatoreSconto

Totem --> Cassiere
Totem --> CarrelloFisico
Totem --> IPagamento
Cassiere --> IControlloSpesa
Cassiere ..> ISessioneControllabile
Cassiere --> IRepositoryClienti
```

```
ControlloSpesa ..> ICarrelloFisico
@enduml
```

Diagramma delle classi – Magazzino, ordini, scorte e persistenza

```
@startuml
title Diagramma delle classi – Magazzino, ordini, scorte e persistenza
skinparam shadowing false
skinparam classAttributeIconSize 0
skinparam nodesep 15
skinparam ranksep 40
left to right direction
hide empty members

class SessioneSpesa
class Magazziniere {
  +rifornisciScaffale(Prodotto, qty)
  +segnalaScorteInsufficientiMagazzino(Prodotto) : boolean
  +aggiorna(Prodotto, ETipoScorta)
}
class ResponsabileDiReparto {
  -numeroReparto : int
  +elaboraOrdine(Fornitore, List<Prodotto>) : OrdineFornitore
  +segnalaScorteInsufficientiScaffale(Prodotto) : boolean
  +aggiorna(Prodotto, ETipoScorta)
}
class Prodotto {
  -codiceABarre : String
  -nome : String
  -prezzo : double
  -sconto : double
  -quantitaScaffale : int
  -quantitaMagazzino : int
  -numeroReparto : int
  +equals(Object) : boolean
  +hashCode() : int
}
class OrdineFornitore {
  -id : int
  -dataOrdine : LocalDate
  -stato : EStatoOrdine
  -totale : double
  +aggiungiProdotto(Prodotto, quantita)
  +isRicevuto() : boolean
  +segnaRicevuto()
}
class RigaOrdine {
  -quantita : int
  +calcolaTotale() : double
}
class Fornitore {
  -nome : String
  -contatto : String
  -email : String
}
class ScortaMagazzino {
  +aggiungiProdotto(Prodotto)
  +getProdottiDaRiordinare() : List<Prodotto>
}
class ParametriScorte {
  {static} SOGLIA_MINIMA = 5
  {static} QUANTITA_RIFORNIMENTO = 20
  {static} QUANTITA_ORDINE_STANDARD = 100
}
class GestoreProdotti
class GestoreScorte {
  -osservatori : List<IScorteObserver>
  +notificaObserver(Prodotto, ETipoScorta)
}
```

```
class GestoreOrdini {
  +riceviOrdine(OrdineFornitore)
}
class DBProdotti
class DBOrdini
class DBConnection {
  -{static} instance : DBConnection
  -DBConnection()
  +{static} getInstance() : DBConnection
  +getConnection(database) : Connection
  +getTestConnection(database) : Connection
  +resetTestDatabase(database)
}

enum EStatoOrdine {
  CREATO
  RICEVUTO
}
enum ETipoScorta {
  SCAFFALE
  MAGAZZINO
}

interface IProdotti {
  +aggiungiProdotto(Prodotto)
  +rimuoviProdotto(Prodotto)
  +modificaProdotto(Prodotto)
  +cercaProdotto(codiceABarre) : Prodotto
  +visualizzaProdotti() : List<Prodotto>
}
interface IScorte {
  +aggiornaScorte(Prodotto, ETipoScorta)
  +aggiungiObserver(IScorteObserver)
  +rimuoviObserver(IScorteObserver)
}
interface IScorteObserver {
  +aggiorna(Prodotto, ETipoScorta)
}
interface IOrdini {
  +creaOrdine(Fornitore) : OrdineFornitore
  +salvaOrdine(OrdineFornitore)
  +aggiungiProdotto(OrdineFornitore, Prodotto, quantita)
  +visualizzaOrdini() : List<OrdineFornitore>
  +riceviOrdine(OrdineFornitore)
}
interface IRepositoryProdotti {
  +salvaProdotto(Prodotto)
  +modificaProdotto(Prodotto)
  +trovaProdottoByCodiceABarre(codice) : Prodotto
  +decrementaQuantitaScaffale(codice, quantita) : boolean
  +spostaDalMagazzinoAlloScaffale(codice, quantita) : boolean
}
interface IRepositoryOrdini {
  +salvaOrdine(OrdineFornitore)
  +trovaTuttiOrdini() : List<OrdineFornitore>
  +aggiornaStato(OrdineFornitore)
}

IProdotti <|.. GestoreProdotti
IScorte <|.. GestoreScorte
IOrdini <|.. GestoreOrdini
IScorteObserver <|.. Magazziniere
IScorteObserver <|.. ResponsabileDiReparto
IRepositoryProdotti <|.. DBProdotti
IRepositoryOrdini <|.. DBOrdini

OrdineFornitore "1" *-- "1..*" RigaOrdine
RigaOrdine --> Prodotto
OrdineFornitore "0..*" -- "1" Fornitore
OrdineFornitore --> EStatoOrdine
```

```
ScortaMagazzino "1" o-- "0..*" Prodotto
ScortaMagazzino ..> ParametriScorte

GestoreScorte "1" o-- "0..*" IScorteObserver
GestoreScorte ..> ParametriScorte
GestoreScorte ..> ETipoScorta
GestoreProdotti --> IRepositoryProdotti
GestoreOrdini --> IRepositoryOrdini
GestoreOrdini --> IRepositoryProdotti

Magazziniere --> IProdotti
Magazziniere --> IScorte
Magazziniere --> IRepositoryProdotti
ResponsabileDiReparto --> IProdotti
ResponsabileDiReparto --> IScorte
ResponsabileDiReparto --> IOrdini
SessioneSpesa --> IScorte
SessioneSpesa --> IRepositoryProdotti

DBProdotti ..> DBConnection
DBOrdini ..> DBConnection
@enduml
```

Pattern Observer

```
@startuml
hide empty members

interface IScorte {
  aggiornaScorte(Prodotto, ETipoScorta)
  aggiungiObserver(IScorteObserver)
  rimuoviObserver(IScorteObserver)
}
interface IScorteObserver {
  aggiorna(Prodotto, ETipoScorta)
}
class GestoreScorte {
  -osservatori : List<IScorteObserver>
  +notificaObserver(Prodotto, ETipoScorta)
}
class Magazziniere {
  +aggiorna(Prodotto, ETipoScorta)
  +rifornisciScaffale(Prodotto, int)
}
class ResponsabileDiReparto {
  +aggiorna(Prodotto, ETipoScorta)
  +elaboraOrdine(Fornitore, List<Prodotto>)
}
class SessioneSpesa
class ParametriScorte {
  {static} SOGLIA_MINIMA = 5
  {static} QUANTITA_RIFORNIMENTO = 20
  {static} QUANTITA_ORDINE_STANDARD = 100
}
enum ETipoScorta {
  SCAFFALE
  MAGAZZINO
}

IScorte <|.. GestoreScorte
IScorteObserver <|.. Magazziniere
IScorteObserver <|.. ResponsabileDiReparto
GestoreScorte "1" o-- "0..*" IScorteObserver : osservatori
SessioneSpesa ..> IScorte : aggiornaScorte(SCAFFALE)
GestoreScorte ..> ParametriScorte
GestoreScorte ..> ETipoScorta
@enduml
```

Pattern Singleton

```
@startuml
hide empty members

class DBConnection {
  -{static} instance : DBConnection
  -{static} properties : Properties
  -DBConnection()
  +{static} getInstance() : DBConnection
  +getConnection(database : String) : Connection
  +getTestConnection(database : String) : Connection
  +resetTestDatabase(database : String)
}
class DBClienti
class DBDipendenti
class DBProdotti
class DBOrdini

DBConnection --> DBConnection : istanza unica
DBClienti ..> DBConnection : getInstance()
DBDipendenti ..> DBConnection : getInstance()
DBProdotti ..> DBConnection : getInstance()
DBOrdini ..> DBConnection : getInstance()
@enduml
```

5 TESTING

5.1 Software Testing

5.1.1 Organizzazione dei test

I test sono distribuiti in 63 classi di test: 48 test di unità nei package del sistema e 15 test di scenario nel package radice. La suite comprende in tutto 266 test. La tabella riporta, per package, le classi di test e il numero di metodi analizzati da PMD (inclusi i metodi di preparazione @BeforeEach e gli eventuali metodi di supporto, per questo il totale, 326, supera il numero di test).

Package	Classi di test	Metodi	Classi
radice (scenari)	15	65	AutenticazioneClienteTest, AutenticazioneDipendenteTest, ControlloSpesaClienteTest, GestioneCarrelloTest, GestioneOrdineTest, GestionePresenzaTest, GestioneProdottoTest, GestioneScorteTest, GestioneSpesaClienteTest, GestioneStipendioTest, GestioneTurnoTest, ParametriScorteTest, RegistrazioneClienteTest, RegistrazioneDipendenteTest, SingletonDBConnectionTest
gestionedipendenti	9	47	CalcoloStipendiTest, GestorePresenzeCredenzialiTest, GestorePresenzeTest, GestoreTurniTest, GestoreTurniValidazioneTest, PresenzaTest, StipendioTest, TurnoOrarioTest, TurnoTest
gestionefedelta	2	11	PromozioneTest, PuntiAccumulatiTest
gestionemagazzino	8	34	GestoreOrdiniRicezioneTest, GestoreOrdiniTest, GestoreProdottiTest, GestoreScorteTest, OrdineFornitoreTest, ProdottoTest, ProdottoUguaglianzaTest, ScortaMagazzinoTest
pagamentiesterni	2	6	PaymentGatewayTest, TransazioneTest
persistenzadati	8	51	DBClientiTest, DBConnectionTest, DBDipendentiConfiguratoreTest, DBDipendentiTest, DBOrdiniStatoTest, DBOrdiniTest, DBProdottiAtomicoTest, DBProdottiTest
selfscanning	11	74	CalcolatoreScontoTest, CarrelloFisicoTest, CarrelloTest, ControlloSpesaTest,

			ElementoCarrelloTest, FlussoPagamentoControlloTest, GeneratoreRicevutaTest, LettoreOtticoTest, SessioneSpesaScorteAtomicheTest, SessioneSpesaTest, TotemTest
sistemaaccesso	8	38	AuthServiceTest, CassiereControlloSpesaTest, CassiereTest, DipendenteTest, MagazziniereRepositoryTest, MagazziniereTest, ManagerTest, ResponsabileDiRepartoTest

5.1.2 Casi di test rappresentativi

Per documentare i test di unità e i flussi funzionali senza duplicare il codice sorgente, i casi più rappresentativi sono descritti attraverso precondizioni, azione, risultato atteso ed esito.

ID	Funzionalità	Precondizioni / input	Risultato atteso
TC-01	Registrazione cliente	Database di test disponibile; cliente con dati validi.	La registrazione va a buon fine e il cliente può essere autenticato con le credenziali registrate.
TC-02	Autenticazione cliente	Cliente registrato; credenziali corrette o password errata.	Con credenziali corrette viene restituito il cliente; con password errata l'autenticazione fallisce.
TC-03	Registrazione dipendente	Repository dipendenti simulato tramite Mockito; dipendente valido.	La registrazione restituisce esito positivo e viene verificato il salvataggio sul repository.
TC-04	Gestione carrello	Prodotto disponibile a scaffale.	Il prodotto viene aggiunto, quantità ripetute vengono gestite correttamente, la rimozione riduce la quantità fino a eliminare l'elemento e il totale viene calcolato con lo sconto del prodotto.
TC-05	Gestione spesa e pagamento	Cliente, carrello e pagamento configurati; controllo casuale reso deterministico.	La spesa segue il flusso previsto e, in caso di pagamento completato, lo stato della sessione viene aggiornato; il pagamento prima di terminare la spesa o con modalità nulla è rifiutato.
TC-06	Aggiornamento scorte	Prodotto con scorta sotto soglia e osservatori registrati.	La notifica viene inviata e il componente interessato aggiorna la scorta secondo la propria responsabilità.
TC-07	Turni	Dipendente e turno validi.	Il turno viene assegnato; un turno sovrapposto per lo stesso dipendente non viene assegnato.
TC-08	Presenze	Dipendente autenticabile e turno del giorno disponibile.	Ingresso e uscita vengono registrati e le ore lavorate risultano calcolabili.

TC-09	Prodotti	Repository prodotti di test.	Inserimento, modifica, ricerca e rimozione del prodotto producono il risultato previsto.
TC-10	Ordini	Ordine con fornitore e righe prodotto.	L'ordine viene creato in stato CREATO, le righe vengono aggiunte e la ricezione incrementa la quantità di magazzino.
TC-11	Stipendi	Dipendente con turni completati in mesi differenti.	Il calcolo mensile considera solo le presenze del mese richiesto e restituisce l'importo atteso.
TC-12	Persistenza	Database H2 di test inizializzato.	Le operazioni di persistenza e recupero restituiscono dati coerenti con quelli salvati.
TC-13	Design pattern - Singleton	Accesso ripetuto a DBConnection.getInstance().	Le chiamate restituiscono la stessa istanza.
TC-14	Pagamento esterno e transazione	PaymentGateway e Transazione con importo positivo, nullo e negativo.	Il pagamento è accettato solo per importi positivi; una transazione nasce non completata e si completa solo con un importo valido.
TC-15	Programma fedeltà	Clienti con diversi saldi di punti e totali di spesa.	Nessuno sconto sotto 100 punti; sconto pari a punti/20 con tetto del 30%; scalo di al massimo 600 punti; accumulo di 1 punto ogni 2 euro.
TC-16	Controllo della spesa e segnalazione	Sessione in controllo, carrello fisico uguale o diverso da quello virtuale.	Con carrelli corrispondenti l'acquisto è completato; con carrello mancante, numero di prodotti diverso o quantità diversa la verifica fallisce e il cliente è segnalato. Senza ControlloSpesa impostato il controllo non parte e il cliente non è segnalato.
TC-17	Premio fedeltà in una sessione di spesa	Cliente con 200 punti, spesa di 20 euro.	Il premio applica lo sconto (totale 18 euro) ma non scala subito i punti; non può essere applicato due volte; con 99 punti non si ottiene sconto; con pagamento rifiutato o sessione segnalata i punti non vengono scalati; a pagamento completato i punti sono scalati (200) e poi vengono aggiunti i nuovi (9), in questo ordine.
TC-18	Ricezione ordine idempotente	Ordine con 100 pezzi di un prodotto con 20 pezzi in magazzino; oppure ordine salvato nel database di test (10 pezzi su 20).	Due ricezioni consecutive aumentano il magazzino una sola volta (120; nel database 30) e il repository modifica il prodotto una volta sola; l'ordine passa a RICEVUTO e lo stato viene salvato; un ordine già ricevuto non modifica il magazzino; senza repository dei prodotti la ricezione solleva IllegalStateException e l'ordine resta CREATO.

TC-19	Confronto dei prodotti per codice a barre	Carrello fisico con una copia (istanza diversa) dello stesso prodotto.	Il carrello fisico riconosce il prodotto con la stessa quantità; non lo riconosce con quantità diversa, con un altro codice a barre o con prodotto nullo; due prodotti con lo stesso codice sono uguali, hanno lo stesso hashCode e fungono da stessa chiave in una mappa; il controllo della spesa considera corrispondenti i due carrelli.
TC-20	Ruoli caricati dal database	Manager e cassiere salvati nel database di test; configuratore dei ruoli impostato sul repository.	Il manager caricato riceve il gestore dei turni e assegna i turni tramite esso; il cassiere caricato esegue il controllo con il ControlloSpesa iniettato; il configuratore è invocato una volta per ciascun ruolo (manager, cassiere, magazziniere, responsabile di reparto).
TC-21	Validazione dei turni	Repository dei dipendenti simulato; turni con orari diversi.	Un turno valido è assegnato e salvato; un turno sovrapposto è rifiutato e non salvato; i turni consecutivi sono accettati; un turno con inizio dopo la fine o uguale alla fine, o con argomenti nulli, è rifiutato; il manager restituisce l'esito del gestore e, senza gestore, non assegna nulla. Turno.isOrarioValido è falso con orari invertiti, uguali o nulli.
TC-22	Presenze con credenziali digitate	Orologio fisso al 02/11/2026; turno di oggi dalle 8 alle 16; due dipendenti con credenziali diverse.	Con credenziali corrette ingresso (9:30) e uscita (15:00) sono registrati e le ore lavorate sono 5,5; con password errata, con credenziali di un altro dipendente, senza turno di oggi o con credenziali nulle la presenza non viene registrata né salvata e l'autenticazione non viene interrogata se le credenziali sono nulle.
TC-23	Pagamento e controllo senza cassiere	Totem con servizio di pagamento simulato; sessione con controllo richiesto.	Senza cassiere il pagamento non viene addebitato e la sessione resta in PAGAMENTO; con il cassiere il controllo non parte e non segnala finché non viene eseguito con il carrello fisico pronto; una discrepanza porta a SEGNALATA solo dopo il controllo e registra la segnalazione; a fine controllo il cassiere risulta assente; dopo un pagamento con controllo non si può pagare di nuovo.
TC-24	Aggiornamento atomico delle quantità	Database di test con un prodotto da 10 pezzi a scaffale e 20 in magazzino; repository simulato per la sessione.	Il decremento non porta mai la quantità sotto zero e rifiuta codici inesistenti e quantità non positive; due repository indipendenti sommano i decrementi (10 − 3 − 4 = 3); lo spostamento non supera la merce in magazzino; la sessione di spesa

			aggiorna lo scaffale con una scrittura atomica per ogni prodotto e mai con modificaProdotto; il magazziniere salva il rifornimento tramite repository e, se impossibile, non lo tocca.
TC-25	Soglie e parametri di magazzino	Prodotti con quantità attorno alla soglia di 5 pezzi.	I parametri valgono 5, 20 e 100; il gestore delle scorte notifica solo sotto la soglia; ScortaMagazzino, magazziniere e responsabile usano la stessa soglia.

5.1.2.1 Tracciabilità tra requisiti e test

Requisito	**Test**
RF-C1 Registrazione e autenticazione cliente	TC-01, TC-02
RF-C2 Punti, storico e segnalazioni	TC-12
RF-C3 Carrello e sconti	TC-04
RF-C4 Pagamento, premio e punti	TC-05, TC-14, TC-15, TC-17, TC-23
RF-C5 Controllo della spesa	TC-05, TC-16, TC-19, TC-23
RF-D1 Registrazione dipendente	TC-03
RF-D2 Autenticazione e presenza	TC-08, TC-22
RF-D3, RF-D4 Assegnazione e vincoli sui turni	TC-07, TC-21
RF-D5 Calcolo degli stipendi	TC-11
RF-D6 Visualizzazione dei turni	TC-07
RF-D7, RF-D8 Prodotti e scorte	TC-06, TC-09, TC-24, TC-25
RF-D8, RF-D9 Ordini e ricezione	TC-10, TC-18
RF-D10 Controllo self-scanner	TC-05, TC-16, TC-20, TC-23

5.1.3 Risultati dell'esecuzione

Indicatore	**Risultato**
Test eseguiti	266
Test superati	266
Failures	0
Errors	0

L'esecuzione completa della suite si è conclusa con 266 test eseguiti e tutti superati, senza errori né fallimenti.

5.1.4 Copertura dei test

La copertura è misurata con EclEmma. Il valore complessivo del progetto (85,7% sulle istruzioni, 86,0% sulle righe, 51,1% sui rami) include il codice di test, che è coperto quasi al 100% per costruzione; per valutare la qualità della suite si considera quindi il codice di produzione (src/main/java).

5.1.4.1 Copertura del codice di produzione

Copertura delle istruzioni

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
Supermercato	85,7 %	18.888	3.161	22.049
src/main/java	60,8 %	4.790	3.093	7.883
unibg.it.Supermercato	0,0 %	0	2.492	2.492
unibg.it.Supermercato.gestionedipendenti	98,2 %	330	6	336
unibg.it.Supermercato.gestionefedelta	100,0 %	56	0	56
unibg.it.Supermercato.gestionemagazzino	96,8 %	539	18	557
unibg.it.Supermercato.pagamentiesterni	100,0 %	35	0	35
unibg.it.Supermercato.persistenzadati	83,3 %	1.882	376	2.258
unibg.it.Supermercato.selfscanning	94,6 %	1.031	59	1.090
unibg.it.Supermercato.sistemaaccesso	86,6 %	917	142	1.059
src/test/java	99,5 %	14.098	68	14.166
unibg.it.Supermercato	99,4 %	3.272	20	3.292
unibg.it.Supermercato.gestionedipendenti	100,0 %	1.779	0	1.779
unibg.it.Supermercato.gestionefedelta	100,0 %	213	0	213
unibg.it.Supermercato.gestionemagazzino	99,1 %	1.153	11	1.164
unibg.it.Supermercato.pagamentiesterni	100,0 %	68	0	68
unibg.it.Supermercato.persistenzadati	99,7 %	2.502	7	2.509
unibg.it.Supermercato.selfscanning	99,1 %	3.194	30	3.224
unibg.it.Supermercato.sistemaaccesso	100,0 %	1.917	0	1.917

Copertura delle righe

Element	Coverage	Covered Lines	Missed Lines	Total Lines
Supermercato	86,0 %	5.948	968	6.916
src/main/java	56,5 %	1.228	946	2.174
unibg.it.Supermercato	0,0 %	0	755	755
unibg.it.Supermercato.gestionedipendenti	95,8 %	92	4	96
unibg.it.Supermercato.gestionefedelta	100,0 %	14	0	14
unibg.it.Supermercato.gestionemagazzino	94,0 %	156	10	166
unibg.it.Supermercato.pagamentiesterni	100,0 %	11	0	11
unibg.it.Supermercato.persistenzadati	79,8 %	435	110	545
unibg.it.Supermercato.selfscanning	91,8 %	256	23	279
unibg.it.Supermercato.sistemaaccesso	85,7 %	264	44	308
src/test/java	99,5 %	4.720	22	4.742
unibg.it.Supermercato	99,3 %	1.338	10	1.348
unibg.it.Supermercato.gestionedipendenti	100,0 %	465	0	465
unibg.it.Supermercato.gestionefedelta	100,0 %	52	0	52
unibg.it.Supermercato.gestionemagazzino	100,0 %	382	0	382
unibg.it.Supermercato.pagamentiesterni	100,0 %	23	0	23
unibg.it.Supermercato.persistenzadati	100,0 %	707	0	707
unibg.it.Supermercato.selfscanning	98,9 %	1.041	12	1.053
unibg.it.Supermercato.sistemaaccesso	100,0 %	712	0	712

Copertura dei rami

Element	Coverage	Covered Branches	Missed Branches	Total Branches
Supermercato	51,1 %	476	456	932
src/main/java	51,1 %	461	441	902
unibg.it.Supermercato	0,0 %	0	247	247
unibg.it.Supermercato.gestionedipendenti	75,8 %	50	16	66
unibg.it.Supermercato.gestionefedelta	100,0 %	10	0	10
unibg.it.Supermercato.gestionemagazzino	72,1 %	49	19	68
unibg.it.Supermercato.pagamentiesterni	100,0 %	4	0	4
unibg.it.Supermercato.persistenzadati	67,5 %	110	53	163
unibg.it.Supermercato.selfscanning	74,3 %	113	39	152
unibg.it.Supermercato.sistemaaccesso	65,1 %	125	67	192
src/test/java	50,0 %	15	15	30
unibg.it.Supermercato		0	0	0
unibg.it.Supermercato.gestionedipendenti		0	0	0
unibg.it.Supermercato.gestionefedelta		0	0	0
unibg.it.Supermercato.gestionemagazzino		0	0	0
unibg.it.Supermercato.pagamentiesterni		0	0	0
unibg.it.Supermercato.persistenzadati	55,6 %	10	8	18
unibg.it.Supermercato.selfscanning	41,7 %	5	7	12
unibg.it.Supermercato.sistemaaccesso		0	0	0

Le poche istruzioni non coperte nei test (68) riguardano metodi di supporto; i rami dei test (30 in totale, tutti nei package persistenzadati e selfscanning) non sono significativi per la qualità della suite.

5.1.4.2 Considerazioni

- La classe App, che contiene l'applicazione dimostrativa a riga di comando, non è coperta (0%): rappresenta circa il 32% delle istruzioni, il 35% delle righe e il 27% dei rami del codice di produzione e non contiene logica di dominio. Escludendola, la copertura di produzione sale all'88,9% delle istruzioni, all'86,5% delle righe e al 70,4% dei rami.
- I package con la logica di business più importante hanno copertura delle istruzioni superiore al 94% (gestionefedelta 100%, pagamentiesterni 100%, gestionedipendenti 98,2%, gestionemagazzino 96,8%, selfscanning 94,6%).
- I valori più bassi sono in persistenzadati (83,3% istruzioni, 79,8% righe, 67,5% rami) e in sistemaaccesso (86,6%, 85,7%, 65,1% sui rami): le parti non coperte comprendono verosimilmente i rami di gestione degli errori dei DAO (blocchi catch) e i casi con valori nulli o ruoli non previsti.
- La copertura dei rami è sempre inferiore a quella delle istruzioni, anche nei package ben coperti (gestionedipendenti 75,8%, selfscanning 74,3%, gestionemagazzino 72,1%): è coerente con i metodi con più punti di decisione indicati dall'analisi di complessità ciclomatica (ad esempio registraIngresso e registraUscita, elaboraOrdine, aggiornaPresenza, effettuaPagamento).

6 MAINTENANCE

6.1 Software Maintenance

6.1.1 Attività di refactoring

Alcune modifiche hanno avuto natura strutturale: hanno migliorato separazione delle responsabilità, modularità o testabilità senza costituire semplicemente l'aggiunta di una nuova funzionalità.

ID	Intervento strutturale	Necessità	Beneficio
R1	Eliminazione delle dipendenze cicliche tra package	JDepend segnalava cicli tra sei package; i cicli sono valutati negativamente per coesione e accoppiamento.	Grafo delle dipendenze aciclico, package più indipendenti e testabili.
R2	Rimozione della classe inutilizzata Segnalazione	L'analisi di copertura EclEmma mostrava la classe a 0% (codice morto).	Meno codice da mantenere e misura di copertura più rappresentativa.
R3	Centralizzazione dei parametri di magazzino nella classe ParametriScorte	La soglia minima (5) era definita in quattro classi (GestoreScorte, ScortaMagazzino, Magazziniere, ResponsabileDiReparto) e le quantità di rifornimento (20) e di ordine (100) erano valori fissi nel codice.	Un'unica definizione dei valori di business, modificabile in un solo punto e verificata da un test dedicato.
R4	Configuratore dei ruoli letti dal database (Consumer<Dipendente> in DBDipendenti, impostato dal composition root)	I ruoli creati da DBDipendenti non avevano i collaboratori (gestori di turni e stipendi, controllo della spesa, gestori di prodotti, scorte e ordini) e persistenzadati non poteva conoscerli senza creare cicli.	Collegamento dei collaboratori dal composition root, senza dipendenze di persistenzadati dal dominio; ruoli sempre completi dopo la lettura dal database.
R5	Orologio iniettabile in GestorePresenze e metodi di Turno che ricevono l'ora	Uso diretto dell'ora di sistema, che rendeva non deterministici i test su presenze e ore lavorate.	Test deterministici (orologio fisso) e possibilità di simulare orari diversi.

6.1.1.1 R1: eliminazione delle dipendenze cicliche tra package

Problema e rilevazione

L'analisi dei package con JDepend ha mostrato sei package contrassegnati come coinvolti in cicli: sistemaaccesso, selfscanning, persistenzadati, gestionemagazzino, gestionedipendenti e il package radice. I cicli sono valutati negativamente nei criteri del corso e rendono i package impossibili da isolare e da testare separatamente.

Ciclo	Causa
persistenzadati ↔ package di dominio	Le interfacce dei repository stavano in persistenzadati, mentre i DAO importavano classi di dominio (Cliente, SessioneSpesa, Turno, Prodotto); a loro volta le classi di dominio importavano persistenzadati
selfscanning ↔ sistemaaccesso	Totem (selfscanning) usava Cassiere, e Cassiere usava SessioneSpesa, CarrelloFisico, Segnalazione e IControlloSpesa di selfscanning
sistemaaccesso ↔ gestionedipendenti	Manager (sistemaaccesso) creava GestoreTurni e CalcoloStipendi, mentre le classi di gestionedipendenti usavano Dipendente e le interfacce dei turni e degli stipendi

Interventi

Intervento	Classi e interfacce coinvolte
Interfacce dei repository spostate nei package di dominio (Dependency Inversion Principle)	IRepositoryClienti e IRepositoryDipendenti in sistemaaccesso; IRepositoryProdotti e IRepositoryOrdini in gestionemagazzino; DBClienti, DBDipendenti, DBProdotti e DBOrdini restano in persistenzadati e implementano le interfacce
Firme dei repository con tipi del dominio o valori semplici	salvaStorico(clienteId, data, totale) al posto di un parametro SessioneSpesa; registraSegnalazione(clienteId, data); trovaSegnalazioniByCliente restituisce le date; AcquistoStorico spostata in sistemaaccesso
Porte del controllo della spesa in sistemaaccesso	ISessioneControllabile (implementata da SessioneSpesa), IControlloSpesa (implementata da ControlloSpesa) e ICarrelloFisico (implementata da CarrelloFisico); Cassiere dipende solo da queste interfacce
Interfacce e modello dei turni e degli stipendi in sistemaaccesso	ITurni, IStipendi, Turno, Presenza e Stipendio spostati in sistemaaccesso; GestoreTurni, GestorePresenze e CalcoloStipendi restano in gestionedipendenti e implementano le interfacce
Iniezione delle dipendenze	Manager riceve ITurni e IStipendi da costruttore o setter; GestoreProdotti e GestoreOrdini ricevono il repository nel costruttore; DBDipendenti riceve IRepositoryClienti; il collegamento delle implementazioni è affidato alla classe di avvio (composition root)
Pulizia	Rimossi gli import inutilizzati tra package

Verifica e risultati

Dopo il refactoring l'elenco dei package con cicli di JDepend è vuoto e la suite di test viene eseguita senza errori né fallimenti. La tabella confronta le metriche prima del refactoring con quelle dell'ultima analisi JDepend, che comprendono anche gli interventi successivi (rimozione di Segnalazione, ParametriScorte, EStatoOrdine).

Package	CC prima → dopo	AC prima → dopo	Ca prima → dopo	Ce prima → dopo	I prima → dopo	Cicli prima → dopo
sistemaaccesso	6 → 10	4 → 11	4 → 4	4 → 1	0,50 → 0,20	sì → no
selfscanning	14 → 12	2 → 1	3 → 1	5 → 4	0,62 → 0,80	sì → no
persistenzadati	5 → 5	4 → 0	5 → 1	4 → 2	0,44 → 0,66	sì → no
gestionemagazzino	9 → 11	4 → 6	4 → 4	1 → 0	0,20 → 0,00	sì → no
gestionedipendenti	6 → 3	3 → 1	3 → 1	2 → 1	0,40 → 0,50	sì → no
pagamentiesterni	2 → 2	1 → 1	2 → 2	0 → 0	0,00 → 0,00	no → no
gestionefedelta	2 → 2	0 → 0	2 → 2	0 → 0	0,00 → 0,00	no → no
radice (App)	1 → 1	0 → 0	0 → 0	8 → 8	1,00 → 1,00	sì → no

6.1.1.2 R2: rimozione della classe Segnalazione

Problema e rilevazione

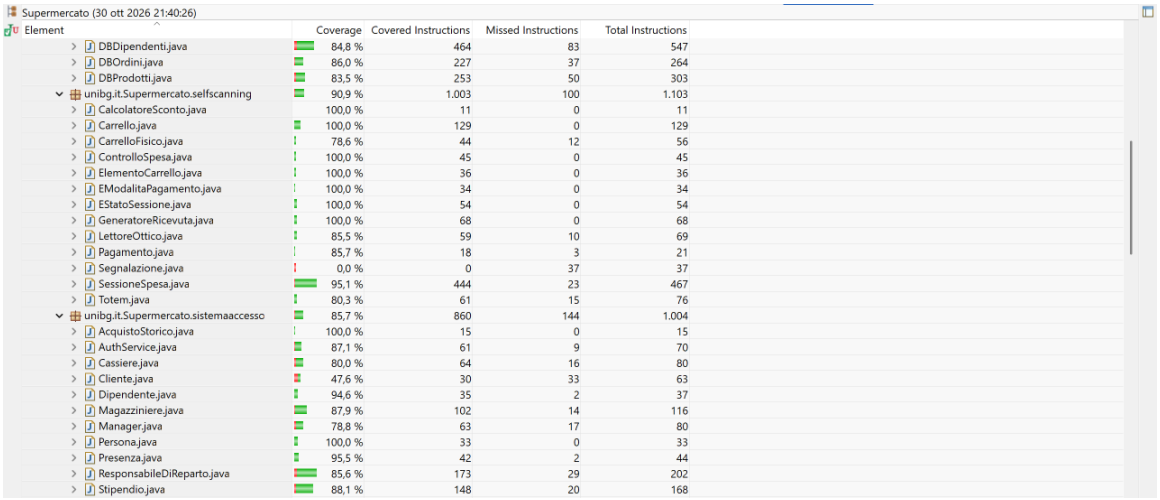
Dopo il refactoring R1 il Cassiere registra la segnalazione di un cliente chiamando IRepositoryClienti.registraSegnalazione(clienteId, data) e aggiornando lo stato del cliente, senza più creare un oggetto Segnalazione. L'analisi di copertura con EclEmma ha mostrato che la classe Segnalazione del package selfscanning non veniva eseguita da nessun test: copertura dello 0,0% (0 istruzioni coperte su 37). Poiché la classe non era usata dal codice di produzione, è stata classificata come codice morto.

Intervento

- Eliminazione della classe Segnalazione dal package selfscanning e dei riferimenti residui nel codice (ad esempio nella classe di avvio dimostrativa).
- Nessuna modifica alla persistenza: le segnalazioni restano nella tabella segnalazioni e nel contrassegno del cliente (segnalato), gestiti da DBClienti.
- Verifica della compilazione e dell'esecuzione della suite di test.

Risultati

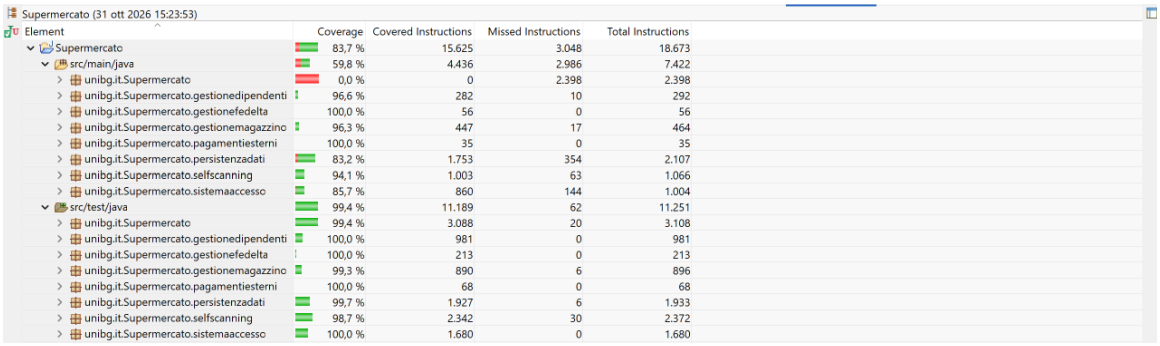
Prima



Supermercato (30 ott 2026 21:40:26)

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
DBDipendenti.java	84,8 %	464	83	547
DBOrdini.java	86,0 %	227	37	264
DBProdotti.java	83,5 %	253	50	303
unibg.it.Supermercato.selfscanning	90,9 %	1.003	100	1.103
CalcolatoreSconto.java	100,0 %	11	0	11
Carrello.java	100,0 %	129	0	129
CarrelloFisico.java	78,6 %	44	12	56
ControlloSpesa.java	100,0 %	45	0	45
ElementoCarrello.java	100,0 %	36	0	36
EModalitaPagamento.java	100,0 %	34	0	34
EStatoSessione.java	100,0 %	54	0	54
GeneratoreRicevuta.java	100,0 %	68	0	68
LettoreOttico.java	85,5 %	59	10	69
Pagamento.java	85,7 %	18	3	21
Segnalazione.java	0,0 %	0	37	37
SessioneSpesa.java	95,1 %	444	23	467
Totem.java	80,3 %	61	15	76
unibg.it.Supermercato.sistemaaccesso	85,7 %	860	144	1.004
AcquistoStorico.java	100,0 %	15	0	15
AuthService.java	87,1 %	61	9	70
Cassiere.java	80,0 %	64	16	80
Cliente.java	47,6 %	30	33	63
Dipendente.java	94,6 %	35	2	37
Magazziniere.java	87,9 %	102	14	116
Manager.java	78,8 %	63	17	80
Persona.java	100,0 %	33	0	33
Presenza.java	95,5 %	42	2	44
ResponsabileDiReparto.java	85,6 %	173	29	202
Stipendio.java	88,1 %	148	20	168

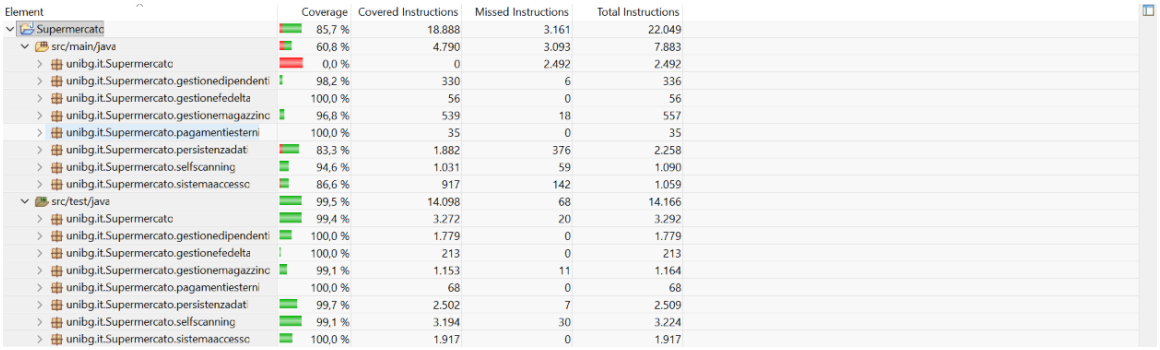
Dopo



Supermercato (31 ott 2026 15:23:53)

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
Supermercato	83,7 %	15.625	3.048	18.673
src/main/java	59,8 %	4.436	2.986	7.422
unibg.it.Supermercato	0,0 %	0	2.398	2.398
unibg.it.Supermercato.gestionedipendenti	96,6 %	282	10	292
unibg.it.Supermercato.gestionefedelta	100,0 %	56	0	56
unibg.it.Supermercato.gestionemagazzino	96,3 %	447	17	464
unibg.it.Supermercato.pagamentiesterni	100,0 %	35	0	35
unibg.it.Supermercato.persistenzadati	83,2 %	1.753	354	2.107
unibg.it.Supermercato.selfscanning	94,1 %	1.003	63	1.066
unibg.it.Supermercato.sistemaaccesso	85,7 %	860	144	1.004
src/test/java	99,4 %	11.189	62	11.251
unibg.it.Supermercato	99,4 %	3.088	20	3.108
unibg.it.Supermercato.gestionedipendenti	100,0 %	981	0	981
unibg.it.Supermercato.gestionefedelta	100,0 %	213	0	213
unibg.it.Supermercato.gestionemagazzino	99,3 %	890	6	896
unibg.it.Supermercato.pagamentiesterni	100,0 %	68	0	68
unibg.it.Supermercato.persistenzadati	99,7 %	1.927	6	1.933
unibg.it.Supermercato.selfscanning	98,7 %	2.342	30	2.372
unibg.it.Supermercato.sistemaaccesso	100,0 %	1.680	0	1.680

Ultima analisi



Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
Supermercato	85,7 %	18.888	3.161	22.049
src/main/java	60,8 %	4.790	3.093	7.883
unibg.it.Supermercato	0,0 %	0	2.492	2.492
unibg.it.Supermercato.gestionedipendenti	98,2 %	330	6	336
unibg.it.Supermercato.gestionefedelta	100,0 %	56	0	56
unibg.it.Supermercato.gestionemagazzino	96,8 %	539	18	557
unibg.it.Supermercato.pagamentiesterni	100,0 %	35	0	35
unibg.it.Supermercato.persistenzadati	83,3 %	1.882	376	2.258
unibg.it.Supermercato.selfscanning	94,6 %	1.031	59	1.090
unibg.it.Supermercato.sistemaaccesso	86,6 %	917	142	1.059
src/test/java	99,5 %	14.098	68	14.166
unibg.it.Supermercato	99,4 %	3.272	20	3.292
unibg.it.Supermercato.gestionedipendenti	100,0 %	1.779	0	1.779
unibg.it.Supermercato.gestionefedelta	100,0 %	213	0	213
unibg.it.Supermercato.gestionemagazzino	99,1 %	1.153	11	1.164
unibg.it.Supermercato.pagamentiesterni	100,0 %	68	0	68
unibg.it.Supermercato.persistenzadati	99,7 %	2.502	7	2.509
unibg.it.Supermercato.selfscanning	99,1 %	3.194	30	3.224
unibg.it.Supermercato.sistemaaccesso	100,0 %	1.917	0	1.917

La rimozione elimina 37 istruzioni non coperte e innalza la copertura delle istruzioni del package di 3,2 punti percentuali senza cambiare le istruzioni coperte. Il comportamento del sistema non cambia e la suite continua a passare. L'ultima colonna riporta la misura più recente, successiva anche all'aggiunta di nuovi test e classi.

6.1.1.3 R3: centralizzazione dei parametri di magazzino

La soglia minima di scorta (5 pezzi) era dichiarata in quattro classi e le quantità di rifornimento (20) e di ordine standard (100) comparivano come valori fissi. Tutti i valori sono stati raccolti nella classe finale ParametriScorte (SOGLIA_MINIMA, QUANTITA_RIFORNIMENTO, QUANTITA_ORDINE_STANDARD), usata da GestoreScorte, ScortaMagazzino, Magazziniere e ResponsabileDiReparto. Il comportamento non cambia; ParametriScorteTest verifica che i valori siano quelli previsti e che i quattro componenti applichino la stessa soglia. Resta da derivare in Promozione il tetto di 600 punti dal tetto di sconto.

6.1.1.4 R4: configuratore dei ruoli letti dal database

DBDipendenti crea Manager, Cassiere, Magazziniere e ResponsabileDiReparto a partire dalle righe del database, ma non può conoscere i loro collaboratori senza dipendere da selfscanning e gestionedipendenti, reintroducendo i cicli eliminati in R1. È stato aggiunto il metodo setConfiguratoreRuoli(Consumer<Dipendente>): il composition root (classe App) fornisce una funzione che collega a ciascun ruolo i suoi collaboratori (GestoreTurni e CalcoloStipendi al Manager, ControlloSpesa al Cassiere, gestori di prodotti, scorte e ordini a magazziniere e responsabile), registrando gli osservatori delle scorte una sola volta per dipendente. DBDipendentiConfiguratoreTest verifica che il configuratore sia invocato per ogni ruolo e che i collaboratori vengano effettivamente usati.

6.1.1.5 R5: orologio iniettabile

GestorePresenze riceve un Clock nel costruttore (con orologio di sistema come valore predefinito) e Turno offre i metodi registraInizio(ora) e registraFine(ora) che ricevono l'ora. I test usano un orologio fisso per verificare ingresso, uscita e ore lavorate senza dipendere dall'ora di esecuzione. L'intervento non è esteso a Pagamento e OrdineFornitore, che usano ancora now().

6.1.2. Reverse engineering

Nel nostro progetto non è stato utilizzato codice preesistente come base del sistema; pertanto, il reverse engineering non costituisce un'attività svolta.

BIBLIOGRAFIA

Libri di testo

- van Vliet, H. Software Engineering: Principles and Practice, 3ª ed. Wiley, 2008.
- Seidl, M., Scholz, M., Huemer, C., Kappel, G. UML @ Classroom: An Introduction to Object-Oriented Modeling. Springer, 2015.

Standard e riferimenti normativi citati nei documenti

- IEEE 830 - Recommended Practice for Software Requirements Specifications.
- ISO/IEC 9126 - Software engineering - Product quality.
- Oracle. Java Code Conventions.

Strumenti

Eclipse, Maven, Papyrus Designer, PlantUML, JUnit 5, Mockito, Log4j 2, H2, PMD, JDepend, SonarQube, EclEmma, GitHub.